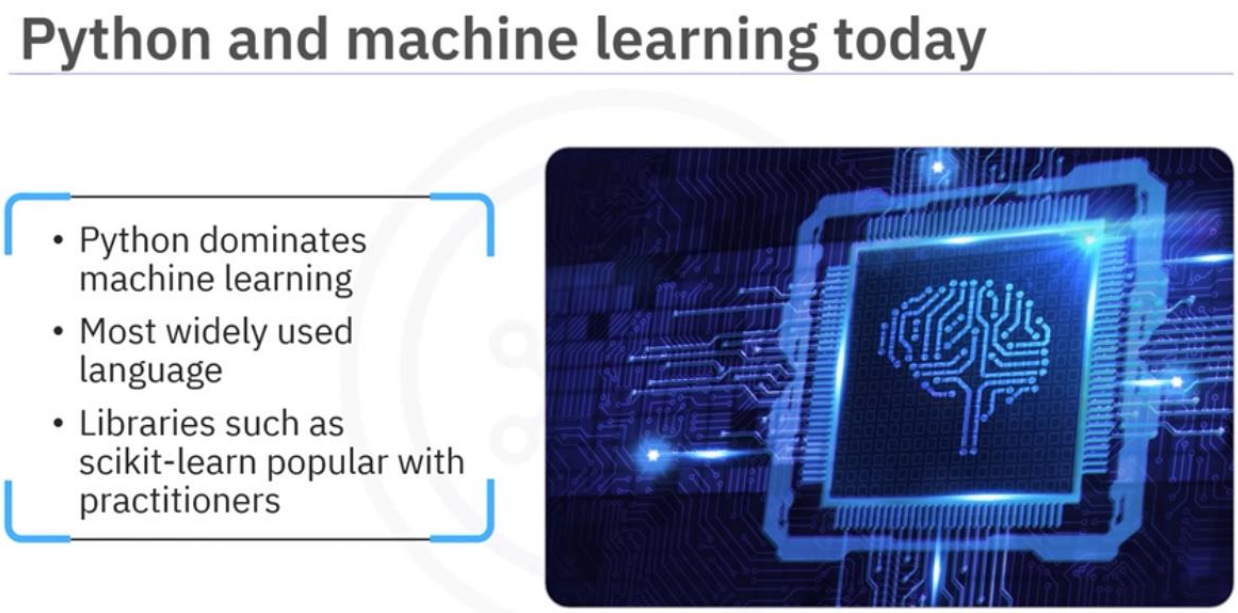
Module 1 Introduction to Machine Learning

In this module, you will explore foundational machine learning concepts that prepare you for hands-on modeling with Python. You will explain the relevance of Python and scikit-learn in machine learning, summarize the IBM AI Engineering certification path, and classify common types of learning algorithms. You'll outline the stages of the machine learning model lifecycle and describe what a typical day looks like for a machine learning engineer. You will also compare key roles in the AI field, identify widely used open-source tools, and learn to utilize scikit-learn to build and evaluate simple models.

Course Introduction

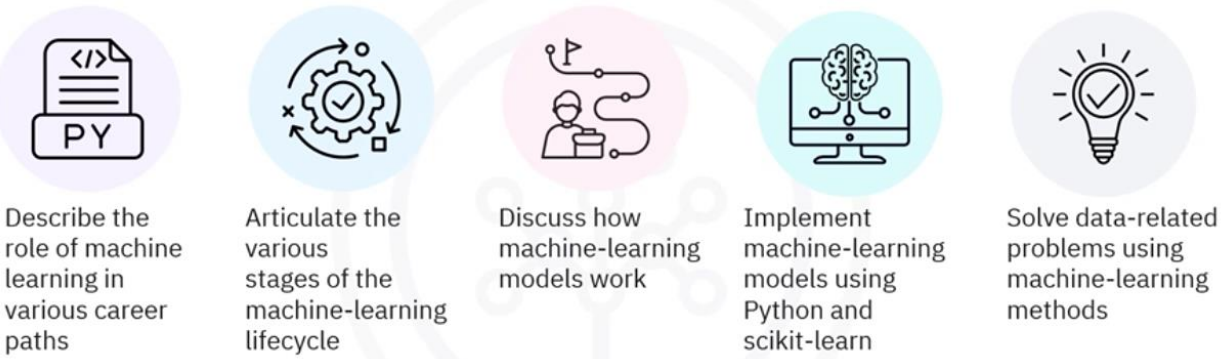
You will learn about **applied machine learning using Python tools**.



This **intermediate-level course** will provide you with a solid foundation in:

- Applied machine learning
- data science.
- AI.

To gain maximum learning from this course, **a working knowledge of** Python, Pandas, NumPy, data preparation, data analysis with Python and familiarity with the field of machine learning **is recommended**.



In this course, you will become aware of the role of machine learning in various career paths.

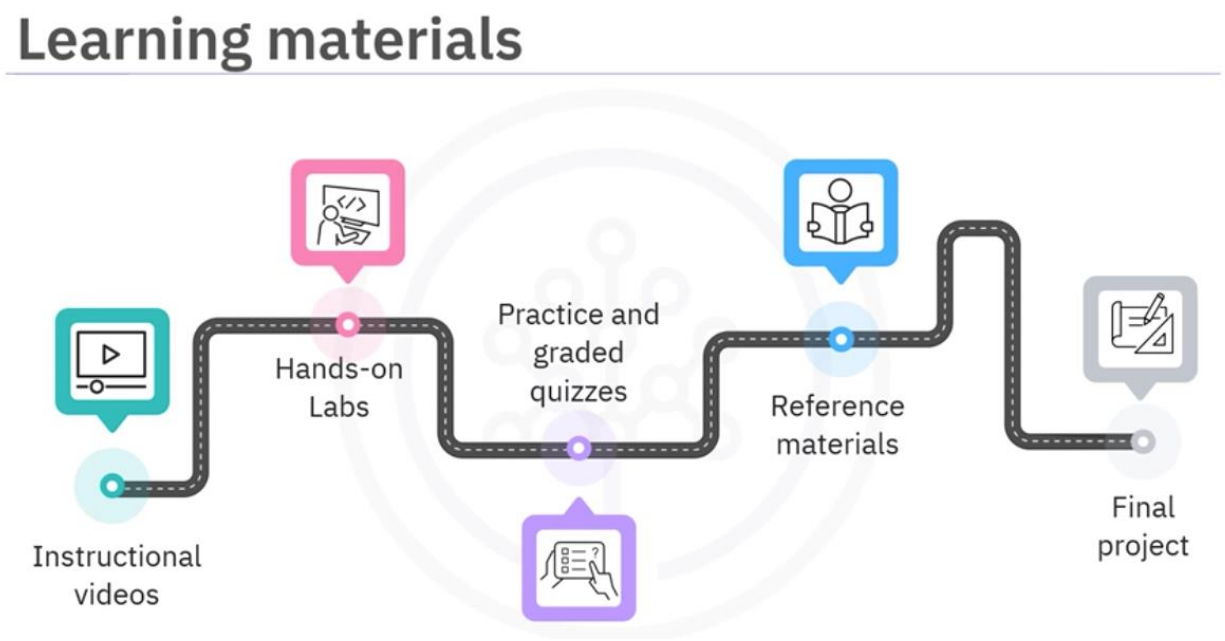
You will gain a foundational knowledge of the machine learning lifecycle and how machine learning models work.

Throughout the course, you will focus on machine learning modeling techniques such as **classification**, **regression**, and **clustering**.

Using real-world data, you will learn how these models fit within the broader **supervised** and **unsupervised learning** frameworks.

You'll also get a brief introduction to advanced methods such as **reinforcement learning, deep learning**, and the world of artificial intelligence.

You will gain **hands-on experience** with building, assessing, and validating various classification, regression, and clustering machine learning models using Python and popular open-source libraries such as Pandas, NumPy, and Scikit-Learn.



You will gain a comprehensive understanding of machine learning using Python through instructional **videos** and then practice the concepts learned in **hands-on labs**.You will assess your learning with practice and graded **quizzes** and strengthen the mastery of concepts with **reference materials** such as a **glossary**. A concluding **final project** in the last module will provide you with the opportunity to apply your knowledge.

Each lesson has one or more labs that reinforce the instructional videos by applying what you've learned in each lesson. Our topics include Multiple Linear Regression and Logistic Regression, Prediction, Fraud Detection, KNN, and SVM.

Course Overview

This course is a part of both the IBM AI Engineering Professional Certificate and the IBM Data Science Professional Certificate. If you're enrolled in the Data Science PC, you might consider continuing with the AI Engineering PC after completion to deepen your expertise. Follow the link to explore these programs and see how they can benefit your career as a machine learning professional!

Course Objectives:

After completing this course, you will be able to:

- Describe how machine learning plays a pivotal role in various career paths

- Articulate the various **stages involved in the machine learning lifecycle**
- Discuss how various machine learning **models work**
- Implement machine learning models using Python and scikit-learn
- Solve data-related problems using machine-learning methods

Course Outline:

This course has 6 modules, listed below. We encourage you to set aside several hours each week to successfully complete all modules in 4-6 weeks. Consistency will best help you achieve your learning goals!

You will benefit most from viewing all the videos and readings and solidifying that knowledge by completing all the hands-on labs and the final culminating project.

Module 1: Introduction to Machine Learning

This module provides you with knowledge of **foundational** machine learning **concepts** to delve deeper into applied machine learning modeling. You will learn that machine learning modeling is an **iterative** process with various **lifecycle stages**. You will also learn about the daily activities of a machine-learning engineer. Here, you will be introduced to various open-source **tools** for machine learning, including the popular Python package *scikit-learn*.

Module 2: Linear and Logistic Regression

This module introduces you to two classical statistical methods foundational to machine learning: linear and logistic regression. You'll learn how linear regression, **pioneered in the 1800s**, models linear relationships while **logistic regression serves as a classifier**. Through implementing these models, you'll understand their **limitations** and gain insight into why modern machine-learning models are often preferred.

Module 3: Building Supervised Learning Models

In this module, you'll learn about implementing **modern supervised machine learning models**. You will start by understanding how **binary classification** works and discover how to construct a multiclass classifier from binary classification components. You'll learn what **decision trees** are, how they learn, and how to build them. Decision trees, which are used to solve classification problems, have a natural extension called ***regression trees,*** which can handle regression problems. You'll learn about other supervised learning models, like **KNN and SVM**. You'll learn what **bias** and **variance** are in **model fitting** and the tradeoff between bias and variance that is inherent to all learning models in various degrees. You'll

learn strategies for mitigating this tradeoff and work with models that do a very good job accomplishing that goal.

Module 4: Building Unsupervised Learning Models

In this module, you'll dive into unsupervised learning, where **algorithms uncover patterns** in data without labeled examples. You'll explore **clustering strategies** and real-world applications, focusing on techniques like hierarchical clustering, k-means, and advanced methods such as DBSCAN and HDBSCAN. Through practical labs, you'll gain a deeper understanding of how to **compare and implement these algorithms effectively**. Additionally, you'll delve into **dimension reduction algorithms** like PCA (Principal Component Analysis), t-SNE, and UMAP to reduce dataset features and simplify other modeling tasks. Using Python, you'll implement these clustering and dimensionality reduction techniques, learning how to integrate them with **feature engineering** to prepare data for machine learning models.

Module 5: Evaluating and Validating Machine Learning Models

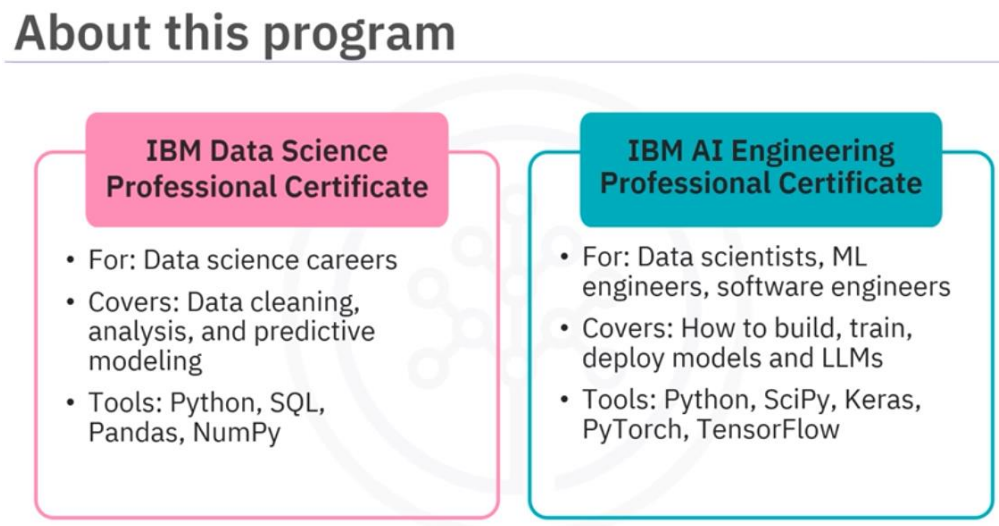
This module covers how to assess model performance on unseen data, starting with key evaluation metrics for classification and regression. You'll also explore **hyperparameter tuning** to optimize models while **avoiding overfitting** using cross-validation. Special techniques, such as regularization in linear regression, will be introduced to handle overfitting due to outliers. Hands-on exercises in Python will guide you through model fine-tuning and cross-validation for reliable model evaluation.

Module 6: Final Exam and Project

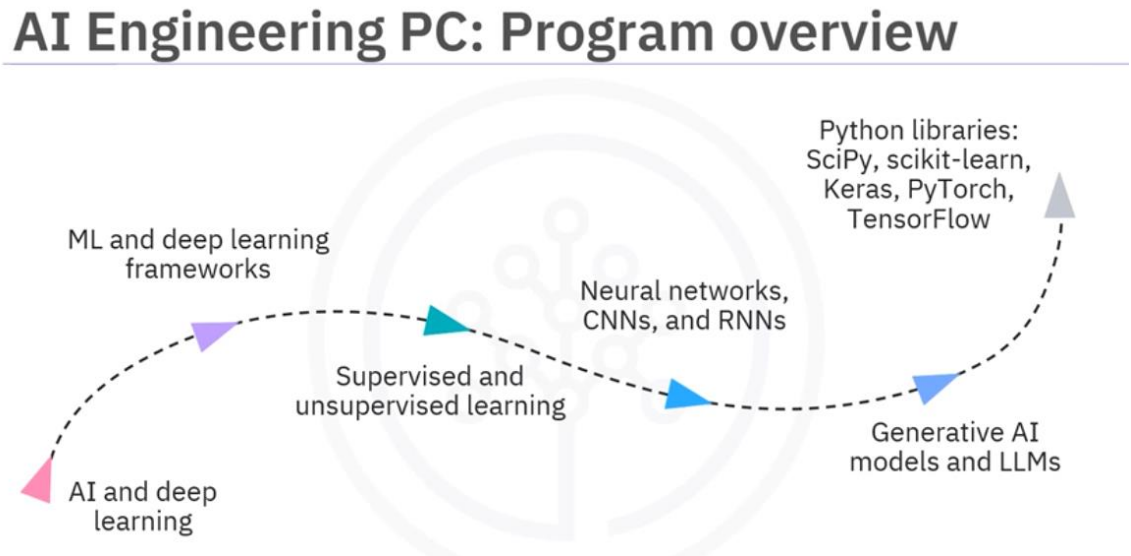
In this concluding module, you'll review the course content, complete a final exam, and work on a hands-on project. You'll receive a course summary cheat sheet, apply your skills in a project on Rain Prediction in Australia, and participate in peer reviews to share feedback. The module wraps up with guidance on next steps in your learning journey.

IBM AI Engineering PC Overview

Welcome to the IBM Professional Certificate (PC) Overview video, which introduces the IBM AI Engineering PC and the IBM Data Science PC. This course is a part of both.



- The Data Science PC **focuses** on skills for **data science careers**, covering data cleaning, analysis, and predictive modeling with tools such as Python, SQL, Pandas, and NumPy to help you build a job-ready portfolio. The IBM Data Science PC is suited for beginners, has **no prerequisites**, and is designed for anyone interested in data analysis, ML, and statistical modeling.
- The AI Engineering PC **prepares** data scientists, machine learning engineers, and software engineers **for AI engineering**. You'll learn to build, train, and deploy models, including large language models (LLMs), using Python and libraries like SciPy, Keras, PyTorch, and TensorFlow. The IBM AI Engineering PC is ideal for learners with knowledge of Python, data analysis, and generative AI basics, as well as experience with data science, machine learning (ML), or software engineering.

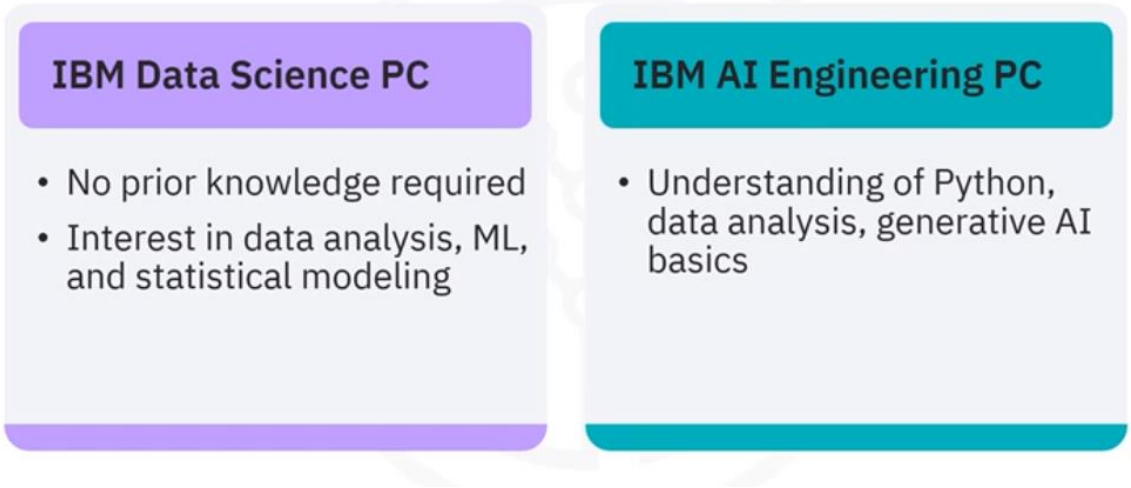


The IBM AI Engineering PC covers topics and skills needed for an entry-level AI engineering role, including Introduction to AI and Deep Learning, ML and Deep Learning Frameworks, Supervised and Unsupervised Learning, Neural Networks, Convolutional Networks, and Recurrent Networks, Generative AI Models and LLMs, and working with Python libraries like SciPy, Scikit-learn, Keras, PyTorch, and TensorFlow.

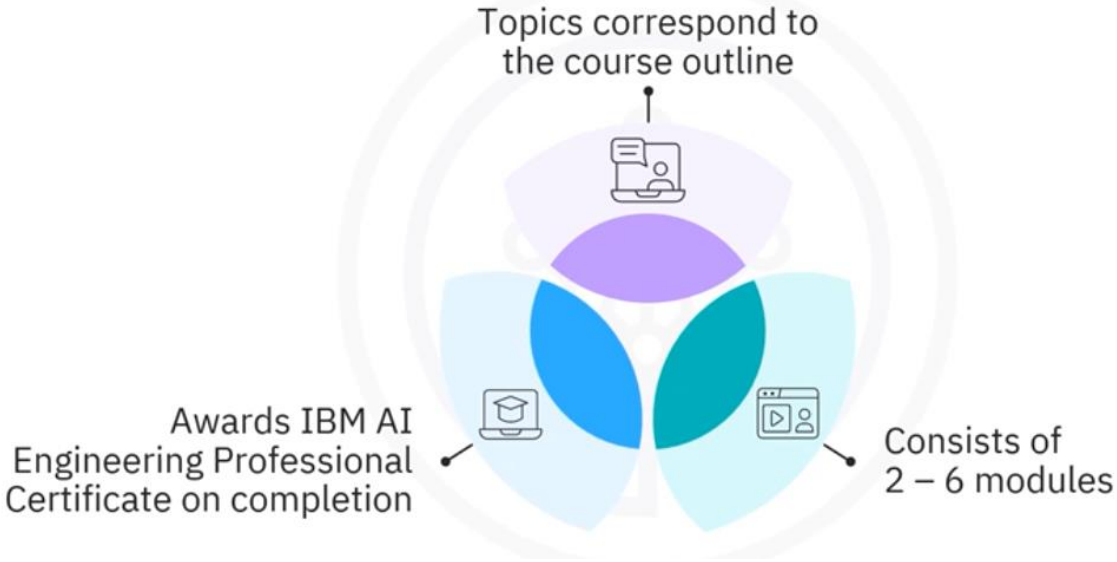
-

If you're completing the IBM Data Science PC, the AI Engineering PC is an excellent next step to expand your AI expertise. Both programs **emphasize hands-on** learning with projects and labs, building real-world experience. These certifications with comprehensive curricula prepare you **for a career in AI or data science**.

Who can take the program?



Program overview



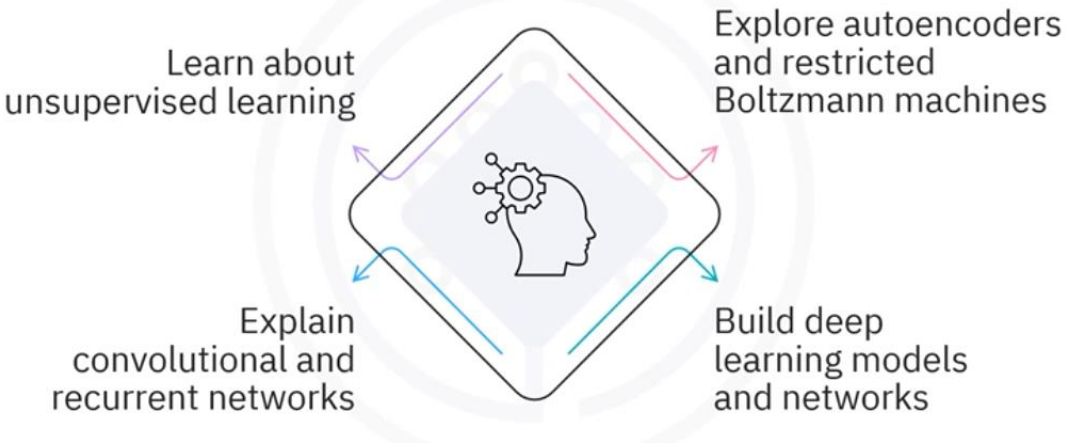
Each topic corresponds to a self-paced online course comprising 2-6 modules.

Completing these courses and the required projects will help you get the IBM AI Engineering Professional Certificate.

This Professional Certificate journey will introduce you to the fundamentals of machine learning using Python. You'll learn how to implement algorithms such as **linear regression, logistic regression, decision trees**, and more to build **predictive models**.

By the end, you'll understand the key concepts of **supervised learning** and be able to apply these models to **solve real-world problems**. **Deep learning basics and neural network architecture** are essential for any AI professional.

Deep learning & neural networks with Keras



You will study **unsupervised deep learning models** such as **autoencoders** and restricted **Boltzmann machines** while also learning about **convolutional and recurrent networks** and building deep learning models and networks using the Keras library.

Explore advanced deep learning techniques with Keras and TensorFlow 2.x, where you'll create custom layers and models and integrate Keras with TensorFlow for enhanced functionality. Develop **convolutional neural networks** (CNNs) and **transformer** models for sequential **and time series** data.

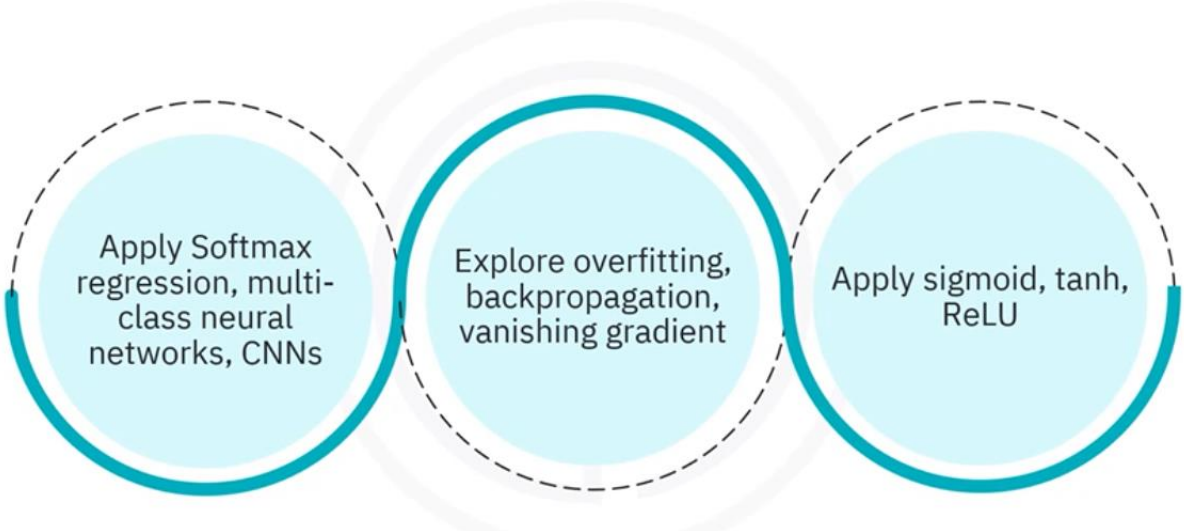
Deep learning with Keras and TensorFlow

The course also delves into unsupervised learning for model optimization, **custom training loops**, deep Q-networks (DQNs) for reinforcement learning, and an introduction to **generative modeling** and **reinforcement learning** principles.

Neural networks and PyTorch

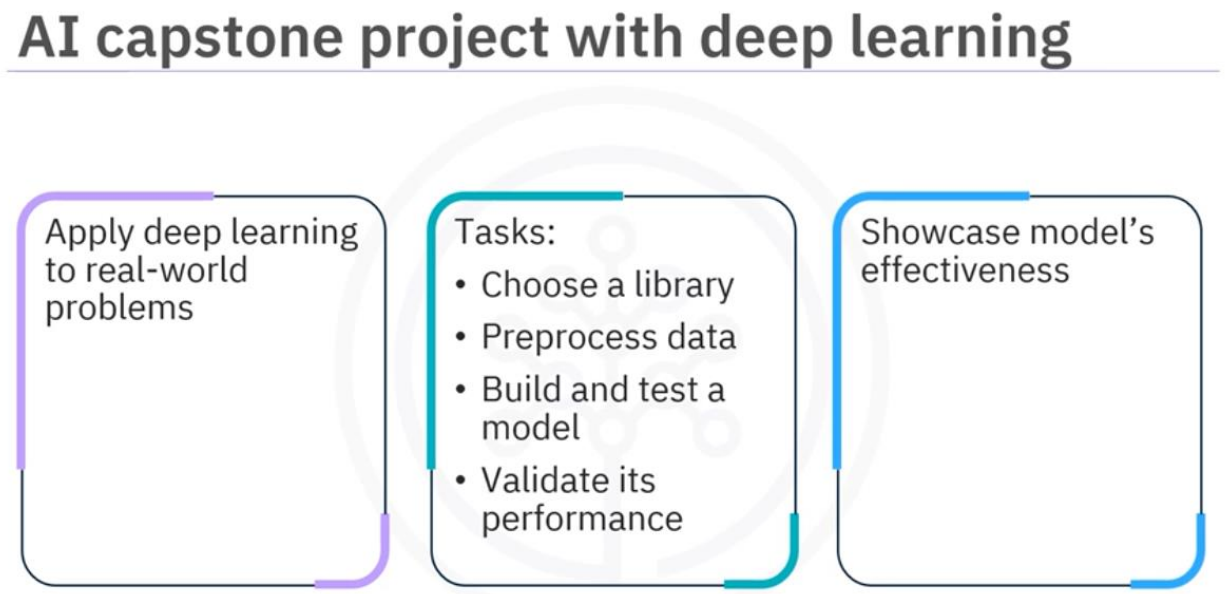
Develop essential skills in PyTorch for designing, training, and **optimizing neural networks**. You'll explore **2D tensors**, **derivatives**, linear regression, and **loss calculation**, along with **batch processing**, **gradient descent**, and logistic regression, key techniques for building models in areas like **image recognition** and **predictive analytics**.

Deep learning with PyTorch

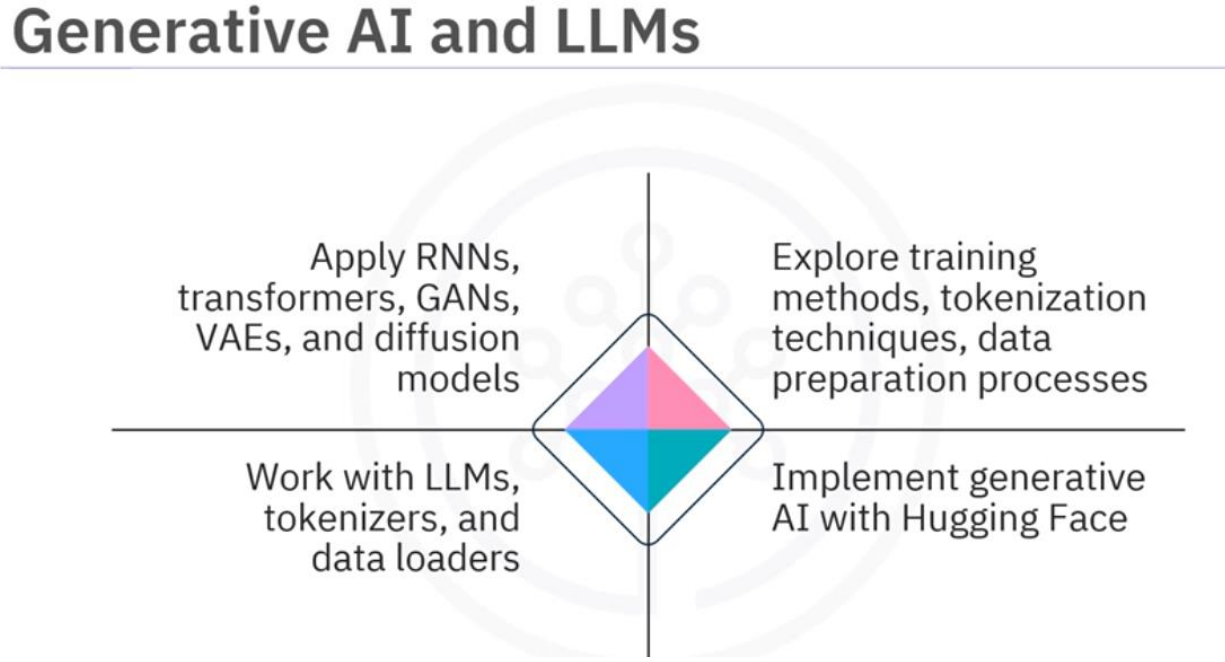


Next, you will advance to complex deep learning techniques with PyTorch, covering **Softmax regression**, multi-class neural networks, and specialized architectures such as CNNs.

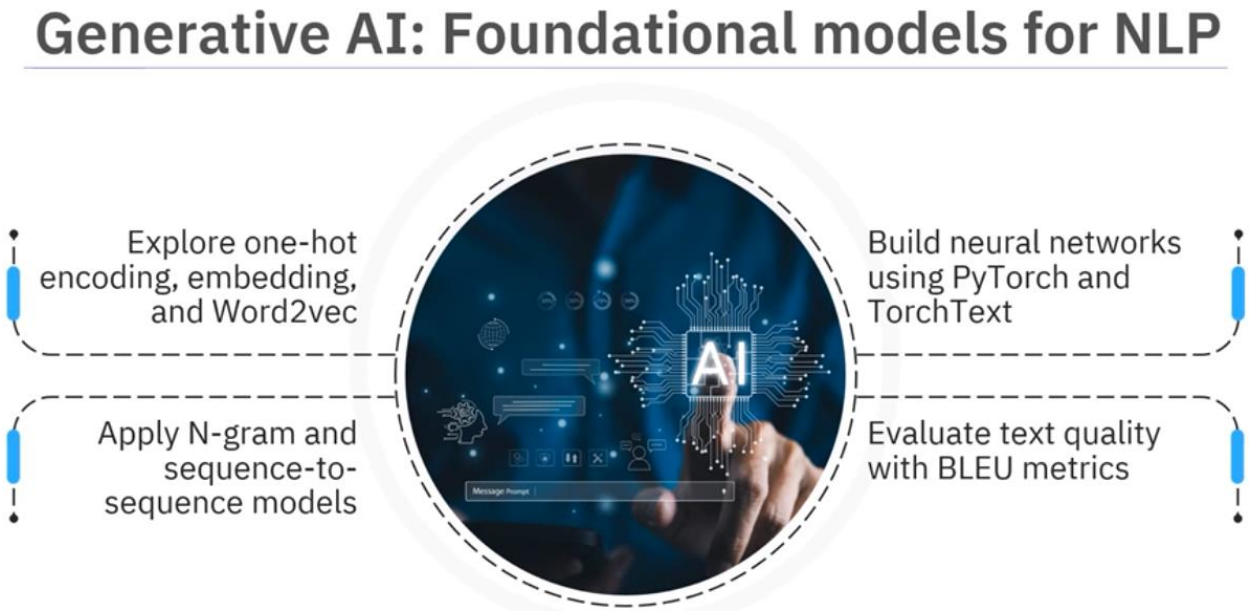
You'll tackle **overfitting**, **backpropagation**, and **vanishing gradient**, and apply **activation functions** such as sigmoid, tanh, and ReLU.



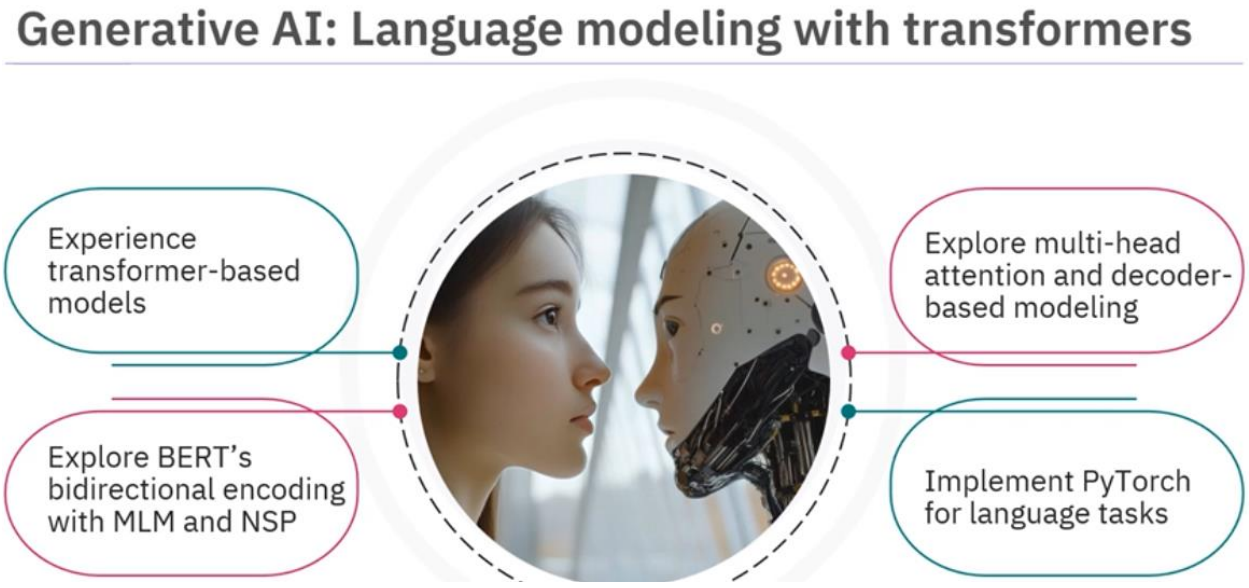
Hands-on projects and exercises will build expertise in creating robust models for real-world applications. The **capstone project** challenges you to apply deep learning expertise to a real-world problem. Choose a library, preprocess data, build and test a model, and validate its performance. A project report will showcase your model's effectiveness and your deep learning proficiency **for your portfolio**.



Next, you will explore the types and applications **of generative AI models**, including RNNs, transformers, GANs, VAEs, and diffusion models. Understand key differences in training methods, **tokenization techniques**, and data preparation processes. Gain skills to work with **large-language models**, LLMs, like GPT and BERT. And learn to use tokenizers and data loaders, preparing you for implementing generative AI with libraries like Hugging Face.

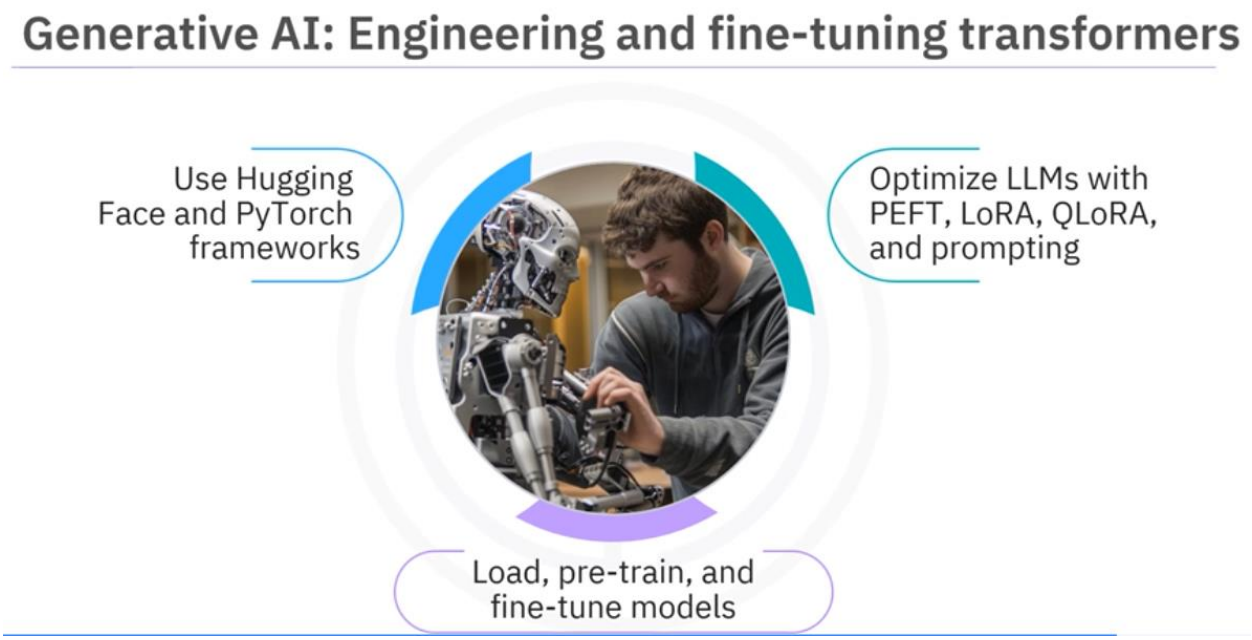


Delve into foundational NLP models, exploring **one-hot encoding**, embeddings, and Word2Vec for **text representation**. Build and optimize neural networks for tasks like **document categorization** and text generation using PyTorch and TorchText. Apply N-gram and sequence-to-sequence models for applications such as **language translation** and **evaluate generated text quality** with BLEU metrics.



Gain expertise in **transformer-based models** like GPT and BERT, focusing on **text classification**, **positional encoding**, word embeddings, and **attention mechanisms for contextual understanding**.

Dive into **multi-head attention** and apply GPT for language translation using decoder-based modeling. Explore BERT's **bidirectional encoding** with techniques like **Massed Language Modeling** (MLM) and **Next Sentence Prediction** (NSP), and implement these models in PyTorch for sophisticated language tasks.



Explore **advanced fine-tuning techniques** for transformers using frameworks like Hugging Face and PyTorch. Learn to optimize large-language models (LLMs) with methods such as Parameter-Efficient Fine-Tuning (PEFT), Low-Rank Adaptation (LORA), Quantized Low-Rank Adaptation (QLORA), and Prompting.

Gain hands-on experience in online labs, including loading, pre-training, and fine-tuning models to enhance performance for specific tasks.

Advanced fine-tuning for LLMs

Refine your skills in **fine-tuning LLMs with human feedback** and direct preference. Explore instruction tuning with Hugging Face, reward modeling, and **reinforcement learning techniques** like RLHF and PPO.

Gain hands-on experience in online labs focused on **reward modeling**, PPO, and DPO to optimize LLM performance for task-specific applications.

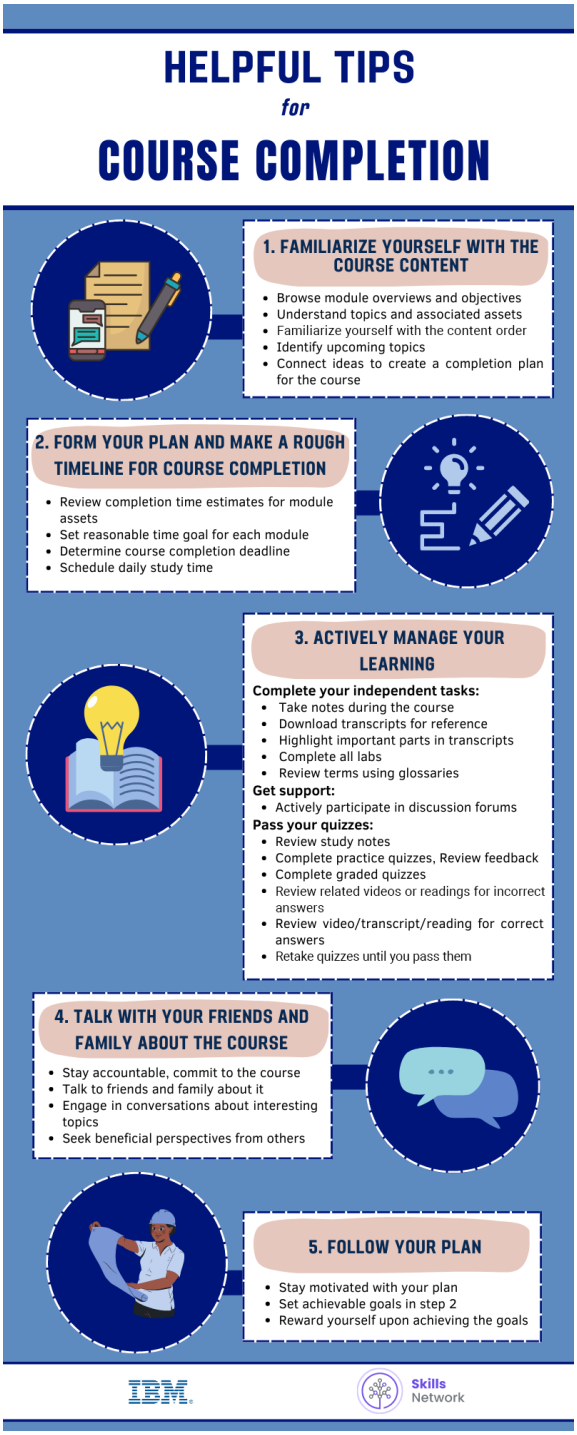
Fundamentals of AI agents using RAG and LangChain

Explore **Retrieval Augmented Generation** (RAG), **Prompt Engineering**, and LangChain concepts. Learn how RAG integrates external data and tokenizers for dynamic responses.

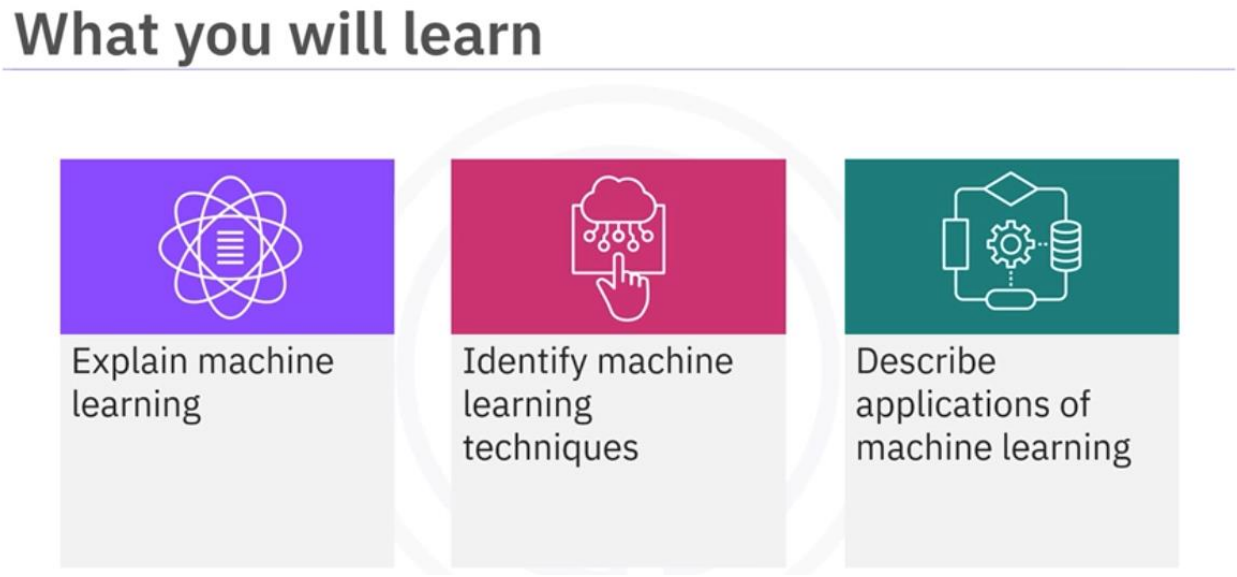
Gain hands-on experience with LangChain tools, components, and LLMs to develop real-world applications. Apply these skills in online labs and a final project for practical, interview-ready experience.

Finally, in the Applied Project-Based course, you'll enhance your knowledge **of LangChain document loaders**, apply text-splitting strategies, and **use watsonx** and vector databases to store and retrieve documents. Develop a QA bot with RAG and create a **Gradio interface** for model interaction. By the end, you'll have a project showcasing your generative AI skills for interviews.

Reading: Helpful Tips for Course Completion



An Overview of Machine Learning



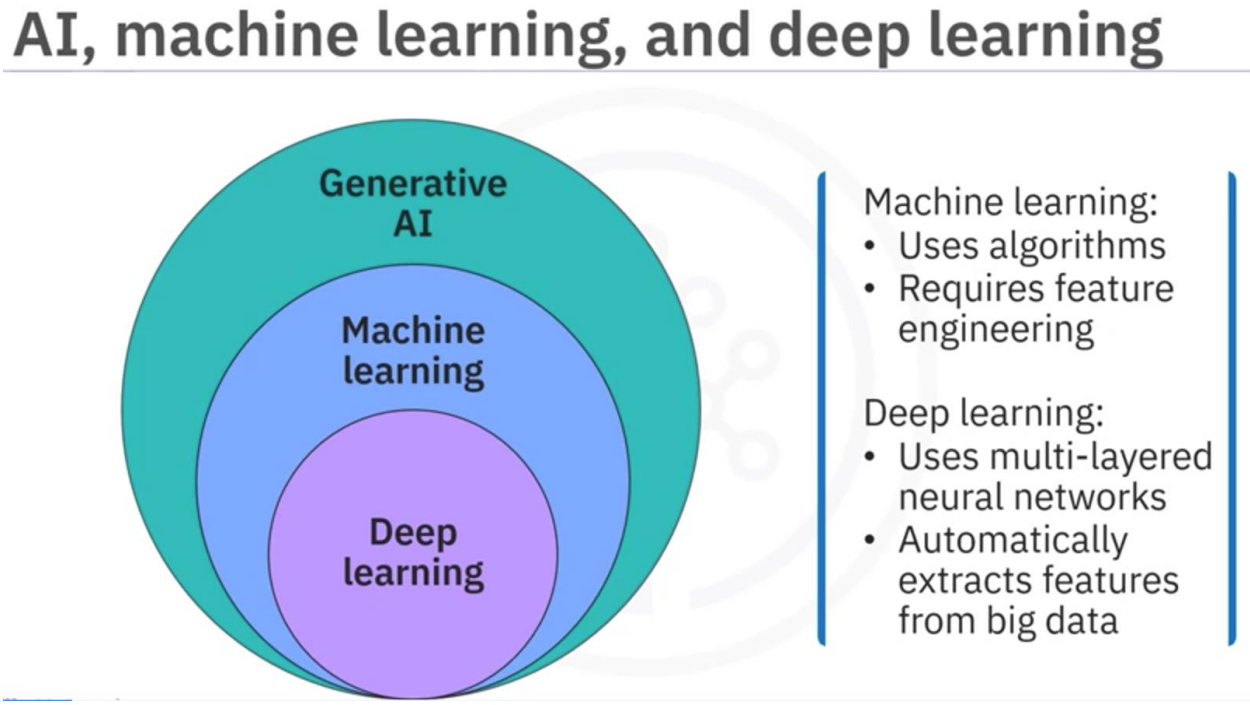
Explain Machine Learning.

Artificial intelligence, or AI, **makes computers appear intelligent** by simulating the cognitive abilities of humans.

AI is a general field with a broad scope. It comprises

- Computer vision.
- Natural language processing.
- Generative AI.
- Machine learning.
- Deep learning.

Machine learning, or ML, is the subset of AI that uses algorithms and **requires feature engineering** by practitioners.

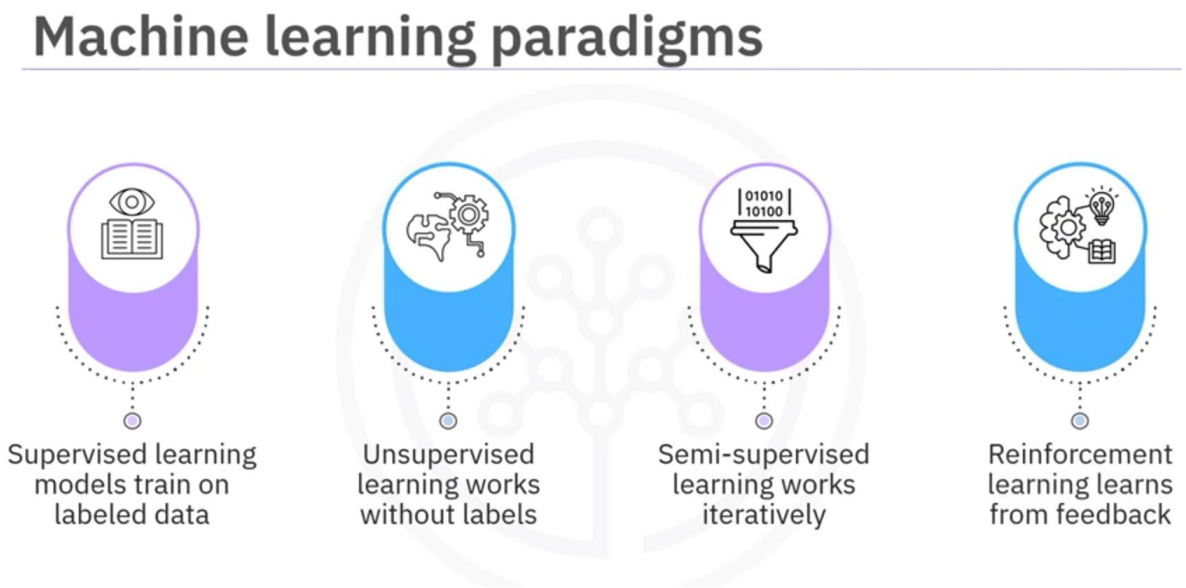


Deep learning distinguishes itself from traditional machine learning by **using** many-layered **neural networks** that automatically **extract features** from highly complex, unstructured big data.

Machine learning **teaches** computers to **learn from data, identify patterns, and make decisions** without receiving any explicit instructions from a human being.

ML algorithms **use** computational methods to learn information directly from data **without depending on any fixed algorithm**.

Machine learning models differ in how they learn.

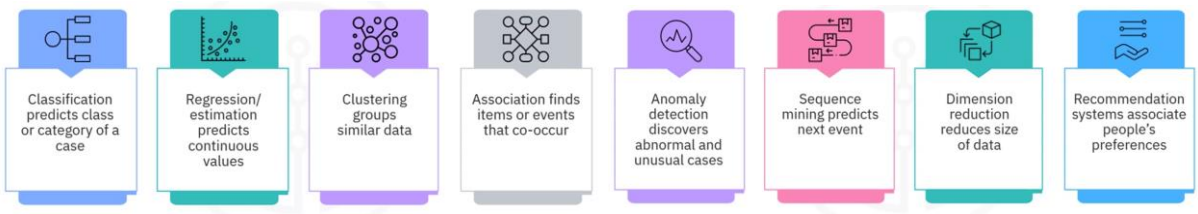


- **Supervised** learning models train on **labeled data** to learn how to make inferences on new data, enabling them to predict otherwise unknown labels.
- **Unsupervised** learning works **without labels** by finding patterns in data.
- **Semi-supervised** learning trains on a **relatively small subset of data that is already labeled, and iteratively retrains itself by adding new labels that it generates** with reasonably high confidence.
- **Reinforcement** learning simulates an artificially intelligent agent interacting with its environment and learns how to make decisions based on **feedback from its environment**.

Identify machine learning techniques.

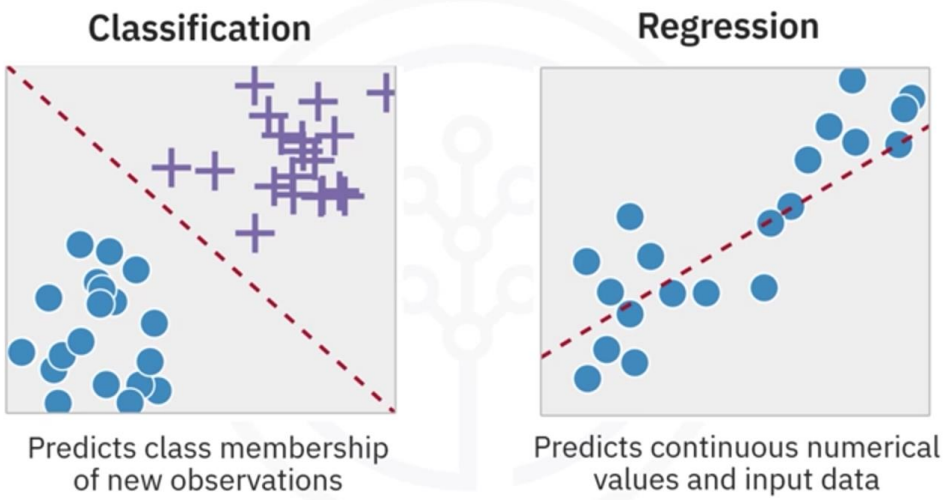
Selecting a machine learning technique depends on:

- The **problem** you're trying to solve.
- The type of **data** you have.
- The available **resources**.
- The desired **outcome**.



- A **classification** technique is used to **predict** the class or **category** of a case, such as whether a cell is benign or malignant or whether a customer will churn.
- The **regression/estimation** technique is used to **predict continuous values**, such as the price of a house based on its characteristics or the CO2 emissions from a car's engine.
- **Clustering** groups of similar cases, for example, can find similar patients or can be used for customer segmentation in the banking field.
- The **association** technique is used to find items or events that often co-occur, such as grocery items usually bought together by a particular customer or market segment.
- **Anomaly detection** is used to discover abnormal and unusual cases. For example, it's used for credit card fraud detection.
- **Sequence mining** is used to **predict** the **next event**. For instance, clickstream analytics in websites.
- **Dimension reduction** is used to reduce data size, particularly the number of features needed.
- **Recommendation systems**. This associates people's preferences with others who have similar tastes and recommends new items to them, such as books or movies.

Classification and regression

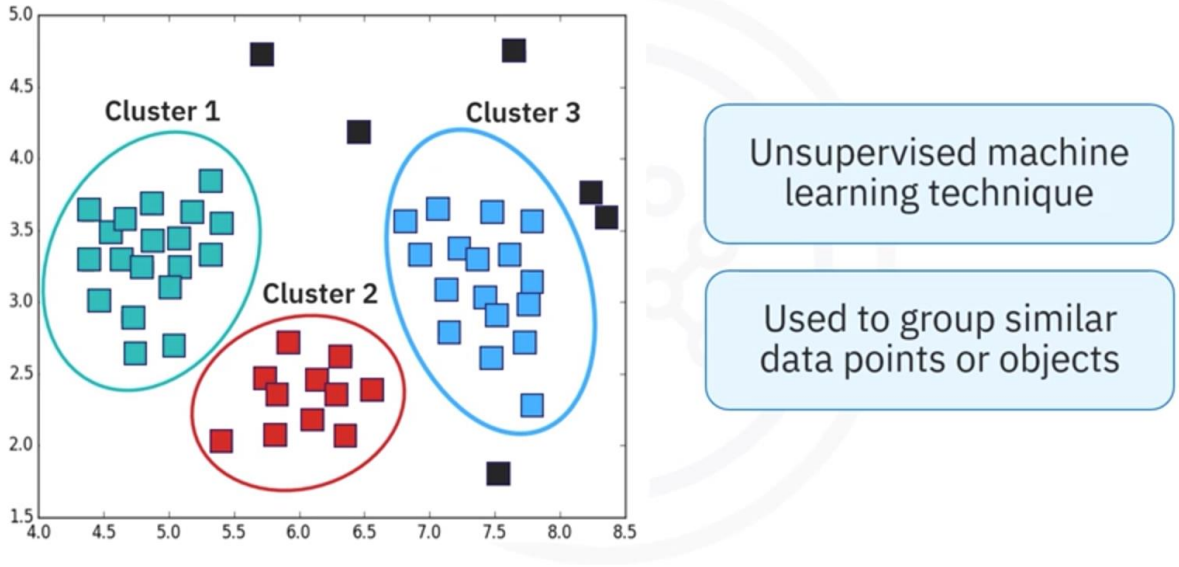


To train your machine learning model on **supervised data**, you can select between the classification or regression technique.

Classification categorizes input data **into** one of several **predefined categories** or classes, and then makes **predictions** about the class membership of new observations.

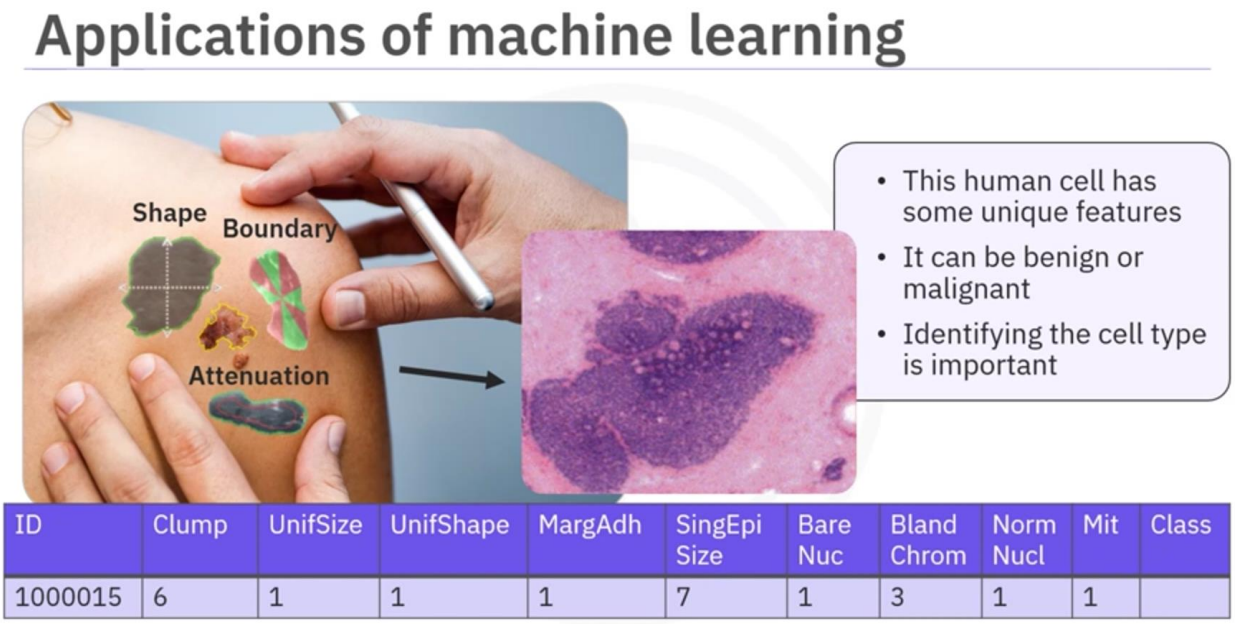
Regression is different from classification. It does not make predictions about the class membership of new input data. Regression **predicts** continuous numerical values from input data.

Clustering



Clustering is one of many **unsupervised** machine learning techniques for data modeling. It's used to **group data points** or objects that are **somehow similar**. For example, in this chart, we use a clustering algorithm to represent data points in green, red, and blue color. The **uncategorized data points** represented in black color are considered **noise**.

Describe applications of machine learning



ID	Clump	UnifSize	UnifShape	MargAdh	SingEpi Size	Bare Nuc	Bland Chrom	Norm Nucl	Mit	Class
1000015	6	1	1	1	7	1	3	1	1	

Let's look at some applications of machine learning now.

This is a human cell sample extracted from a patient. It has some unique characteristics.

For example, its clump thickness is 6, its uniformity of cell size is 1, its marginal adhesion is 1, and so on.

An interesting question we can ask by looking at this chart is, is this a benign or malignant cell?

Compared to a benign cell, a malignant cell invades its surrounding tissue and spreads around the body.

Identifying the cell type early can be the key to the patient's survival.

One could easily presume that only a doctor with years of experience could identify the cell and confirm if the patient is developing a cancer or not.

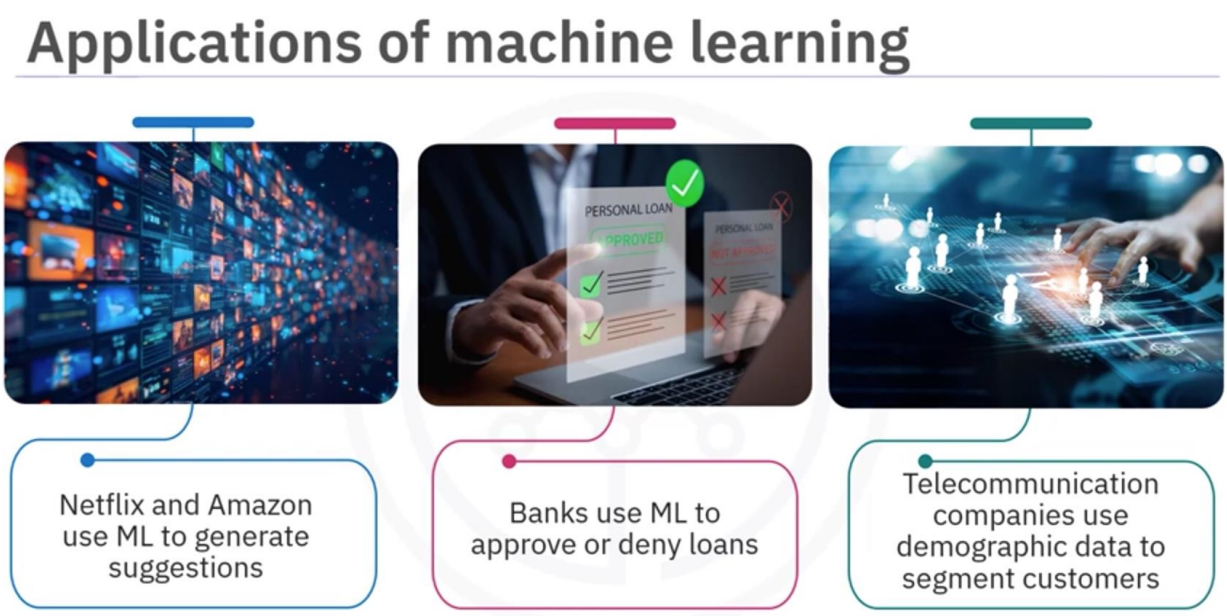
Applications of machine learning

Imagine you've obtained a dataset containing characteristics of thousands of human cell samples extracted from patients believed to be at risk of developing cancer. Analysis of the original data showed that many of the characteristics differed significantly between benign and malignant samples.

You can use the values of these cell characteristics in samples from other patients to indicate whether a new sample might be benign or malignant.

You should **clean your data**, **select a proper algorithm** for building a prediction model, and **train your model** to understand patterns of benign or malignant cells within the data. Once the model has been trained by going through data iteratively, it can be used to **predict your new or unknown cell** with a rather high accuracy. This is the power of machine learning.

Machine learning impacts society in a very meaningful way.



How do Amazon and Netflix **recommend content** and products for their users? They use machine learning. This is similar to how your friends might recommend a television show to you based on their knowledge of the types of shows you like to watch.

How do you think banks make a decision when **approving a loan application**? They use machine learning to predict each applicant's default probability and then augment their decision to approve or deny the loan application based on that probability.

How do telecommunication companies find how many of their **customers will churn**? They use demographic data to segment their customers and predict whether a customer will unsubscribe within the next month.

Let's use computer vision as an example of how machine learning works.

Assume you have a large collection of images of cats and dogs.

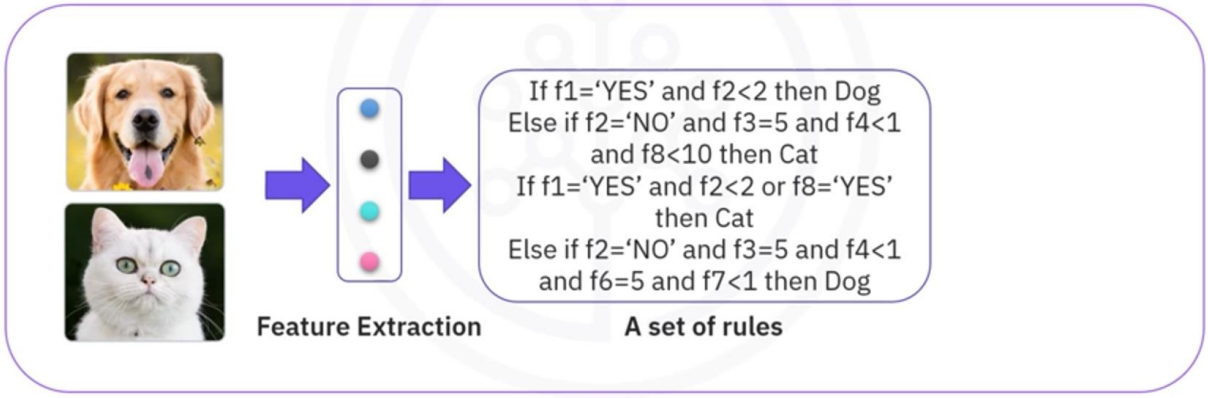
You want to develop a software application to differentiate cats from dogs.

The first thing you must do here is **interpret the images as a progression of features**.

- does the image show the animal's eyes?
- If so, what is their size?
- Does it have ears?
- What about a tail?
- How many legs?
- Does it have wings?

Image recognition with machine learning

- **Data:** Images of cats and dogs
- **Before ML:** Create rules to detect the animals

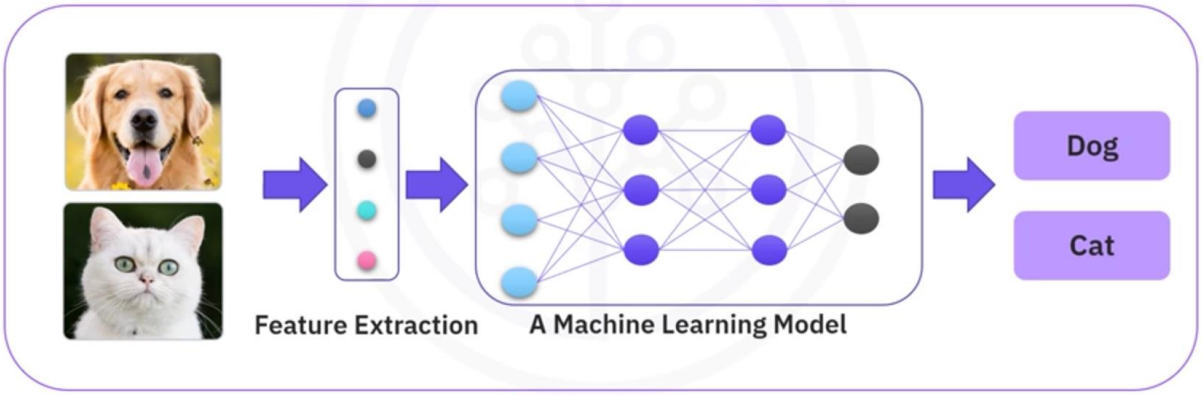


For machine learning, we had to **painstakingly create rules to get computers to appear intelligent** and detect the animals, but it was a failure. It needed a lot of rules, was too dependent on the current dataset, and therefore unable to characterize unseen cases.

Machine learning **algorithms** for computer vision **have evolved** to be able to identify dogs and cats in images.

Image recognition with machine learning

- **Data:** Images of cats and dogs
- **Before ML:** Create rules to detect the animals
- **With ML:** Build a model to infer the animal type



With machine learning, you can build a model that **learns** a set of distinguishing animal **features from known examples** and use them to infer the animal type automatically.

Although machine learning applications in the modern world are increasing, **humans are still in the mix**. If an ML algorithm refuses a loan application, a banker wants to know why and initiates a follow-up.

There are many other applications of machine learning that we see every day, such as virtual assistants like chatbots logging into our phones using face recognition or even playing computer games like chess.

Each of these use different machine learning techniques and algorithms.

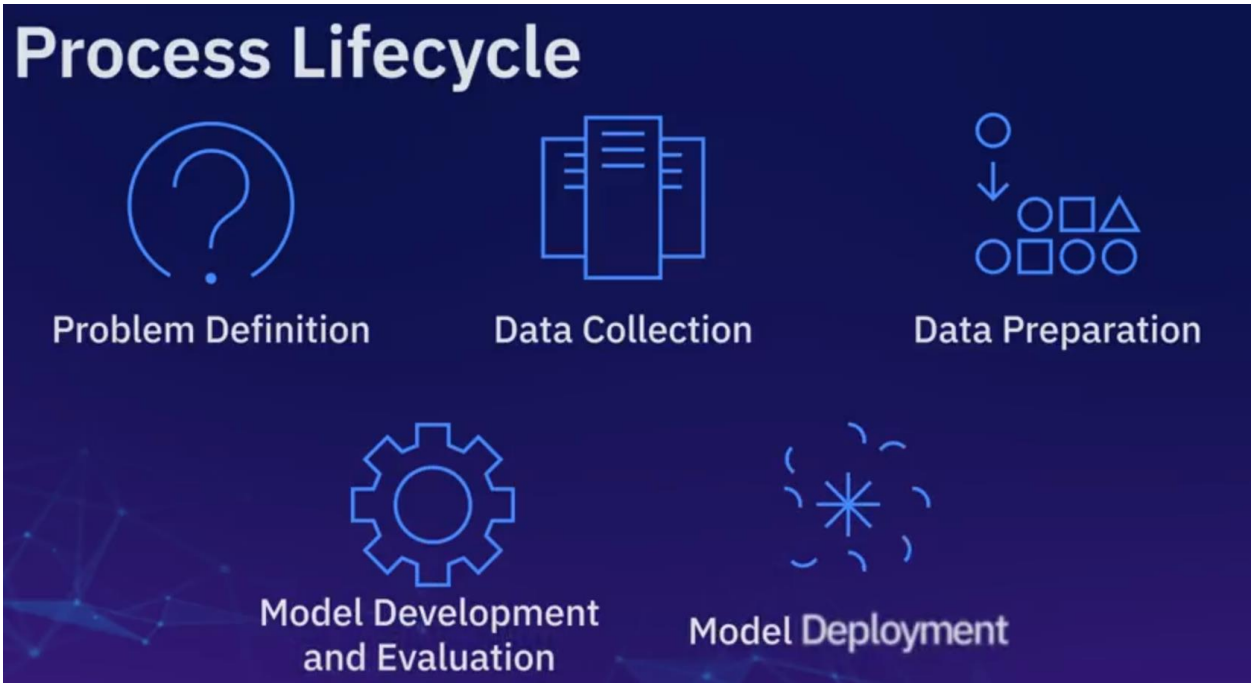
In this video, you learned that machine learning is the subset of AI that uses algorithms and requires feature engineering by practitioners.

Machine learning models learn using supervised, unsupervised, semi-supervised, and reinforcement learning.

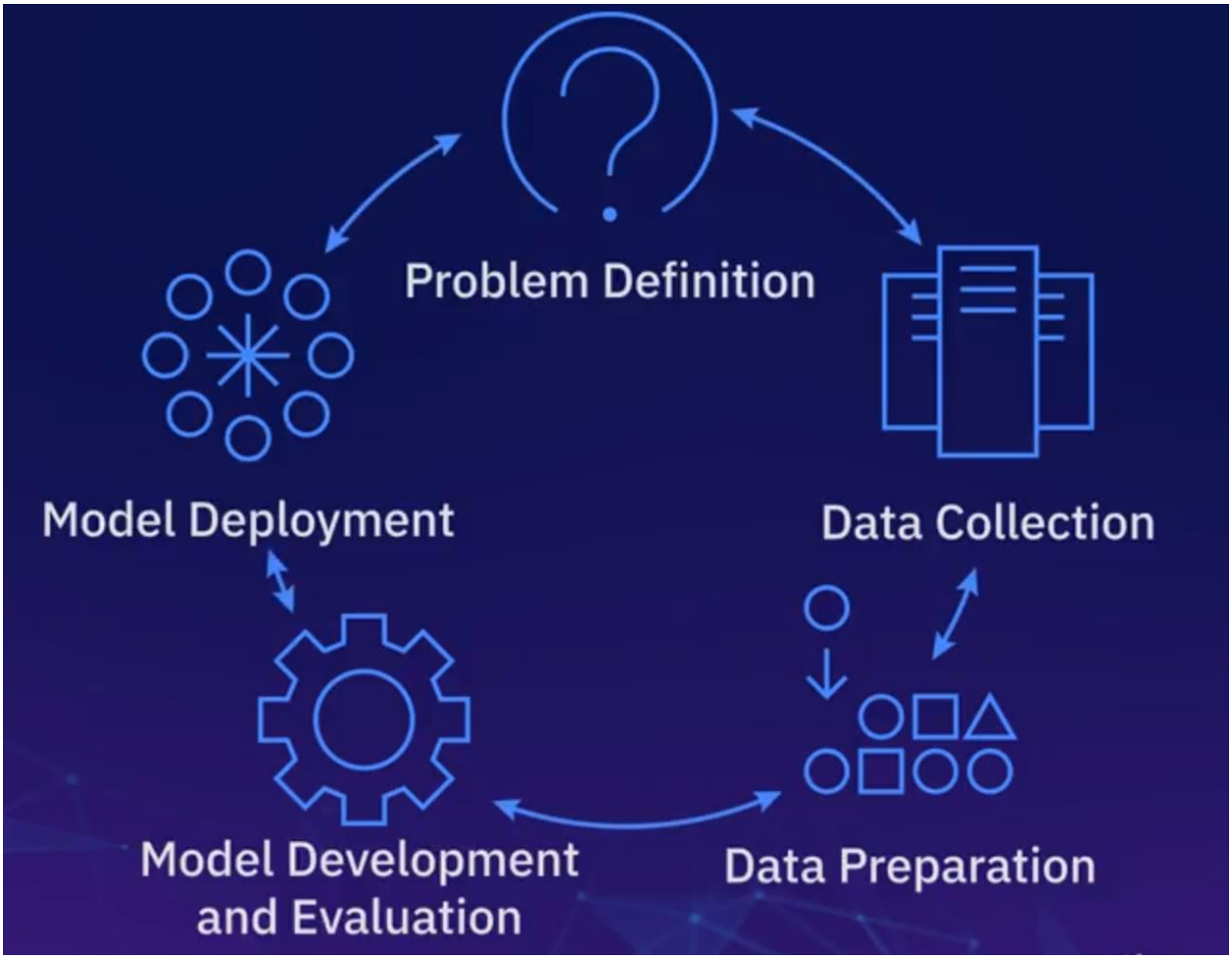
Machine learning consists of several techniques, namely classification, regression, clustering, association, anomaly detection, sequence mining, dimension reduction, and recommendation systems.

Finally, you learned how machine learning is applied to predict diseases, analyze consumer behavior, and recognize images.

Machine Learning Model Lifecycle



In reality, the model lifecycle is **iterative**, which means that we tend to go back and forth between these processes.

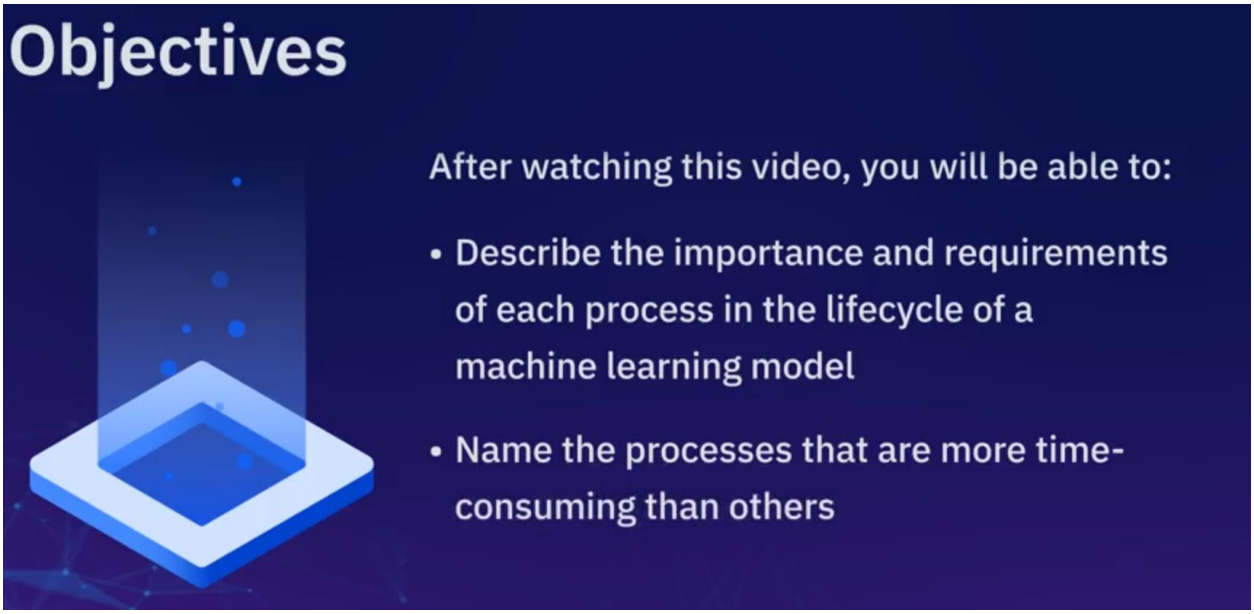


Data collection and preparation are called the Extract, Transform, and Load, or ETL process.

The ETL process involves collecting data from **various sources**, then **cleaning**, **transforming**, and **storing** it into a new single place.

The data is then accessible to the machine learning engineer, allowing them to perform tasks like building a machine learning model.

A Day in the life of a Machine Learning Engineer.



Now, let's go through the Lifecycle of a Machine Learning Model in a project that I am currently working on.

To help increase business revenue, I have been tasked with creating and deploying a **model that recommends similar products to what the customer has already bought**.

I have worked together with the client to come up with an **end-user's pain point**: "As a beauty product customer, I would like to receive recommendations for other products based on my purchase history so that I will be able to address my skincare needs and improve the overall health of my skin."

Defining the problem or stating the situation is very important, because I want to make sure the machine learning solution I am providing is aligned with the client's needs.

Now that I understand the client's needs, the next step is to begin **data collection**. I should determine what kind of data the company has and identify the sources it will come from.

This could be user data such as demographics, purchase history, and anything related to completed transactions. I can also get the product data, that is, the inventory of products and what they do, their ingredients, how popular they are, their customer ratings, and so on. Further, I may look at other data that includes information such as a user's saved products, liked products, search history, most visited products, and so on.

Then, I will go ahead and do some major transforming by wrangling, aggregating, joining, merging, and mapping the **data onto one central source**. This reduces the need to deal with multiple databases every time we need to pull data.

The next step in the process is **data preparation**. Most of the time, data from multiple sources will contain errors, different formatting, and missing data. This process overlaps with the Data Collection process as they can be done in tandem. The area of focus here is preparing a somewhat final version of the data. I will need to make sure that the data is cleaned to filter out irrelevant data, extreme values are removed to avoid influencing the data set, missing values are removed or randomly generated, depending on what the missing data may mean, and that each data column is in the proper format. For example, dates should be in date formats and strings should be properly identified.

I may also need to create additional features. For example, I may need to calculate the average duration between transactions for each user and find which products they buy the most. Or, I may need a feature that identifies what kind of skin issues each product targets and assign them to each user.

I can create plots to visually identify patterns, validate the data based on information that the beauty product subject matter expert has given me, and do some correlation analysis to identify what variables or features are very important to the users' buying habits and needs. This is called **Exploratory data analysis**.

I can also identify how I plan on splitting the data for training and testing. For example, do I want to randomly split the data or use the most recent transaction as a test set? In this example, I decided to put the most recent transaction in the test set and make sure that there was at least one transaction by that the same user in the training set.

In the Model Development step, I will go ahead and build a Machine Learning model.

Realistically, I try to leverage as many pre-existing frameworks and resources as possible, so I don't create anything from scratch. For this task, I will use a technique called content-based filtering. This technique finds the similarity between products, based on product content. For example, if someone is using a cleanser with lots of water, it is likely that the user has dry skin and will want a moisturizer that is highly moisturizing as well.

One of the steps I might take here is to create a similarity score of the products a user has purchased and rank them to other products.

I might recommend the most similar product while bearing in mind that there may be other factors that could come into play. For example, I might notice that the user has searched for products without particular ingredients, so I want to make sure that we are not recommending a product that they absolutely won't use.

I will also use a technique called Collaborative Filtering that uses the user's data. Here, I am creating similarities between two users based on how they view a product. For example, I can create a similarity, based on how two users rate their product.

First, I group users into a bucket based on their characteristics. This could be age, region, and skin type, products the users rated, and or purchased.

Then, I can take the average ratings for existing members and assume that the new user will be somewhere around the average, and recommend a product based on what others have rated highly.

The final model will be a combination of the two techniques.

After I am done building the model, I will go ahead and test that the model is performing well and that recommendations represent what the users want.

This is called **the Model Evaluation** step;

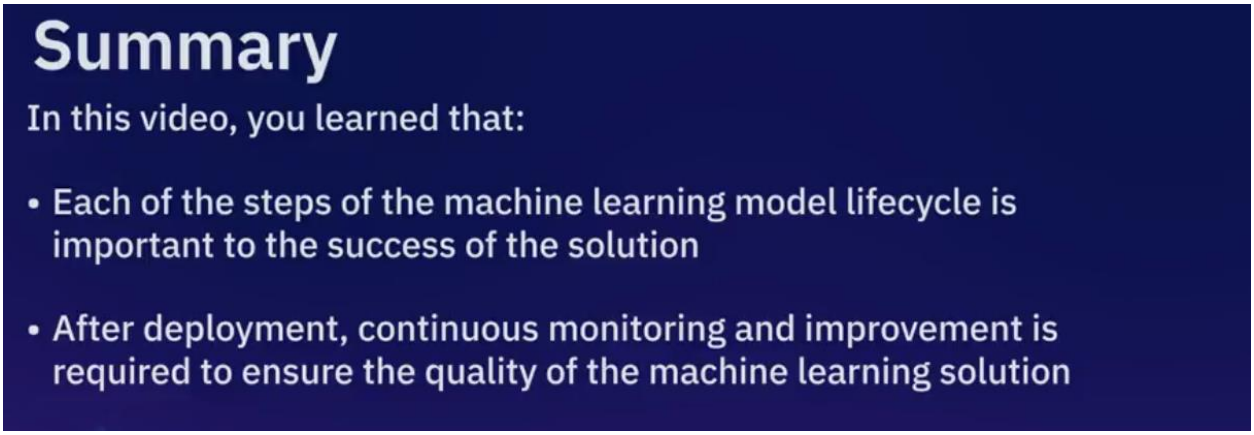
the initial stages of Model Evaluation will involve me tuning the model and doing some testing on the data set I had kept earlier for testing.

Once I am satisfied with the results, I will further evaluate the model by experimenting with the recommendations on a group of users and asking for their feedback.

The feedback will include asking the group of users to rate the recommendations and collecting data on the number of people who clicked and bought the recommended products, along with any other necessary metrics.

Now that I am done with building and testing, the model is ready to go to production. For this project, it will be a part of the beauty product app and website. While this is the last step, I still need to track the **deployed model's performance** to make sure it continues to do the job that the business requires.

Future iterations may include retraining the model based on new information in order to expand its capabilities.



Data Scientist vs AI Engineer

For many years, data science has been called the sexiest job of the 21st century, but in recent years, it seems like there's a new job vying for that title, the AI engineer.

So who even are these new kids on the block? Are they just data scientists in disguise? What's up y'all?

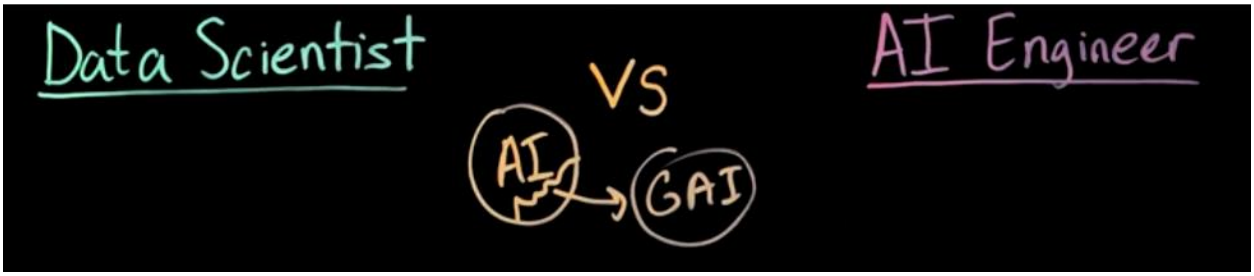
I'm Isaac Key, and I'm a former data scientist turned AI engineer at IBM.

To answer these questions, I'm going to lay out four key areas in which the work of a data scientist differs from an AI engineer, specifically a generative AI engineer.

What's happening in the industry?

So traditionally, data scientists have always used AI models to do their analysis. So what's changed? Well, with the advent of generative AI, the boundaries of what AI can do are being pushed in ways that we've never seen before.

And so these breakthroughs have been so groundbreaking **that generative AI has split off** into its own distinct field, and we call that AI engineering.



Let's dive into the differences between a Data Scientist and a AI Engineer.

Use cases

So at a very high level, think of a **data scientist as a data storyteller**.They take massive amounts of messy real-world data, and they use mathematical models to translate this data into insights.

On the other hand, think of an **AI engineer as an AI system builder**. They **use foundation models** to build generative AI systems that help to transform business processes.

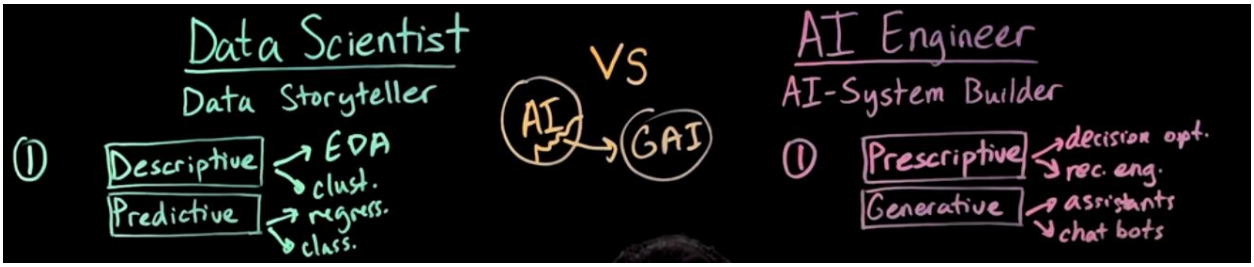
So since data scientists are fantastic storytellers, they use a lot of descriptive analytics to **describe the past**. One example of this is through what's called **exploratory data analysis**, or EDA, which is all about graphing the data and doing statistical inference.

They can also do this through what's called **clustering**, which group similar data points based off similar characteristics, such as, say, doing customer segmentation.

Now every good story has a reader trying to figure out what's going to come next, and that's where predictive use cases come in.

As opposed to a book, however, a data scientist does not have the end already written, so they must **use** what are called **machine learning models to make their predictions**. An example of this is called **regression** models, which predict a numeric value such as, say, a temperature or a revenue.

Another type of these models are **classification** models, which predict categorical value such as success or failure.



So, putting on the AI engineering hat now, one of the main use cases that AI engineers work on are called **prescriptive** use cases, which are all about choosing the best course of action.

An example of this is a technique called **decision optimization**, which enables businesses to assess a set of possible actions and then choose the most optimal path based off a set of requirements or standards. Another example of a prescriptive use case is through creating what are called **recommendation engines**. As an example, this can involve suggesting targeted marketing campaigns for a select customer base.

In addition to prescriptive use cases, there are also generative use cases, hence the name generative AI.

Now foundation models, which I will touch on more in a bit, enable the creation of what are called **intelligent assistants**, for example, a coding assistant or a digital advisor.

They also enable the creation of **chatbots**, as an example, which enable conversational search through information retrieval and the summarization of various content.

Data

So after we have a use case identified, we need data.

Now people say that data is a **new oil** because like oil, you have to **search** for and **find** the right data and then use the right processes to transform it into various products, which then power various processes.

For a **data scientist**, the oil of choice is often structured data, aka **tabular data**. Do note that data scientists still work with unstructured data, but not as much as AI engineers.

Now these tables are often in the order of **hundreds to hundreds of thousands of observations**, and they require a lot of **cleaning** and **preprocessing** before the data can be modeled. Some of the **cleaning** involved, for example, involves removing outliers or joining and filtering on a new table, or even creating new features altogether. This clean data is then used to train various machine learning models.

Now on the other hand, an **AI engineer**, for them, the oil of choice is mainly **unstructured data** such as text, images, videos, audio files, etc.

Let's take a text-based foundation model called an LLM, or Large Language Model, as an example. These models require anywhere between **billions to trillions of tokens of text to be trained** on, which is a lot larger scale compared to traditional machine learning models.

Models

So, the data science toolbox consists of **hundreds of different models** and different algorithms that they can choose from. Due to the nature of these models, each different use case requires gathering a different dataset and thus requires training a different model.

And so as a result, the **scope** of these individual models is a lot narrower, meaning that **it's harder for them to generalize** past the domain of data that they've been trained on.

And these models are a lot **smaller** in **size** in terms of the number of **parameters**, they take less **compute power** to train and do inference, and they require **less time** to train, anywhere between seconds to hours.

Now on the other hand, the generative AI toolbox is a lot less cluttered, and it only contains **one type of model**, and that is called the foundation model.

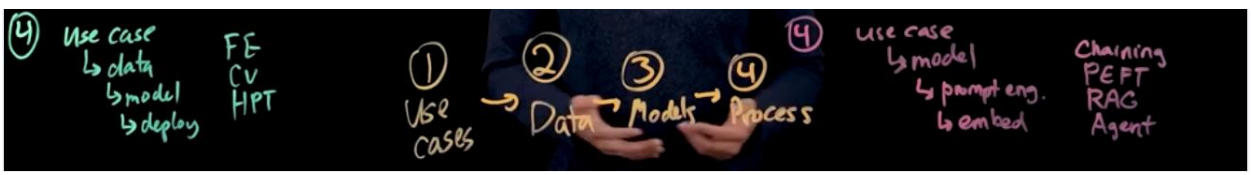
Now foundation models are revolutionary because they allow one single model to **generalize to a wide range of tasks without having to be retrained**.

Thus, their scope is called **wider**. And due to the sophistication of these models, they are a lot **larger in size**, often **billions of parameters**, they require a lot **more computing power** to train, we're talking hundreds to thousands of GPUs, and they require a lot **more training time**. Now we're talking anywhere between weeks to months.

Process

Due to the differences in the intrinsic nature between traditional machine learning models and foundation models, this also means that the underlying **processes** and techniques that are used to develop solutions with these also **differ**.

So a typical **data science process** will look something like this. You start off with a **use case**, and then from that use case, you pick **the right data**. Then after that data is prepared, you use it to **train and validate a model** using techniques such as **feature engineering**, **cross-validation**, or **hyperparameter tuning**, as an example. This model then **is deployed** at some endpoint, for example in the cloud, to do real-time prediction and inference.



Now on the other hand, the generative AI process also starts off with a **use case**, but then we can skip directly working with a **pre-trained model**. And what makes this possible is a

phenomenon called **AI democratization**, which is a big fancy word that simply means making AI more widely accessible to everyday users.

Some of the **best foundation models out there are published to open-source communities** such as Hugging Face, and since these models are so generalizable and so powerful out of the box, they make it easy for developers to get started.

AI engineers interact with these foundation models via natural language instructions to prompt them to do various tasks, and this process is known as **prompt engineering**. Now prompt engineering can be used in conjunction with different frameworks to then build larger AI systems.

An example of these frameworks include, as one, **chaining different prompts together**, or doing what's called **parameter-efficient fine-tuning**, or PEFT, on domain-specific data, or doing **retrieval-augmented generation**, aka RAG, to ground answers in truth, or even by creating **autonomous agents** to reason through very complex multi-step problems.

So these are just a few of the examples of the building blocks that can be used to build larger AI applications.

The last step is to **then embed the AI in a larger system or workflow**. This can take on the form of creating assistants or virtual agents, building a larger application with a UI, or even doing some sort of automation.

So okay, let's take a step back and let's look at all the differences at a very high level.

As we can see, the breakthroughs in generative AI underpin many of the differences in the use cases, data, models, and processes that data scientists and AI engineers work on.

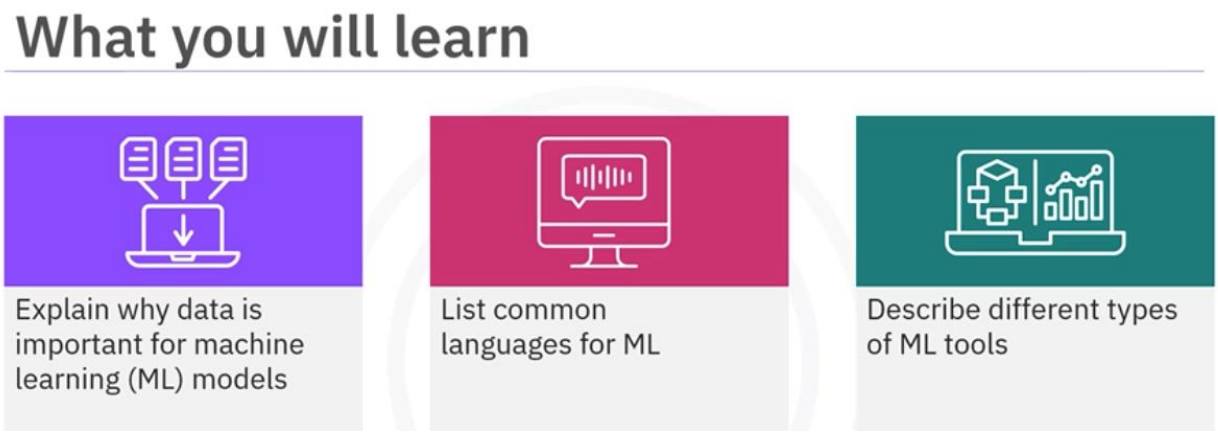
It's important to note that there is still **overlap between the two fields**.

For example, data scientists will still work on prescriptive use cases, or an AI engineer will still work with structured data.

Regardless of these differences, both of these fields are continuing to evolve at a blazing fast pace with new research papers, new models, new tools coming out every single day.

With data, AI, and a creative mind, really anything is possible with these.

Tools for Machine Learning



Welcome to Tools for Machine Learning.

- Explain why data is essential for machine learning models.
- List common languages for machine learning.
- Describe tools for:
    - data processing and analytics.
    - data visualization.
    - machine learning.
    - deep learning.
    - computer vision.
    - natural language processing, or NLP.
    - generative artificial intelligence, or AI.

What is data?

Data is a **collection of raw facts**, figures, or information used to draw insights, inform decisions, and fuel advanced technologies.

Data is central to every machine learning algorithm and the source of all the information the algorithm uses **to discover patterns and make predictions**.

What are machine learning tools.

Machine learning tools provide functionalities for machine learning pipelines, which include modules for data preprocessing and building, evaluating, optimizing, and implementing machine learning models.

These tools use algorithms to simplify complex tasks, such as handling big data, conducting statistical analyses, and making predictions.

- **Pandas** library, used for data manipulation and analysis.
- **Scikit-learn** library, which provides a wide range of supervised and unsupervised learning algorithms **for linear regression**.

Machine Learning programming languages.

A machine learning programming language is a programming language for building machine learning models and decoding the hidden patterns in data.

- **Python** is a widely used language due to its extensive collection of libraries for analyzing and processing data and its ease in developing machine learning models.
- **R** is another popular **language for statistical learning** that also contains many libraries for data exploration and machine learning.
- **Julia** is a high-performance language with **parallel and distributed** numerical computing support used by researchers.
- **Scala** is a scalable, widely used language for processing big data and building machine learning **pipelines**.
- **Java** is a multi-purpose language that supports scalable machine learning applications deployed in production.
- **JavaScript** is used to **run machine learning models in web browsers** to serve client-side applications.

Machine learning tools purposes.

You can use them to store and retrieve data, work with plots, graphs, and dashboards, explore, visually inspect, and clean data, and prepare data for developing machine learning models. These tools can be divided into specific categories, as shown here:

Tools for data processing and analytics.

Data processing and analytics tools process, store, and **interact with data** to serve machine learning models.

- **PostgreSQL** is a powerful **open-sourced object-relational database** system based on SQL, a language designed for storing, manipulating, and retrieving data in databases.
- **Hadoop** is an open-source, highly scalable **disk-based** solution for storing and batch-processing massive data.
- **Spark** is a distributed, **in-memory** data processing framework for real-time big data processing. It supports DataFrames and SQL. It is faster and easier to use than Hadoop.
- **Apache Kafka** is a **distributed streaming platform** for building big data pipelines and running real-time analytics.
- **Pandas** is a popular Python library for **exploring and wrangling** data. Central to Pandas is the DataFrame, a tabular data structure for cleaning and transforming structured data.
- **NumPy** is a Python library that provides comprehensive **mathematical functions**, random number generators, and linear algebra routines for distributed and graphics processing unit, or **GPU-based computing platforms**.

Tools for data visualization.

Data visualization tools help you understand and visualize the structure of your data.

- **Matplotlib** is an extensive foundational library for generating **customizable** plots and **interactive visualizations**.
- **Seaborn** is a library based on Matplotlib. It provides a **high-level interface** for drawing attractive and informative statistical graphics.
- **ggplot2** is an open-source data visualization **package in R**. It allows you to build and add elements to your graphics in layers.
- **Tableau** is a business intelligence tool for interactive data visualization **dashboards**.

Tools for machine learning.

Machine learning tools are used to create and tune your machine learning models.

- **NumPy** provides **foundational machine learning support** with efficient numerical computations on large, multidimensional data arrays.
- **Pandas** is used for data analysis, visualization, cleaning, and **preparing data** for machine learning.

- **SciPy**, built on NumPy, is used for **scientific computing** and has modules for optimization, integration, linear regression, and more.
- **Scikit-learn** is used for building **classical machine learning models**, offering a full suite of classification, regression, clustering, and dimensionality reduction algorithms. It's built on NumPy, SciPy, and Matplotlib.

Tools for deep learning.

Deep learning tools are frameworks for designing, training, and testing neural network-based machine learning models.

- **TensorFlow** is an open-source library for numerical computing and large-scale machine learning.
- **Keras** is an easy-to-use deep learning library for implementing **neural networks**.
- **Theano** is for efficiently defining, optimizing, and evaluating **mathematical expressions involving arrays**.
- **PyTorch** is an open-source library for **deep learning** applications and **computer vision** and **NLP**. It also allows experimentation to test ideas.

Tools for computer vision.

Computer vision tools are used for tasks such as object **detection**, image **classification**, **facial recognitions**, and image **segmentation**. All of the deep learning tools can be tailored to computer vision applications.

- **OpenCV**, or Open Source Computer Vision Library, is a library for real-time computer vision applications such as object **detection**, image **classification**, and **augmented reality**.
- **Scikit-Image**, built on SciPy and compatible with Pandas, offers image processing algorithms such as **filters**, **segmentation**, **feature extraction**, and **morphological operations**.
- **TorchVision**, part of the PyTorch project, consists of **popular datasets**, image loading, **pre-trained deep learning architectures**, and common image transformations for computer vision.

Tools for natural language processing, or NLP.

NLP tools help developers and data scientists build applications that **understand, interpret, and generate human language**.

- **Natural Language Toolkit**, or NLTK, is a comprehensive library that offers text processing, **tokenization**, and **stemming** tools.

- **TextBlob** is a library for tasks like part-of-speech tagging, noun-phrase extraction, **sentiment analysis**, and **translation**.
- **Stanza** is an NLP library from the Stanford NLP Group with accurate **pre-trained models** for many NLP tasks, including part-of-speech tagging, named entity recognition, and dependency parsing.

Tools for generative artificial intelligence, or AI.

Generative AI tools leverage AI to **generate new content** based on input data or prompts, such as text, images, music, code, or other media.

- **Hugging Face Transformers** is a powerful library of **transformer models for NLP** tasks, such as text generation, language translation, and sentiment analysis.
- **ChatGPT** is a powerful **language model used for** text generation, building chatbots, and other NLP tasks.
- **DALL-E** is a tool from OpenAI for **generating images from textual descriptions**.
- **PyTorch** uses deep learning to create generative models, such as Generative Adversarial Networks, or GANs, and Transformers for text and image generation.

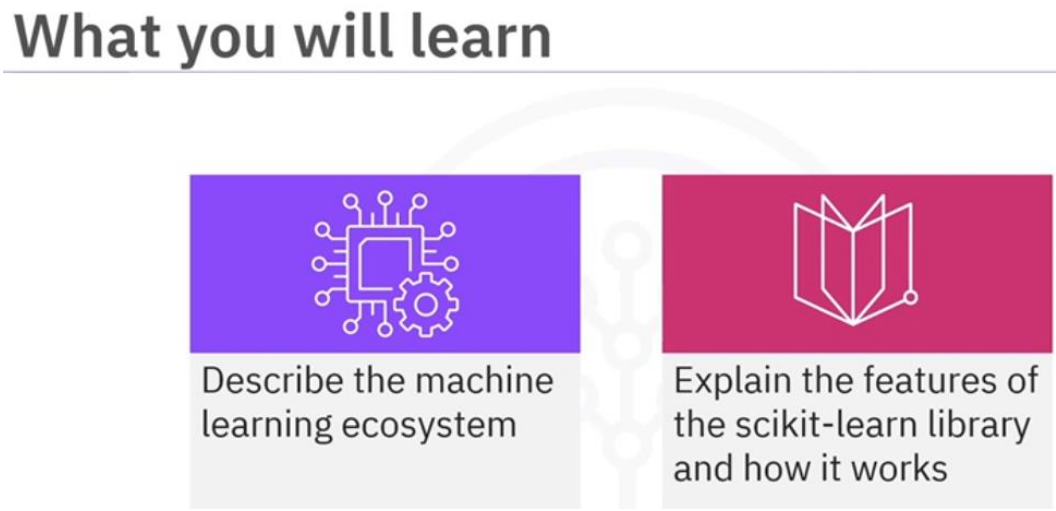
Recap

- Data is central to an ML algorithm
- ML tools simplify tasks and provide functionalities for ML pipelines
- ML programming language is used to build ML models and decode data patterns
- Common languages: Python, R, Julia, Scala, Java, and JavaScript

Tools:

- **Data processing and analytics**: PostgreSQL, Hadoop, Spark, Apache Kafka, pandas, and NumPy
- **Data visualization**: Matplotlib, Seaborn, ggplot2, and Tableau
- **Machine learning**: NumPy, Pandas, SciPy, and scikit-learn
- **Deep learning**: TensorFlow, Keras, Theano, and PyTorch
- **Computer vision**: OpenCV, scikit-image, and TorchVision
- **NLP**: NLTK, TextBlob, and Stanza
- **Generative AI**: Hugging face transformers, ChatGPT, DALL-E, and PyTorch

Scikit-learn Machine Learning Ecosystem

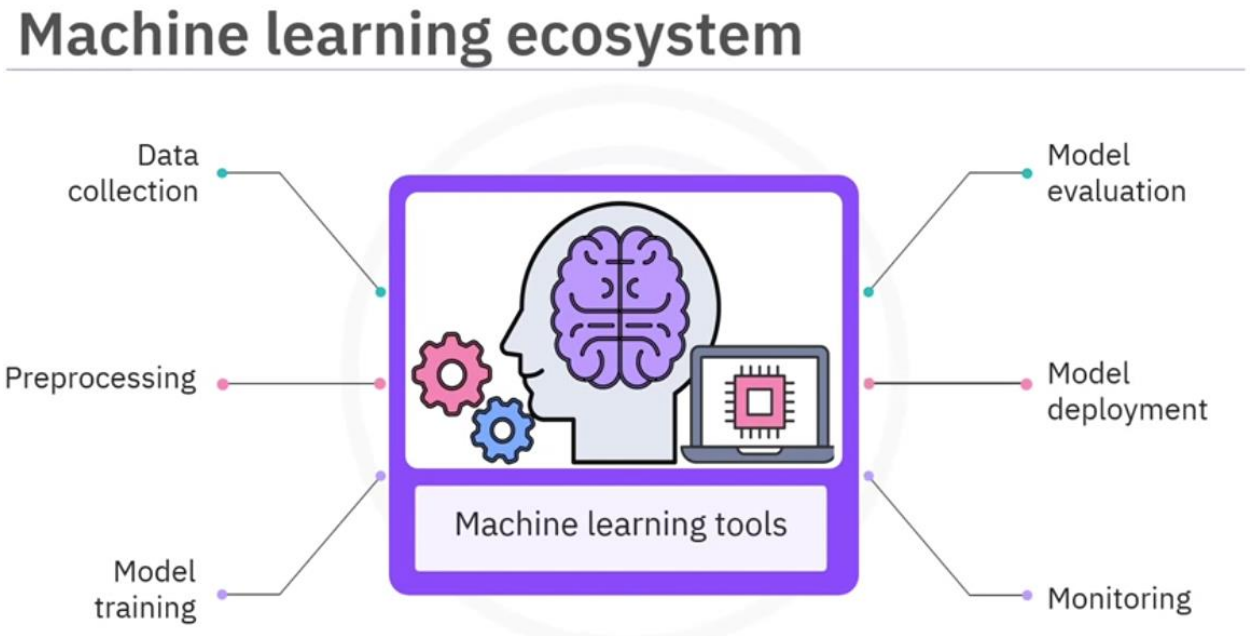


Machine Learning ecosystem

Imagine you developed a **music streaming app** that allows users to play and download music, share music files, and create playlists.

To **increase your app's user base**, you **collect information** on your users' listening habits, such as what songs they play, how long they listen to songs, and which songs they skip.

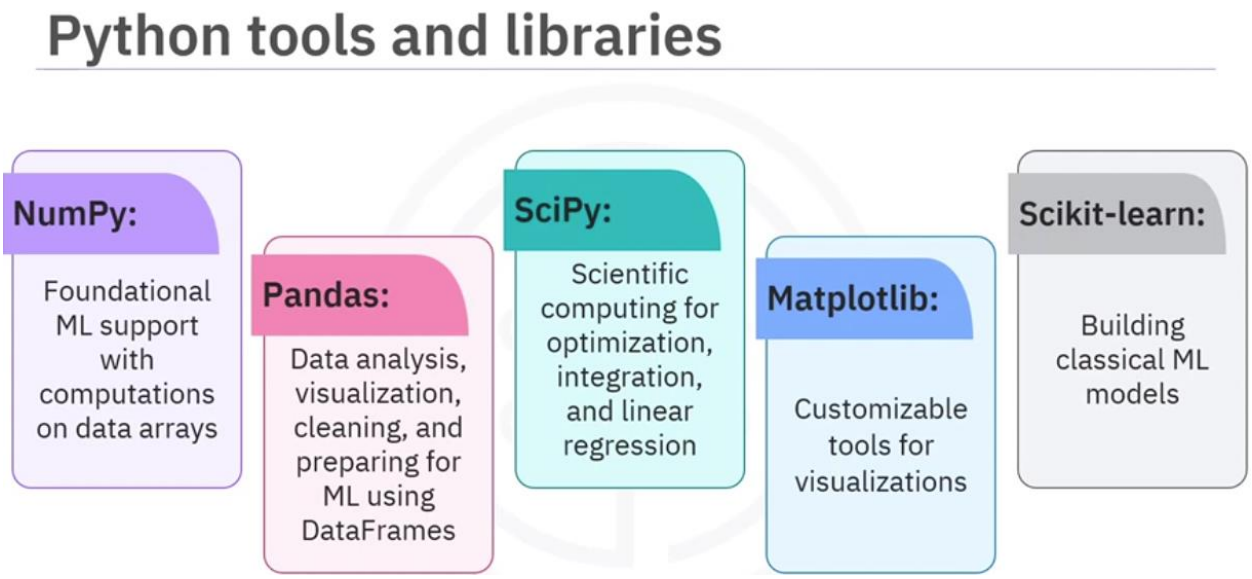
Once you've collected the information, you want to **normalize** it by finding inconsistent data, missing values, and outliers.



Machine learning tools can generate this type of information. They are useful for data collection, preprocessing, model training, model evaluation, model deployment, and monitoring.

The machine learning, or ML ecosystem, refers to the interconnected tools, frameworks, libraries, platforms, and processes that support developing, deploying, and managing machine learning models.

Python tools and libraries for machine learning.

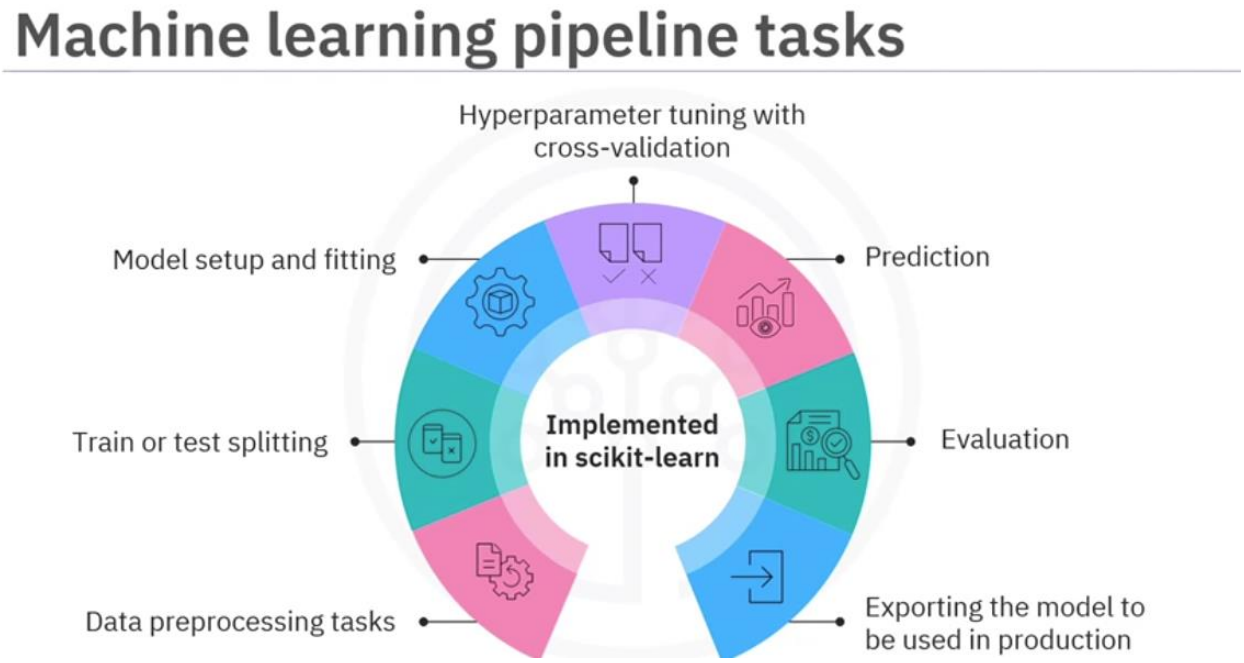


Several open-sourced Python libraries comprise one of the most widely used ecosystems for machine learning.

- **NumPy** provides foundational machine learning support with its efficient numerical computations on large multidimensional data arrays.
- **Pandas**, built on NumPy and Matplotlib, is used for data analysis, visualization, cleaning, and preparing data for machine learning. Pandas uses versatile arrays called data frames to handle data.
- **SciPy**, built on NumPy, is used for scientific computing and has modules for optimization, integration, linear regression, and more.
- **Matplotlib** is built on NumPy and has an extensive, highly customizable set of visualization tools.
- **Scikit-learn**, built on NumPy, SciPy, and Matplotlib, is used for building classical machine learning models.

scikit-learn LIbrary.

- Scikit-learn is a **free** machine learning library for the Python programming language.
- It has a wide, up-to-date selection of **classification**, **regression**, **clustering**, and **dimensionality reduction** algorithms.
- It's designed to **work with** the Python numerical and **scientific libraries** NumPy and SciPy.
- It has **excellent documentation** and a large community support network.
- Scikit-learn is **constantly evolving** with contributions from a community of thousands and is exceeded only by Pandas.
- Implementing machine learning models with scikit-learn is **easy**, with just a few lines of Python code.



Most of the tasks that need to be done in a machine learning pipeline are already implemented in scikit-learn, including data **preprocessing** tasks like data cleaning, scaling, feature selection and feature extraction, train or test **splitting**, **model setup** and **fitting**, hyperparameter **tuning with cross-validation**, **prediction**, **evaluation**, and **exporting** the model to be used in production.

Scikit-learn workflow.

Consider a data set X and a target variable Y, both stored as NumPy arrays.

Scikit-learn's pre-processing package provides several common utility functions and transformer classes to help you prepare data for modeling.

For instance, this first code block scales your data by **standardizing** it.

```
from sklearn import preprocessing
X = preprocessing.StandardScaler().fit(X).transform(X)
```

In supervised learning, you want **to split your data set into train and test sets** to train your model and then test the model's accuracy separately. Scikit-learn can split arrays and matrices into random train and test subsets for you in one line of code. Here, 33% of the data is reserved for testing.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
```

Next, you can **instantiate a classifier model** using a support vector classification algorithm. This line of code generates a classification model object, called clf, and initializes its parameters, gamma and C.

```
from sklearn import svm
clf = svm.SVC(gamma=0.001, C=100.)
```

After initializing your model clf, you can **train your model on the training data**. The clf model learns to predict the classes for unknown cases by passing the training set to the fit method.

```
clf.fit(X_train, y_train)
```

Then you can use the test data to **generate predictions**.

```
clf.predict(X_test)
```

The result tells you the predicted class for each observation in the test set.

```
yhat = clf.predict(X_test)
```

You can also use different metrics to **evaluate** your model accuracy, such as a **confusion matrix** to compare the predicted and actual labels for the test set.

```
from sklearn.metrics import confusion_matrix
print(confusion_matrix(y_test, yhat, labels=[1,0]))
```

And finally, you can **save your model** as a pickle file and retrieve it whenever you like.

```
import pickle
s = pickle.dumps(clf)
```

Recap

In this video, you have learned that a machine learning ecosystem refers to the interconnected tools, frameworks, libraries, platforms, and processes that support developing, deploying, and managing machine learning models.

It uses several Python tools and libraries, such as NumPy, Pandas, SciPy, Matplotlib, and scikit-learn.

Scikit-learn is a free machine learning library for Python that uses classification, regression, clustering, and dimensionality reduction algorithms.

Most tests required in a machine learning pipeline are already implemented in scikit-learn.

Finally, you learned the basic machine learning workflow using the scikit-learn library.

Practice Quiz: Introduction to Machine Learning

1. Which one of the following best describes machine learning?

   - (●) Machine learning teaches computers to learn from data, identify patterns, and make decisions without receiving explicit instructions from a human being.
   - ( ) Machine learning enables computers to automatically make decisions without the need for data or explicit instructions.
   - ( ) Machine learning enables computers to detect anomalies and find patterns to assign data into similar groups.
   - ( ) Machine learning teaches computers to learn under supervision how to make predictions from labeled data.

2. Which one of the following tasks is a machine learning engineer more likely to perform than a data scientist?

   - ( ) Creating informative plots for illustrating key insights and telling stories about data.
   - (●) Build a data pipeline for deploying a machine learning model.
   - ( ) Investigate causal relationships between variables by developing and testing hypotheses.
   - ( ) Research the latest trends in data analytics.

3. What are the key lifecycle stages of a machine learning model?

- ○ Monitor, train, test, and deploy your model.
- ○ Collect the data and develop, evaluate, and deploy a model into production.
- ○ Define the problem, collect the data, and develop, evaluate, and deploy a model.
- ◉ Define the problem, collect the data, preprocess the data, and develop, evaluate, and deploy a model.

4. Which library is at the core of an open-source Python machine learning ecosystem that enables you to develop machine learning models?

- ◉ Scikit-learn
- ○ SciPy
- ○ NumPy
- ○ Pandas

5. Which library is a tool for data analysis, visualization, cleaning, and preparing data for machine learning?

- ○ NumPy
- ○ SciPy
- ○ Scikit-learn
- ◉ Pandas

Summary and Highlights

- Artificial intelligence (AI) **simulates human cognition**, while machine learning (ML) uses algorithms and requires feature engineering to **learn from data**.
- Machine learning includes different types of models: **supervised** learning, which uses **labeled** data to make **predictions**; **unsupervised** learning, which finds **patterns** in **unlabeled** data; and **semi-supervised** learning, which trains on a small subset of labeled data.
- **Key factors** for choosing a machine learning technique include the type of **problem** to be solved, the available **data**, available **resources**, and the desired **outcome**.
- Machine learning techniques include **anomaly detection** for identifying unusual cases like fraud, **classification** for categorizing new data, **regression** for predicting continuous values, and **clustering** for grouping similar data points without labels.

- Machine learning tools support **pipelines** with modules for data **preprocessing**, model **building**, **evaluation**, **optimization**, and **deployment**.
- **R** is commonly used in machine learning for **statistical analysis** and data exploration, while Python offers a vast array of libraries for different machine learning tasks. Other programming languages used in ML include Julia, Scala, Java, and JavaScript, each suited to specific applications like high-performance computing and web-based ML models.
- **Data visualization tools** such as Matplotlib and Seaborn create customizable plots, ggplot2 enables building graphics in layers, and Tableau provides interactive data dashboards.
- **Python libraries** commonly used in machine learning include NumPy for numerical computations, Pandas for data analysis and preparation, SciPy for scientific computing, and Scikit-learn for building traditional machine learning models.
- **Deep learning frameworks** such as TensorFlow, Keras, Theano, and PyTorch support the design, training, and testing of neural networks used in areas like computer vision and natural language processing.
- **Computer vision** tools enable applications like object detection, image classification, and facial recognition, while natural language processing (NLP) tools like NLTK, TextBlob, and Stanza facilitate text processing, sentiment analysis, and language parsing.
- Generative AI tools use artificial intelligence to create new content, including text, images, music, and other media, based on input data or prompts.
- Scikit-learn provides a range of functions, including classification, regression, clustering, data preprocessing, model evaluation, and exporting models for production use.
- The machine learning ecosystem includes a network of tools, frameworks, libraries, platforms, and processes that collectively support the development and management of machine learning models.

Graded Quiz: Introduction to Machine Learning

1. You are building a model to estimate the exact blood sugar level of diabetic patients based on age, weight, and lifestyle habits. Which machine learning techniques will help in this task?

   - ◯ Regression technique
   - ◯ Clustering technique
   - ◯ Association technique
   - ◯ Classification technique

2. Which of the following correctly describes the machine learning model lifecycle workflow?

   - ◯ Define problem → Collect data → Prepare data → Develop and evaluate model → Deploy model
   - ◯ Prepare data → Design UI → Train model→ Collect feedback
   - ◯ Define problem → Prepare data → Deploy model
   - ◯ Prepare model → Clean model → Visualize model → Report model

3. In a machine learning project, you are aggregating user behavior logs into a central table. What does this step indicate?

   - ◯ Providing new recommendations
   - ◯ Draft PowerPoint slides for marketing
   - ◯ Building a user dashboard
   - ◯ Transforming data before training the model

4. Sana is designing statistical plots for visualizing trends in her dataset and wants to leverage a Python tool designed specifically for this purpose. Which tool should she use to customize plots and visualizations?

- ◯ Scikit-learn
- ◯ Matplotlib
- ◯ Pandas
- ◯ Python

5. Twinkle wants to train and evaluate a machine learning model using Python and prefers a library with built-in support. Which of the following statements best describes the capabilities of a scikit-learn library?

- ◯ Scikit-learn preprocesses data, trains, evaluates, and exports data.
- ◯ Scikit-learn works with unstructured video and audio data.
- ◯ Scikit-learn generates AI art and images before data preprocessing.
- ◯ Scikit-learn plots and graphs to gain data insights and visualization.

6. Which Python library is commonly used for data manipulation and analysis in machine learning?

- ◯ Matplotlib
- ◯ Keras
- ◯ Pandas
- ◯ NumPy

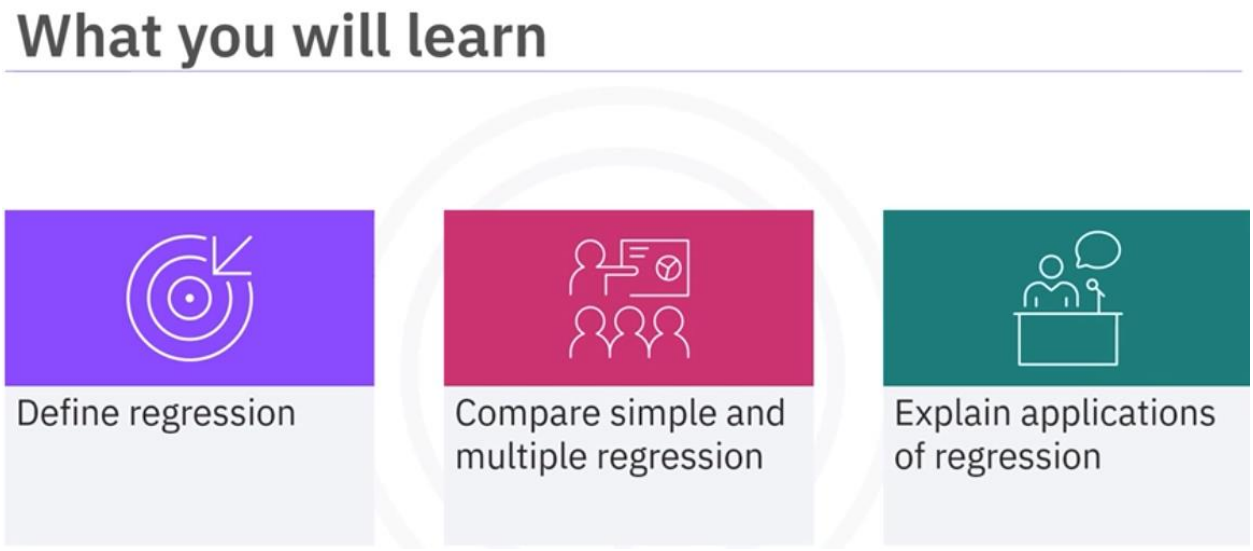
7. Pristin is training a machine learning model using Scikit-learn but notices inconsistent accuracy. She decides to adjust values like the number of neighbors in KNN and the regularization strength in logistic regression. What is the purpose of this step?

- ◯ To scale the data
- ◯ To change the model's architecture
- ◯ To adjust the model's internal parameters automatically
- ◯ To fine-tune the parameters that control the model's behavior

Module 2 Linear and Logistic Regression

In this module, you will explore two essential regression techniques used in machine learning—linear and logistic regression. You'll explain the role of regression in predicting outcomes, describe the differences between simple and multiple linear regression, and apply both using scikit-learn on real-world data. You will also interpret how polynomial and non-linear regression models capture complex patterns. The module introduces logistic regression as a classification method and guides you in training and testing classification models effectively. To support your learning, you'll receive a Cheat Sheet: Linear and Logistic Regression that summarizes key concepts, formulas, and use cases.

Introduction to Regression



Regression

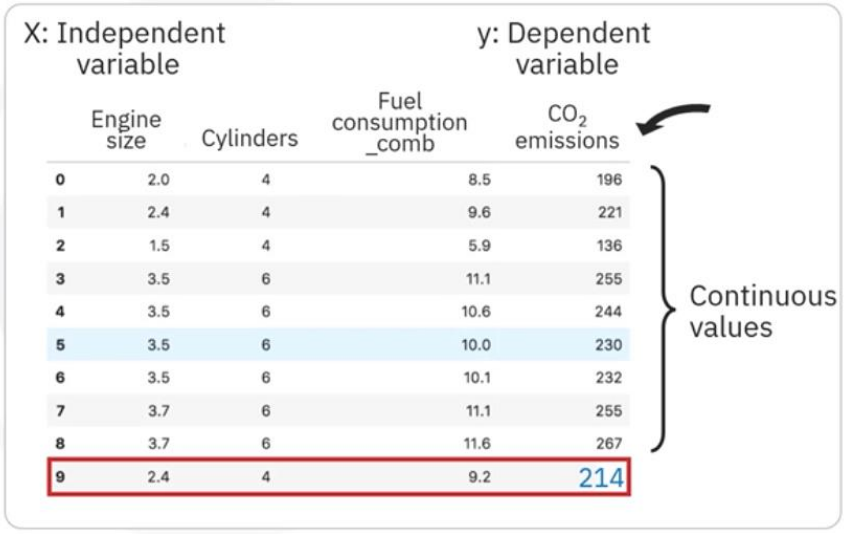
Regression is a type of **supervised learning model**.

It models a **relationship** between a **continuous target variable and explanatory features**.

Consider this dataset related to CO2 emissions from different cars. The features in this dataset include engine size, number of cylinders, fuel consumption, and CO2 emissions from various automobile models. Given this dataset, is it possible to predict the CO2 emission of a new car from the listed features? The answer is yes.

Linear regression

- Supervised learning model
- Models a relationship between a continuous target variable and explanatory features



	Engine size	Cylinders	Fuel consumption _comb	CO_2 emissions
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	214

Using potentially predictive features, you can use regression to predict a continuous value, such as CO2 emissions.

Let's consider the previous dataset comprising some features in **past cars**. Then, from that data, a model can be trained to estimate the CO2 emissions of each car. Let's use regression to build such a predictive model. Then, the model is used **to predict the expected CO2 emission for a new** or hypothetical car.

Simple vs multiple regression

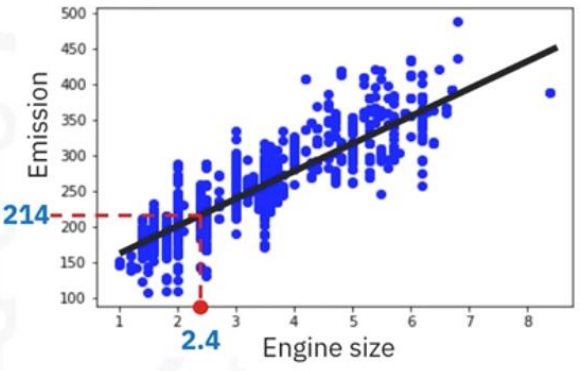
Deciding which one to use depends on the data you have for the dependent variable and the type of model that provides the best fit.

Simple regression

Two types

Simple linear regression

Simple nonlinear regression

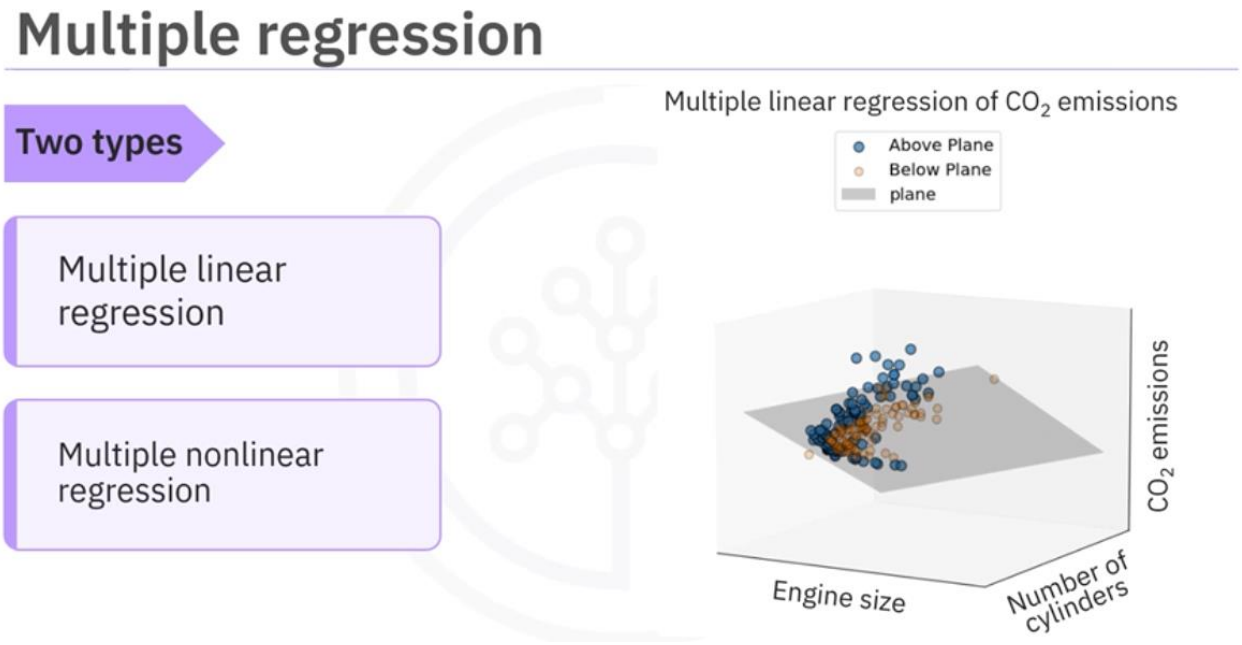


Simple regression is when a single independent variable estimates a dependent variable.

Simple regression can be linear or nonlinear.

For example, predicting CO2 emission using the variable engine size.

Simple linear regression imposes a linear relationship between the dependent and independent variables. Similarly, nonlinear regression creates a nonlinear relationship between those variables.



When **more than one independent variable** is present, the process is called multiple regression.

The distinctions between linear and nonlinear relationships apply to simple and multiple regression.

For example, predicting CO2 emission using engine size and the number of cylinders in any given car. Again, depending on the relation between dependent and independent variables, it can be either linear or nonlinear regression.

Applications

Essentially, we use regression when we want to **estimate a continuous value**.

- **Sales forecasting**: You can predict a salesperson's yearly sales from independent variables, such as customers, number of leads, and order history.
- **Price Prediction**: Regression analysis can predict the price of a house in an area based on its size, number of bedrooms, and so on.

- **Predictive Maintenance**: Predict when an automobile or an industrial machine will require maintenance rather than waiting for it to fail.
- **Employment income**: Predict from independent variables, such as hours of work, education, occupation, sex, age, years of experience, and so on.
- **Rainfall estimation**: based on meteorological factors, such as temperature, humidity, wind speed, and air pressure.
- **Probability and severity of wildfires**:
- **Infectious diseases**: Predict the spread of infectious diseases.
- **Chronic disease**: Estimate the likelihood of developing diseases, such as diabetes, heart disease, or cancer, based on patient data.

Indeed, you can find many examples of the usefulness of regression analysis in these and many other fields or domains, such as finance, healthcare, and retail.

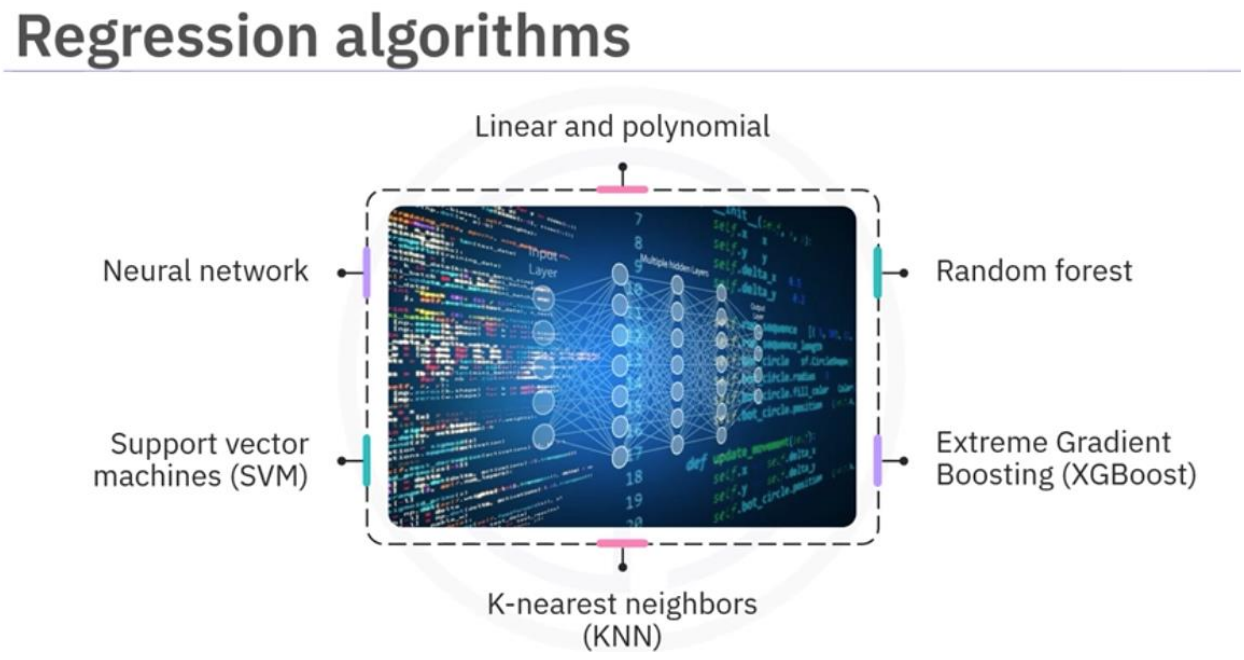
Regression algorithms.

Each algorithm is important in the appropriate context and suited to specific conditions.

Linear and polynomial regression are **classical** statistical modeling methods.

Random forest and XGBoost are **modern** machine learning regression models.

Other modern regression algorithms include k-nearest neighbors, support vector machines, and neural networks.



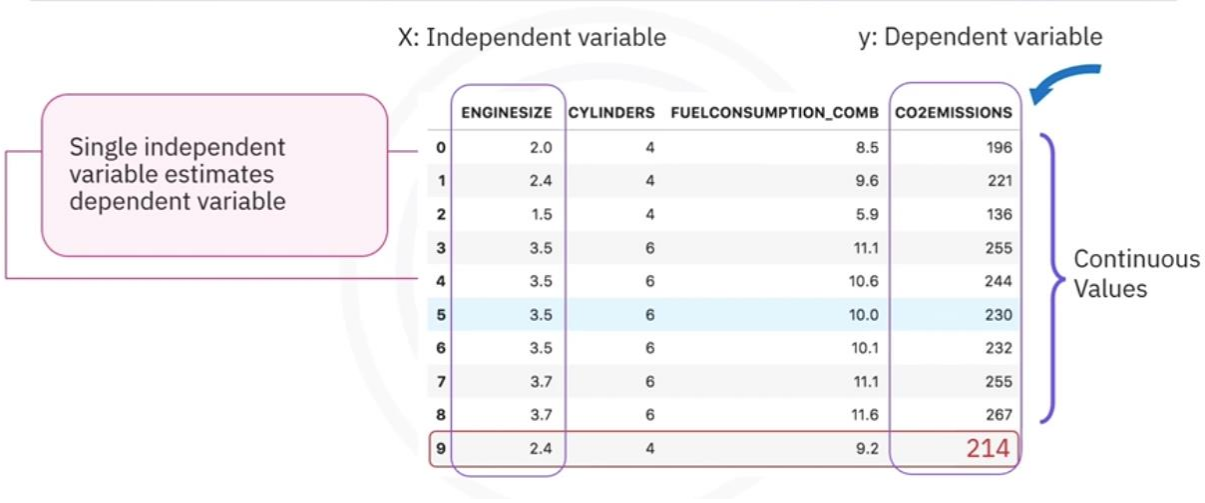
Recap

- Learned regression is a machine-learning technique that models a relationship between a continuous target variable and explanatory features.
- Simple regression is when a single independent variable estimates a dependent variable. This regression can be linear or nonlinear.
- When more than one independent variable is present, the process is called multiple regression.
- Applications of regression: forecast sales, predict maintenance expenses, estimate rainfall, and spread of infectious disease.

Introduction to Simple Linear Regression

Linear regression models a linear **relationship between a continuous target variable and explanatory features**.

In simple linear regression, a single independent variable estimates the dependent variable.

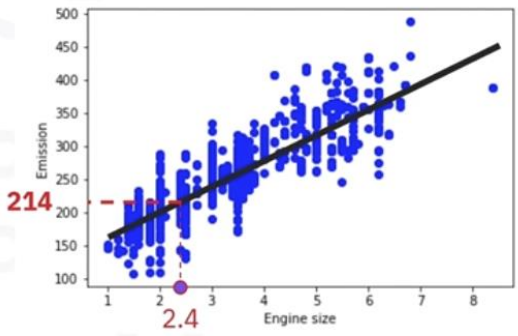


	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	214

For example, in our dataset, CO2 emission is predicted using the engine size variable.

Let's plot engine size as an independent variable and CO2 emissions as the target value that we want to predict.

	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	?

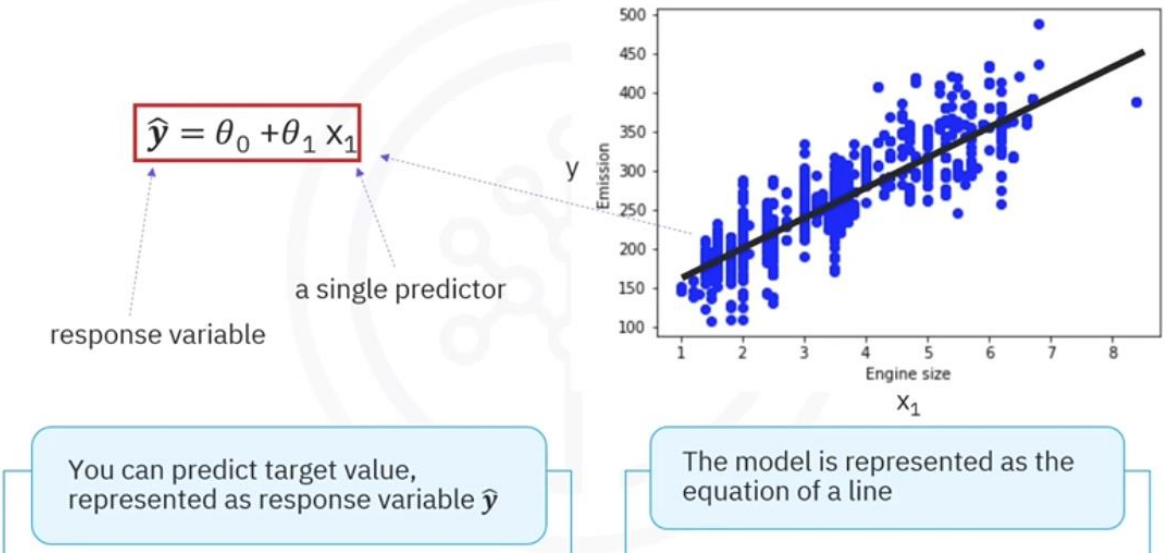


A **scatter plot** clearly **shows the correlation** between variables where **changes in one variable explain or possibly cause changes in the other variable**.

With simple linear regression, you can determine a **best-fit line through** the data. As engine size increases, so does CO2 emissions, and the relationship is approximately linear.

You can use simple linear regression to **predict the emission of an unknown car**.For example, the predicted emission for a sample car with an engine size of 2.4 is 214.

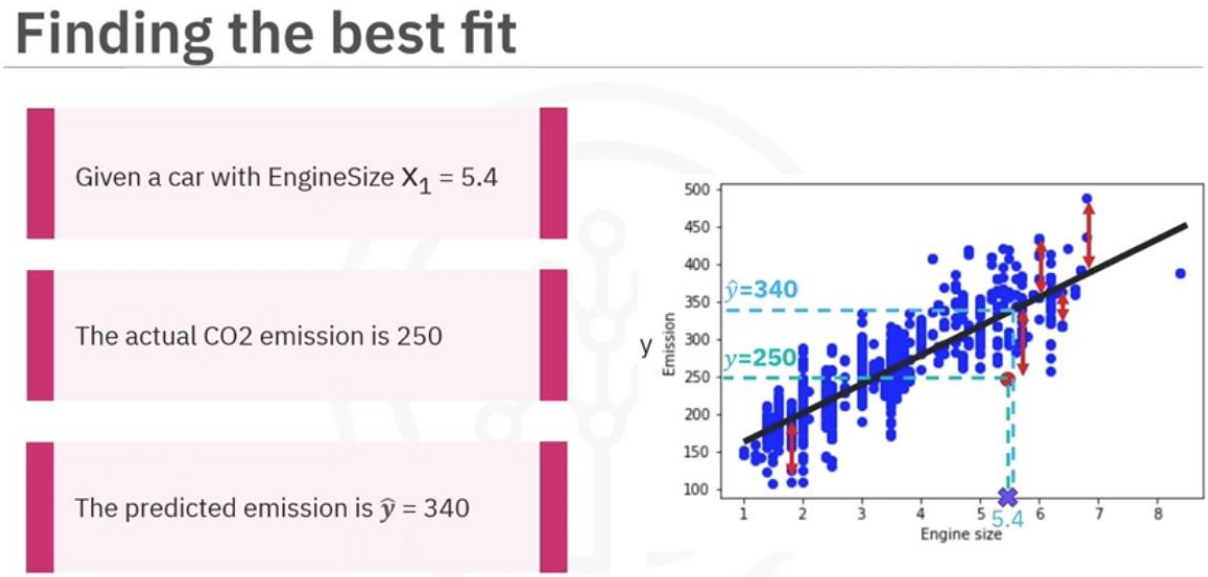
Representing the simple linear regression model



You can predict the target value, CO2 emissions, represented as the response variable, y-hat.

The **independent variable**, which in this case is engine size, is represented by the **single predictor variable, x1**. The model is represented as the equation of a line here**. y-hat is the predicted response** expressed in terms of x1 using a **y-intercept**, theta zero, and a **slope**,

theta one. **Theta zero and theta one are called the coefficients** of the linear regression model, selected by the linear regression algorithm to determine a best-fit line.



Let's consider some new data points and check how well they align with the regression line.

Given a car with engine size x1 equals 5.4, its actual CO2 emission is 50, while its predicted emission is y-hat equals 340.Comparing the actual value to the predicted one, there's a 90-unit discrepancy. **The residual error is the vertical distance from the data point to the fitted regression line.** The average of all residual errors measures how poorly the regression line fits the data. Mathematically, it can be shown by the equation **mean squared error**, shown as MSE. Linear regression aims to **find the line for minimizing the mean of all these residual errors**. This form of regression is commonly known as **ordinary least squares regression, or OLS regression**.

Estimating the coefficients of the linear regression model

	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267

X_1 (ENGINESIZE) y (CO2EMISSIONS)

We can use two formulas to calculate coefficients θ_0 and θ_1

It requires that we calculate the means, y bar $[\bar{y}]$ and x bar $[\bar{x}]$, of the independent and dependent variables

We can use two formulas to calculate the coefficients theta zero and theta one of the linear regression model. The solution was derived independently by the mathematicians Gauss and Legendre in the early 1800s.

It requires that we **calculate the means**, y-bar(226.2) and x-bar (3.0), of the independent and dependent variables. The xi and yi in the equation for theta one refer to the ith values of x and y. Then, going through the calculations, you'll find that theta one equals 39. You can use this result to calculate the first parameter line intercept as 125.7.

So, these are the two parameters for the line, where **theta zero is also called the bias coefficient** and **theta one is the coefficient** for the CO2 emission column.

Predictions with linear regression

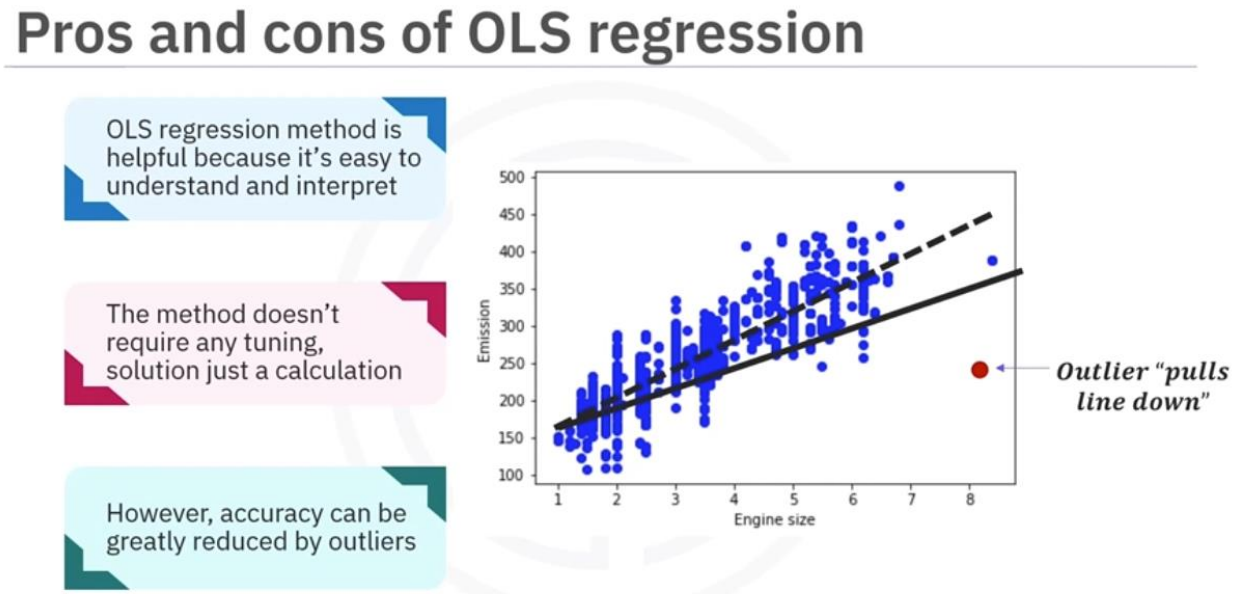
	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	?

You can predict CO2Emission from EngineSize using the following equation:

$$CO2Emission = 125 + 39 \times EngineSize$$

Given the parameters of the linear equation, making a prediction is as simple as solving the

equation for a particular input value. For example, you can predict the CO2 emission from engine size for the automobile in record number 9 using the following equation. For an engine size of 2.4, we can predict that the CO2 emission of the car would be 214.



The OLS regression method is helpful because it's easy to understand and interpret. The method **doesn't require any tuning** and **its solution is just a calculation**. This also makes OLS regression **fast, especially for smaller datasets**.

On the other hand, a linear model may be far **too simplistic to capture complexity**, such as a nonlinear relationship in the data.Outliers can greatly reduce its accuracy, giving them far too much weight in the calculations.

Recap

In this video, we looked at several use cases of simple linear regression.

We learned how to predict a continuous value, such as a car's CO2 emissions.

In simple linear regression, **a single independent variable estimates the dependent variable**.

We also learned how to determine the best fit line through a chart showing regression values.

We learned about the concept of **Mean Squared Error, or MSE**, which **measures how poorly the regression line fits the data**.

Linear regression aims to find the line for minimizing the mean of all these residual errors.

This form of regression is commonly known as **Ordinary Least Squares Regression**, or OLS regression. The OLS regression method is useful because it's easy to understand and interpret.

However, **outliers can greatly reduce its accuracy**, giving them far too much weight in the calculations.

Lab: Simple Linear Regression

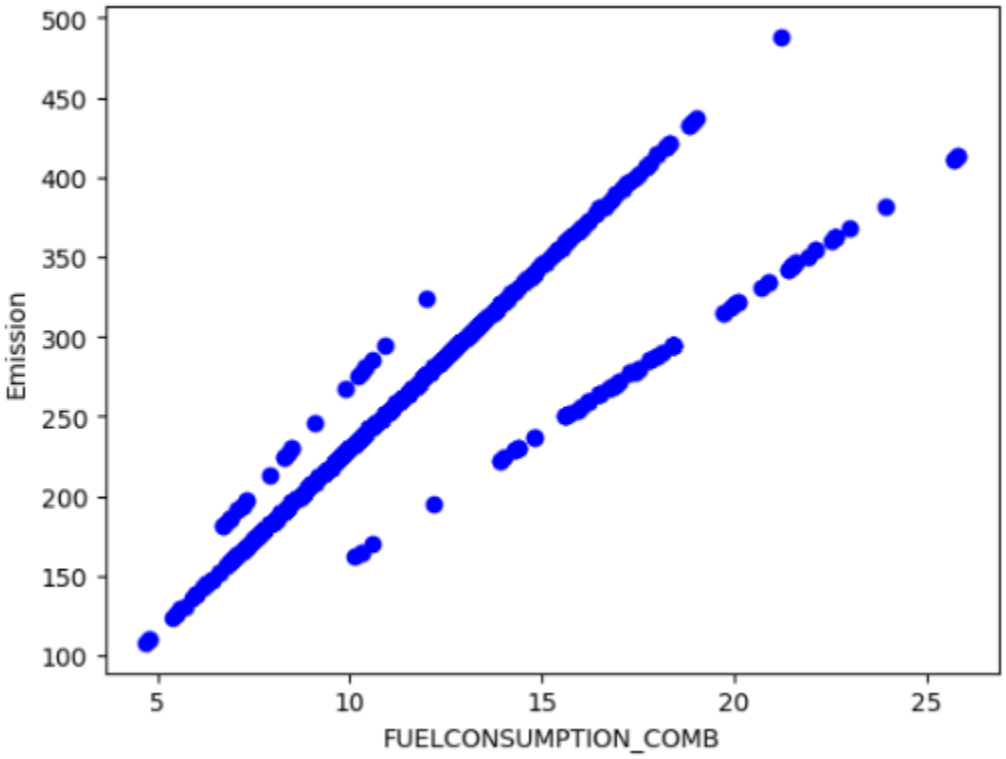
Importa versions concretas de los librerias

```
!pip install numpy==2.2.0
!pip install pandas==2.2.3
!pip install scikit-learn==1.6.0
!pip install matplotlib==3.9.3
```

Diferencia entre df.head(5) (las primeras) y df.sample(5) (aletorias)

Las variables con std=0 tienen un único valor y no aportan.

UN Scatter Plot con tres relaciones lineales.



Del dataframe, las features se conviertes a series numpy.

```
X = cdf.ENGINESIZE.to_numpy()
y = cdf.CO2EMISSIONS.to_numpy()
```

Splitting training and test data sets, The smaller your data, the larger your training set needs to be because it's easier to find spurious patterns in smaller data The downside is that your evaluation of generalizability will have less reliability.

Reshape:

```
type(X_train), np.shape(X_train), np.shape(X_test), np.shape(y_train), np.shape(y_test),
```

```
(numpy.ndarray, (853,), (214,), (853,), (214,))
```

```
# train the model on the training data
# X_train is a 1-D array but sklearn models expect a 2D array as input for the training data, with shape (n_observations, n_features).
# So we need to reshape it. We can let it infer the number of observations using '-1'.
regressor.fit(X_train.reshape(-1, 1), y_train)
```

X_train is a 1-D array but sklearn models expect a 2D array as input for the training data, with shape (n_observations, n_features). So we need to reshape it. We can let it infer the number of observations using '-1'.

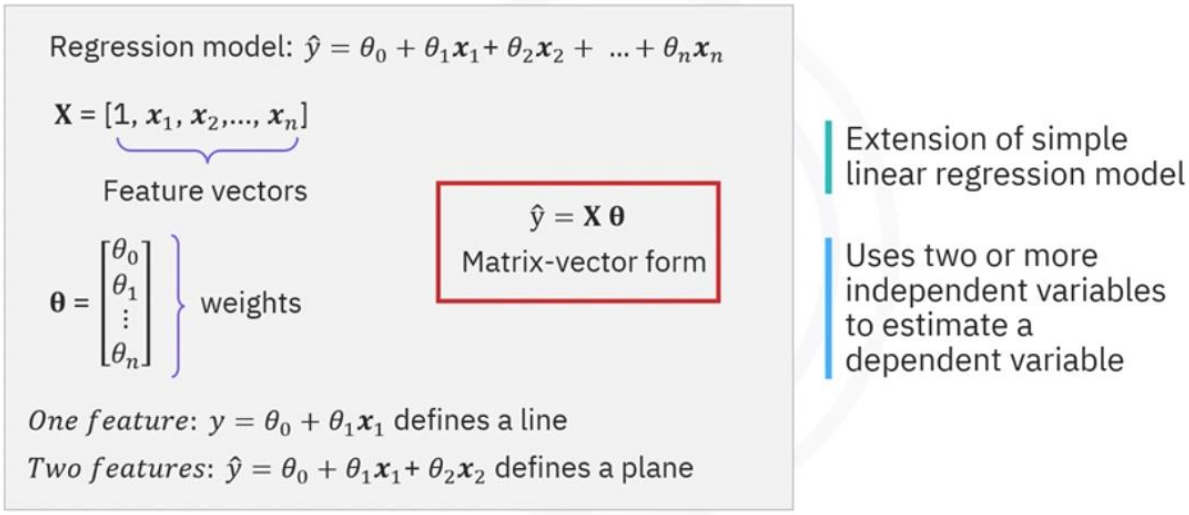
Model Evaluation

- Mean Absolute Error: It is the mean of the absolute value of the errors. This is the easiest of the metrics to understand since it’s just an average error.
- Mean Squared Error (MSE): MSE is the mean of the squared error. In fact, it's the metric used by the model to find the best fit line, and for that reason, it is also called the residual sum of squares.
- Root Mean Squared Error (RMSE). RMSE simply **transforms the MSE into the same units as the variables being compared**, which can make it easier to interpret.
- R-squared is not an error but rather a popular metric used to estimate the performance of your regression model. It represents **how close the data points are to the fitted regression line**. The higher the R-squared value, the better the model fits your data. The best possible score is 1.0 and it can be negative (because the model can be arbitrarily worse).

Multiple Linear Regression

Describe multiple linear regression.

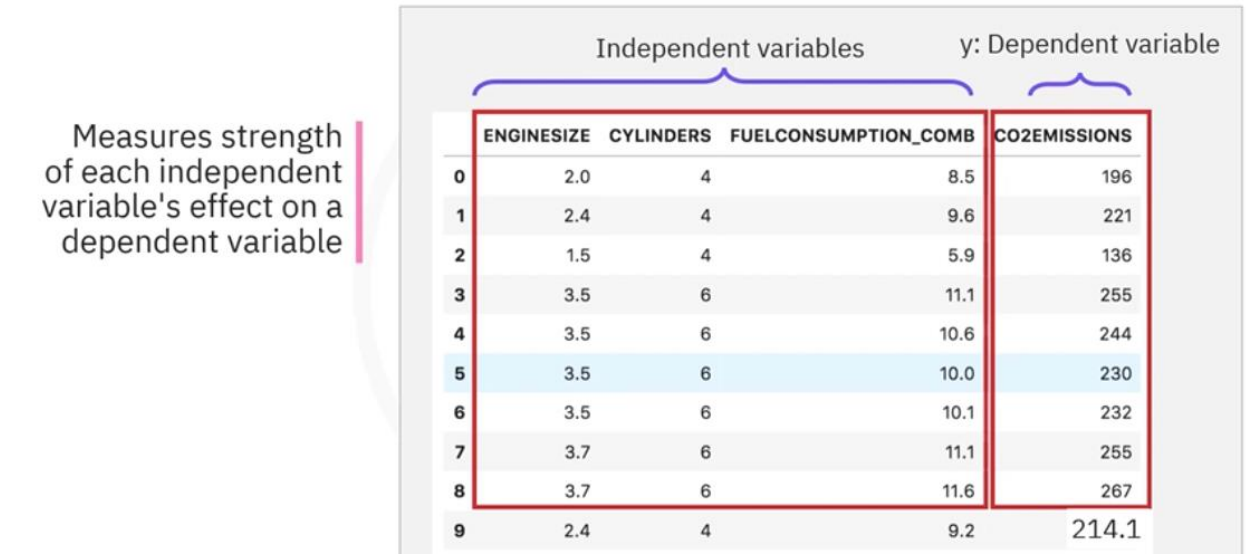
Multiple linear regression is an extension of the simple linear regression model. It uses **two or more independent variables** to estimate a dependent variable.



Mathematically, the multiple regression model is a linear combination of the form y hat equals theta zero plus theta one x one, where the x one are the **feature vectors** that can be represented as a **matrix x** that includes a **constant value of one in the first entry** to account for the bias or intercept term theta underscore zero, and the **thetas are the unknown weights**, which can be represented as matrix theta.

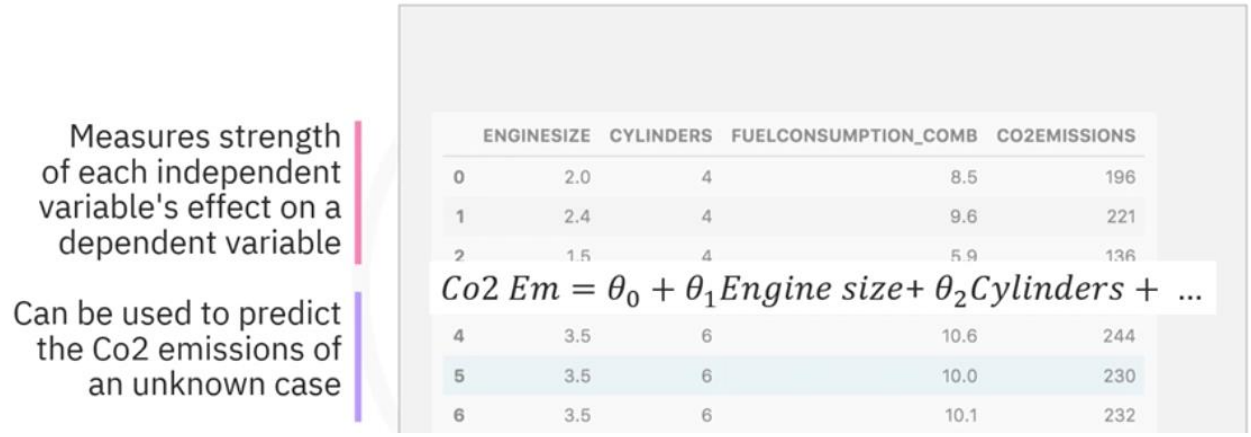
Let's consider the data set in this table. Multiple linear regression can be used to measure the **strength of each independent variable's effect on a dependent variable**.

You can predict the CO2 emission of a motor car from features like engine size, number of cylinders, and fuel consumption by forming a linear combination of the features using trained weights, theta i.



	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
3	3.5	6	11.1	255
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232
7	3.7	6	11.1	255
8	3.7	6	11.6	267
9	2.4	4	9.2	214.1

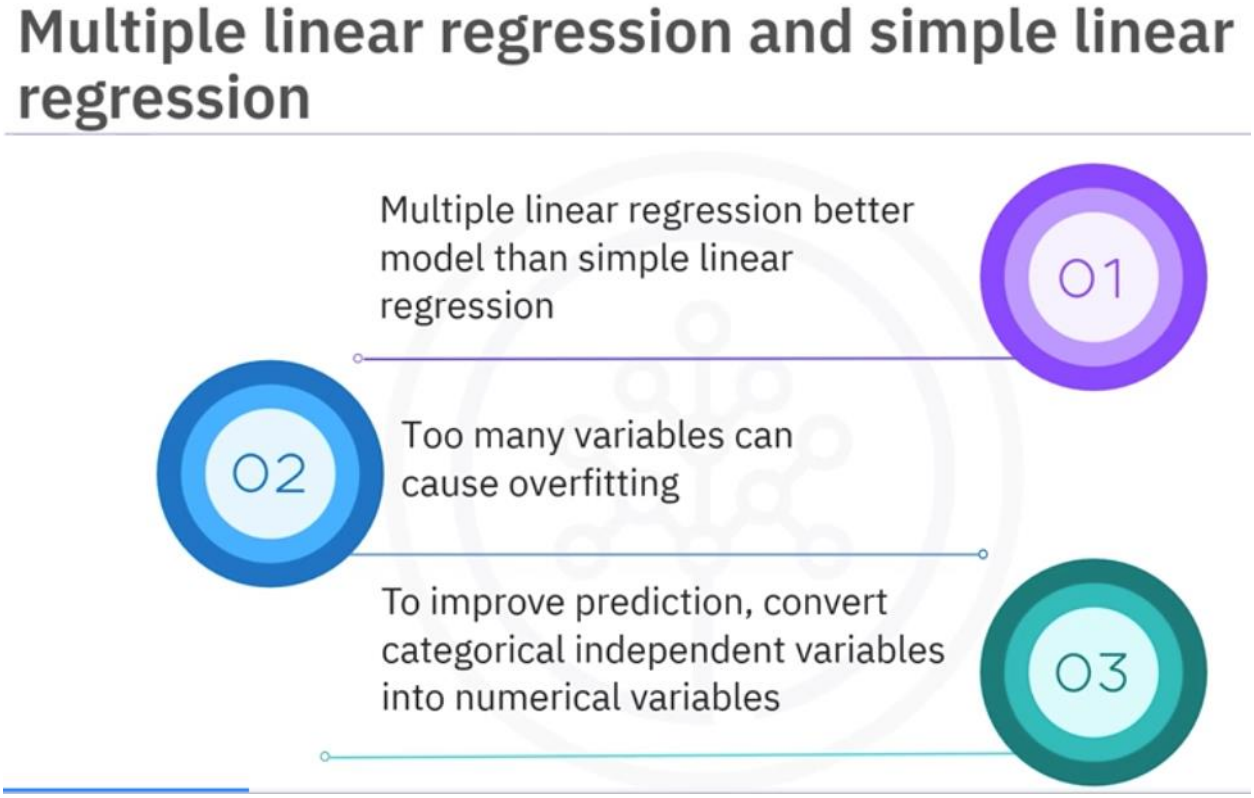
You can use the trained model to predict the expected CO2 emission of an unknown case, such as record number 9.



	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_COMB	CO2EMISSIONS
0	2.0	4	8.5	196
1	2.4	4	9.6	221
2	1.5	4	5.9	136
4	3.5	6	10.6	244
5	3.5	6	10.0	230
6	3.5	6	10.1	232

Compare multiple linear regression and simple linear regression

Multiple linear regression results in a **better model** than using a simple linear regression. However, adding too **many variables** can cause your model to **overfit or essentially memorize the training data, making it a poor predictor for unseen observations**.



To improve prediction, **categorical independent variables can be incorporated** into a regression model by converting them into numerical variables. For example, given a binary variable such as car type, the code zero for manual and one for automatic cars can be substituted to make it numerical. For a categorical variable with more than two classes, you can opt to transform it into new Boolean features, one for each class.

Multiple linear regression has applications in every industry.

It is widely used in the education sector to **predict outcomes** and **explain relationships between variables**. For example, do revision time, test anxiety, lecture attendance, and gender affect student exam performance?

Multiple linear regression can also be used to predict the impact of changes in what-if scenarios. **What-if scenarios** involve hypothetical **changes to one or more of your model's input features to see the predicted outcome**. For example, suppose you were reviewing a person's health data. In that case, multiple linear regression might be able to tell you how much that person's blood pressure would rise or fall for every change in a patient's body mass index, BMI.

List the pitfalls of multiple linear regression.

The what-if scenario can sometimes **provide inaccurate findings in the following situations**.

You might **consider impossible scenarios** for your model **to obtain predictions**. You might extrapolate scenarios that **are too distant from the realm of data it was trained** on.

Your model might depend on more than one variable amongst a group of correlated or collinear variables. When two **variables are correlated**, they are **no longer independent** variables because they are predictors of each other. **They are collinear**. You can perform a what-if scenario with a linear regression model by changing a single variable while holding all other variables constant**. However, if the variable is correlated with another feature, then this is not feasible because the other variable must also change realistically**. The solution for avoiding pitfalls from correlated variables is to **remove any redundant variables** from the regression analyses.

To build your multiple regression model, you must **select your variables** using a balanced approach considering uncorrelated variables, which are most understood, controllable, and most correlated with the target.

Multiple linear regression assigns **a relative importance to each feature**. Imagine you are predicting CO2 emission, or Y, from other variables for the automobile in record number 9. Once you find the parameters, you can plug them into the linear model equation model.

For example, let's use theta 0 equals 125, theta 1 equals 6.2, theta 2 equals 14, and so on.

$$\theta = (125, 6.2, 14, \ldots)$$

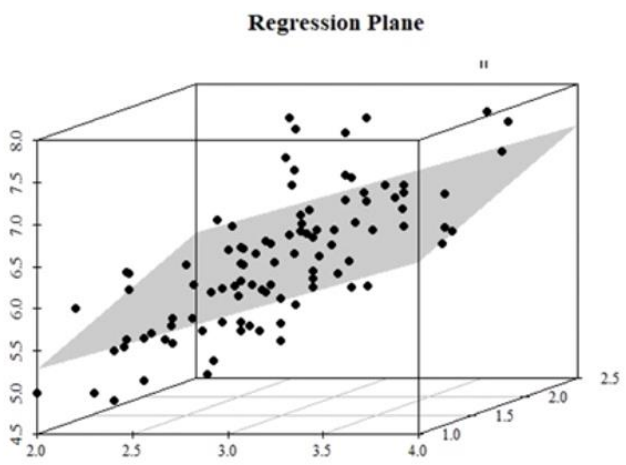
$$\hat{y} = 125 + 6.2x_1 + 14x_2 +$$

$$Co2Em = 125 + 6.2EngSize + 14 \text{ Cylinders} + \ldots$$

If we map these values to our dataset, we can rewrite the linear model as CO2 emission equals 125 plus 6.2, multiplied by engine size plus 14, multiplied by cylinder, and so on. Now, let's plug in the 9th row of R and calculate the CO2 emissions for a car with a 2.4L engine. So CO2 emission equals 125 plus 6.2 times 2.4 plus 14 times 4, and so on. We can predict the CO2 emission for this specific car will be 214.1.

For a **simple linear regression**, where there is only one feature vector, the regression in the equation defines a **line**. For multiple linear **regression using two features**, the solution describes a **plane**. **Beyond two dimensions**, it describes a **hyperplane**.

Fitting a hyperplane



Simple linear regression: Regression equation defines a line

Multiple linear regression using two features: Solution describes a plane

Beyond two dimensions: Solution describes a hyperplane

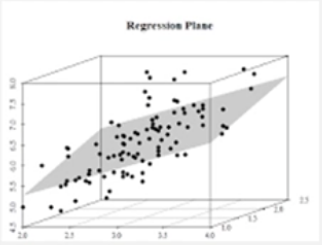
Like with simple linear regression**, the values in the weight vector theta can be determined by minimizing the mean square prediction error**.

Given a set of parameters, consider a linear model based on the linear combination of the parameters with the features. You can measure the residual error for each car in the dataset as the difference between its true CO2 emission value and the value predicted by the model.

Model error

One feature: $y = \theta_0 + \theta_1 \boldsymbol{x}_1$ defines a line

Two features: $\hat{y} = \theta_0 + \theta_1 \boldsymbol{x}_1 + \theta_2 \boldsymbol{x}_2$ defines a plane



N features: $y = \theta_0 + \theta_1 \boldsymbol{x}_1 + \theta_2 \boldsymbol{x}_2 + \ldots + \theta_N \boldsymbol{x}_N$ an N−dimensional hyperplane

Residual error for each car in the data set = Difference between its true Co2 emission value and predicted value

Average of all residual errors indicates how poorly model predicts the actual values

For example, if the model predicts 140 as the value for the first car in the dataset, using the actual value of 196, you can see the residual error is 196 minus 140, or 56.The average of all the residual errors indicates how poorly the model predicts the actual values. This information is called **the mean squared error**, or MSE.

MSE is not the only way to expose the error of a linear model. However, it is the **most popular**. With this metric, the best model for the dataset is the one with the least squared error.

Least squares solution

$$\text{Minimizing MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Same as minimizing $\text{SE} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2$

==> least-squares linear regression

Factor of 1/n in the MSE equation unnecessary to minimize the error

Method is called least-squares linear regression

The factor of 1 slash n in the MSE equation isn't necessary to include to minimize the error, so this method is called **least squares linear regression**. So, multiple linear regression aims to minimize the MSE equation by finding the best parameters.

Estimating parameters for multiple linear regression

- Gradient descent starts optimization with random values for each coefficient
- Useful for a large data set

There are many ways to estimate the value of these coefficients.

Ordinary least squares and an optimization approach **are the most common methods**. Ordinary least squares estimate the values of the coefficients by minimizing the mean squared error. This approach uses **the data as a matrix and uses linear algebra operations to calculate the optimal values for theta**.

Another option is to use an **optimization algorithm to find the best parameters**. That is, you can use a process of optimizing the coefficients by **iteratively minimizing the model's error on your training data**. For example, you can use the gradient descent method, which **starts the optimization with random values for each coefficient**. Gradient descent is a good approach if you have **a large dataset**.

Recap.

Multiple linear regression is an **extension** of the **simple** linear regression model. It uses two or more independent variables to estimate a dependent variable.

It is widely used in the education sector to **predict outcomes** and **explain relationships between variables**.

Multiple linear regression can also be used to predict the impact of changes in **what-if scenarios**. Adding too many variables can cause your **model to overfit** or essentially memorize the training data, making it a poor predictor for unseen observations.

To build your multiple regression model, you must select your variables using a balanced approach, considering **uncorrelated variables**, which are most understood, controllable, and most correlated with the target.

There are many ways to estimate the parameters for multiple linear regression. However, **ordinary least squares and an optimization with random values approach** are the most common methods.

Lab: Multiple Linear Regression

Drops categorical variables and ones with unique values.

Analize how features correlate with target variable. Each exceeding 85% in magnitude are good candidates.

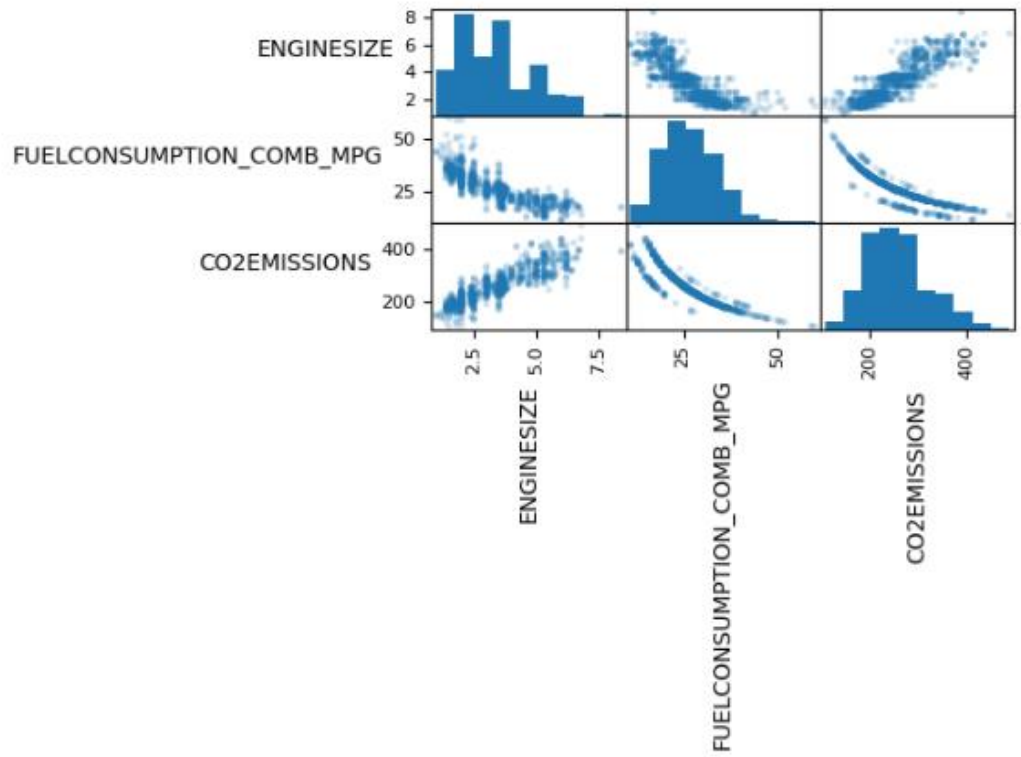
Analize pairs of features with high correlation magnitude to discard one in the pair.

[7]:

	ENGINESIZE	CYLINDERS	FUELCONSUMPTION_CITY	FUELCONSUMPTION_HWY	FUELCONSUMPTION_COMB	FUELCONSUMPTION_COMB_MPG	CO2EMISSIONS
ENGINESIZE	1.000000	0.934011	0.832225	0.778746	0.819482	-0.808554	0.874154
CYLINDERS	0.934011	1.000000	0.796473	0.724594	0.776788	-0.770430	0.849685
FUELCONSUMPTION_CITY	0.832225	0.796473	1.000000	0.965718	0.995542	-0.935613	0.898039
FUELCONSUMPTION_HWY	0.778746	0.724594	0.965718	1.000000	0.985804	-0.893809	0.861748
FUELCONSUMPTION_COMB	0.819482	0.776788	0.995542	0.985804	1.000000	-0.927965	0.892129
FUELCONSUMPTION_COMB_MPG	-0.808554	-0.770430	-0.935613	-0.893809	-0.927965	1.000000	-0.906394
CO2EMISSIONS	0.874154	0.849685	0.898039	0.861748	0.892129	-0.906394	1.000000

'ENGINESIZE' and 'CYLINDERS' are highly correlated, but 'ENGINESIZE' is more correlated with the target, so we can drop 'CYLINDERS'.

Similarly, each of the four fuel economy variables is highly correlated with each other. Since FUELCONSUMPTION_COMB_MPG is the most correlated with the target, you can drop the others: 'FUELCONSUMPTION_CITY,' 'FUELCONSUMPTION_HWY,' 'FUELCONSUMPTION_COMB.'



As you can see, the relationship between 'FUELCONSUMPTION_COMB_MPG' and 'CO2EMISSIONS' is non-linear.

Recommends separate scalation for train set and for test set.

Unstandardize the model brings an abstract model to a real one.

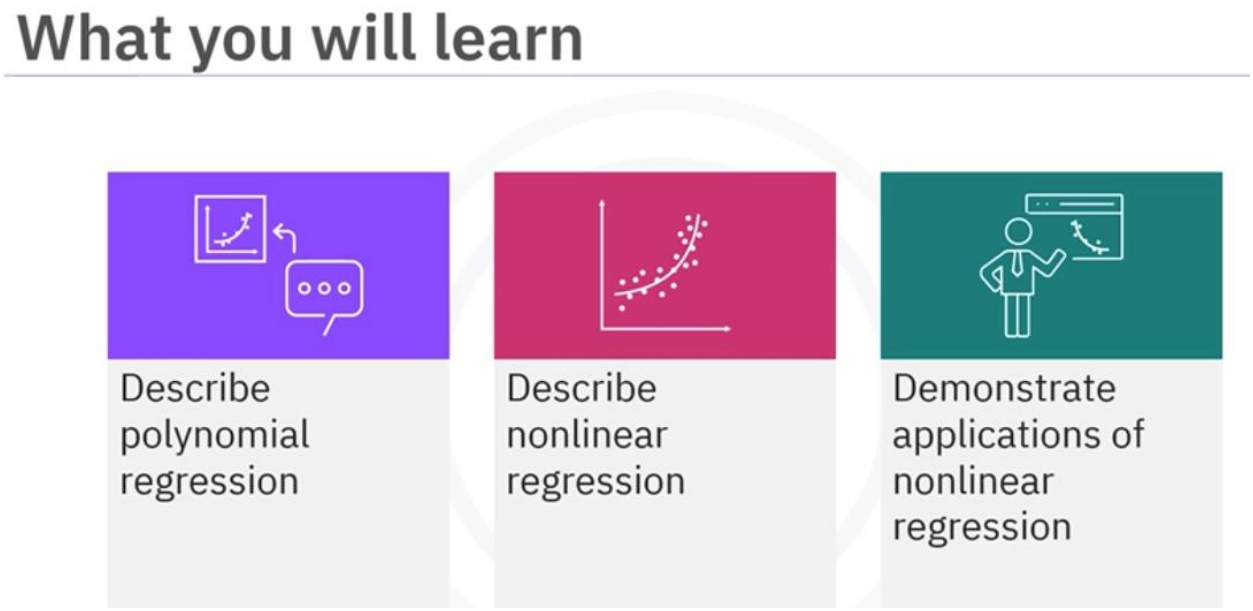
```
# Get the standard scaler's mean and standard deviation parameters
means_ = std_scaler.mean_
std_devs_ = np.sqrt(std_scaler.var_)

# The least squares parameters can be calculated relative to the original, unstandardized feature space as:
coef_original = coef_ / std_devs_
intercept_original = intercept_ - np.sum((means_ * coef_) / std_devs_)

print ('Coefficients: ', coef_original)
print ('Intercept: ', intercept_original)
```

```
Coefficients:  [[17.8581369  -5.01502179]]
Intercept:  [329.1363967]
```

You would expect that for the limiting case of zero ENGINESIZE and zero FUELCONSUMPTION_COMB_MPG, the resulting CO2 emissions should also be zero. This is **inconsistent** with the 'best fit' hyperplane, which **has a non-zero intercept** of 329 g/km.

The answer must be that the **target variable does not have a very strong linear relationship to the dependent variables**, and/or the data has outliers that are biasing the result. **Outliers** can be handled in preprocessing, or as you will learn about later in the course, by using regularization techniques. One or more of the variables might have a nonlinear relationship to the target. Or there may still be **some colinearity amongst the input variables**.

Polynomial and Non-Linear Regression

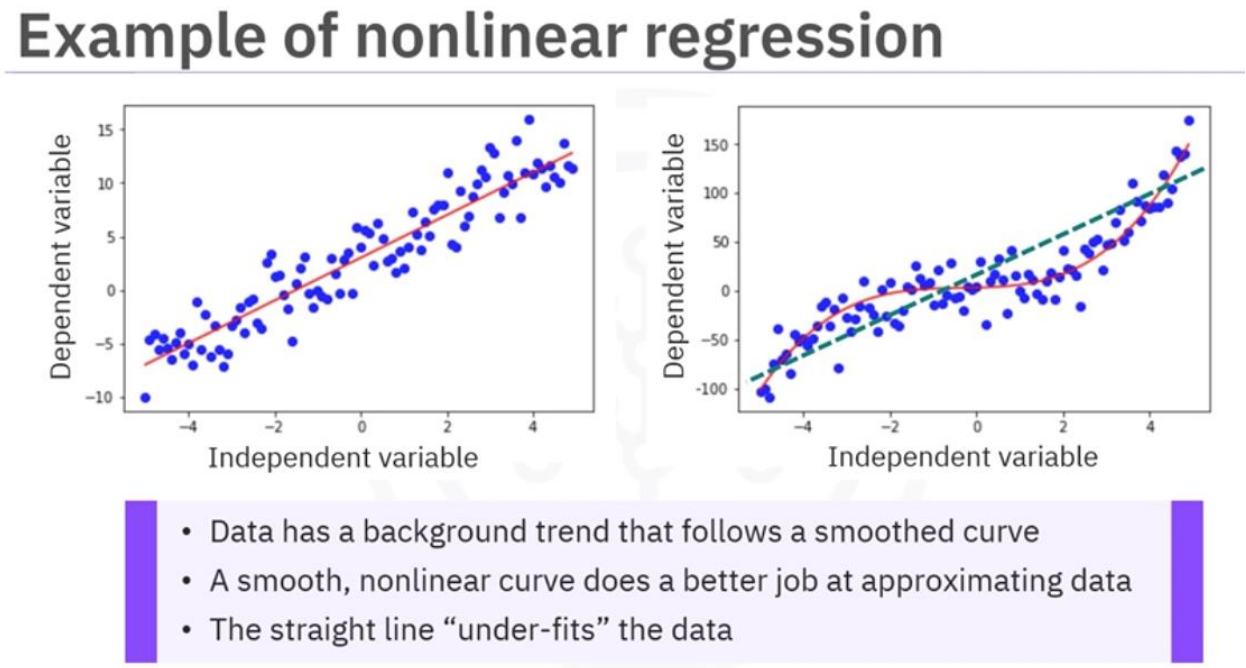


Nonlinear regression

Nonlinear regression is a statistical method for modeling the relationship between a dependent variable and one or more independent variables, where the **relationship is**

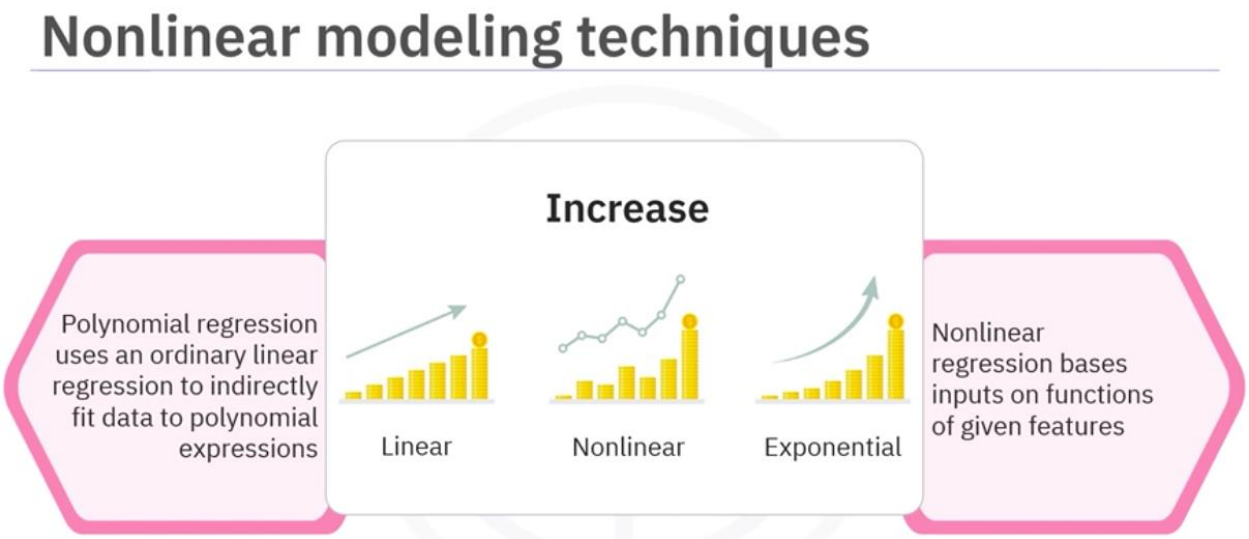
represented by a nonlinear equation. (polynomial, exponential, logarithmic, or any other not using linear parameters).

Nonlinear regression is **useful** when there is a complex **relationship** between variables that **cannot be captured through a straight line**. For instance, you would use nonlinear regression when you are using a dataset that follows an exponential growth pattern.



Real-world data, the relationship between your input and target variables is rarely linear. More commonly, your data has a **background trend** that follows a **smoothed curve** rather than a straight line. Like in the right chart, clearly a smooth nonlinear curve does a better job at approximating data than the straight line. **The straight line underfits the data**.

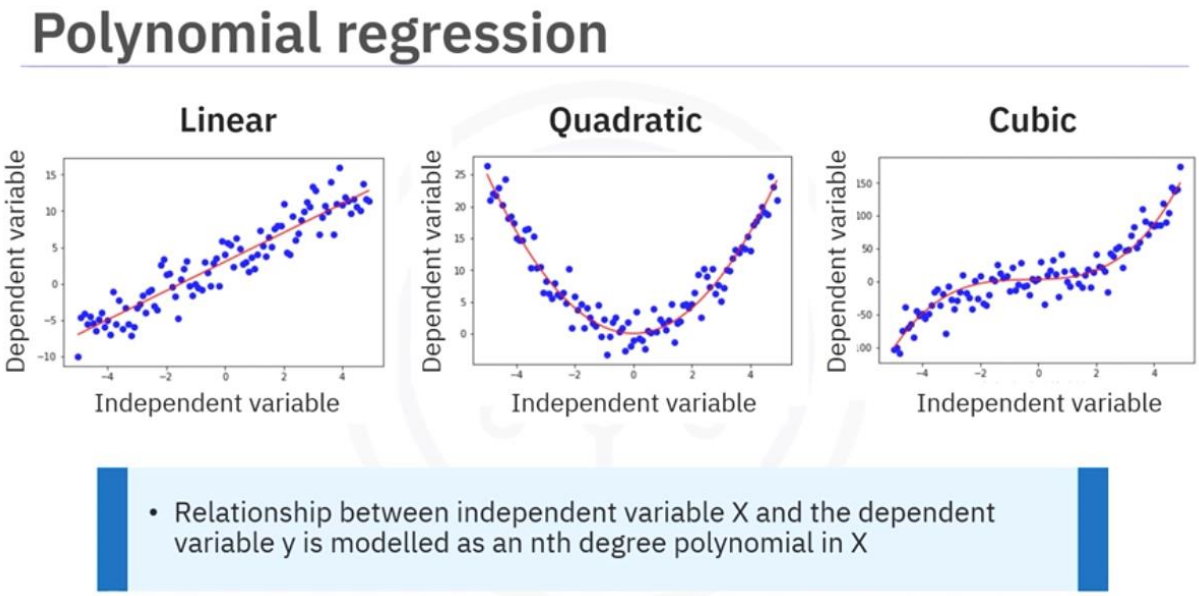
You can use many kinds of nonlinear regression methods to model your dataset.



- **Polynomial regression** uses an ordinary linear regression to indirectly fit your data to **polynomial expressions of the features**, rather than the features themselves.
- **Nonlinear regression** follows the same idea, but bases its inputs on **functions of the given features**, such as the logarithm or exponential of the features.

Nonlinear regression doesn't necessarily reduce to linear regression like polynomial regression does.

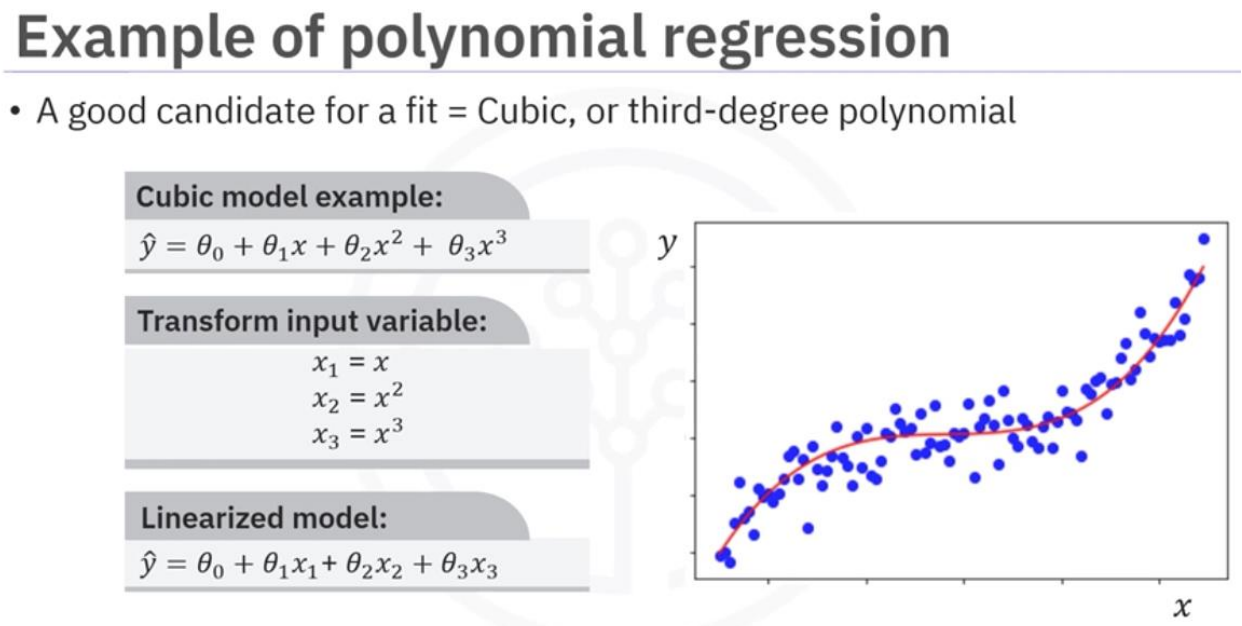
Polynomial regression



In this chart, you can see linear, quadratic, and cubic regression curves that fit the patterns in the data quite well. And this approach can go on and on to polynomials of arbitrary degree.

We can call all of these polynomial regression, where the **relationship** between the independent variable x and the dependent variable y is **modeled as an nth degree polynomial** in independent variable **x**.

Consider this polynomial regression example where a good candidate for a fit is a **cubic or third degree polynomial**, given by y equals theta zero plus theta one x plus theta two x squared plus theta three x cubed.



Here, the thetas are parameters to be estimated that best fit the underlying data, by **introducing new variables** as x underscore one equals x, x underscore two equals x to the power of two, and x underscore three equals x to the power of three. The model can now be expressed as a **linear combination of the new variables** as y equals theta zero plus theta one x one plus theta two x two plus theta three x three.

Because the resulting **model has been linearized**, you can simply **use** ordinary multiple linear regression to find the best fit parameters.

Example of polynomial regression

- Multiple linear regression helps find best-fit parameters

Cubic model example:

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3$$

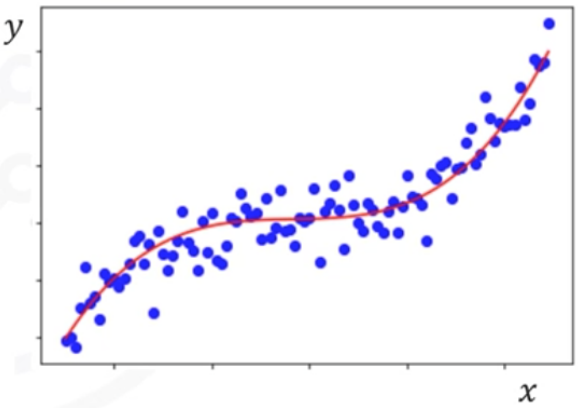
Transform input variable:

$$x_1 = x$$
$$x_2 = x^2$$
$$x_3 = x^3$$

Linearized model:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$

Apply multiple linear regression

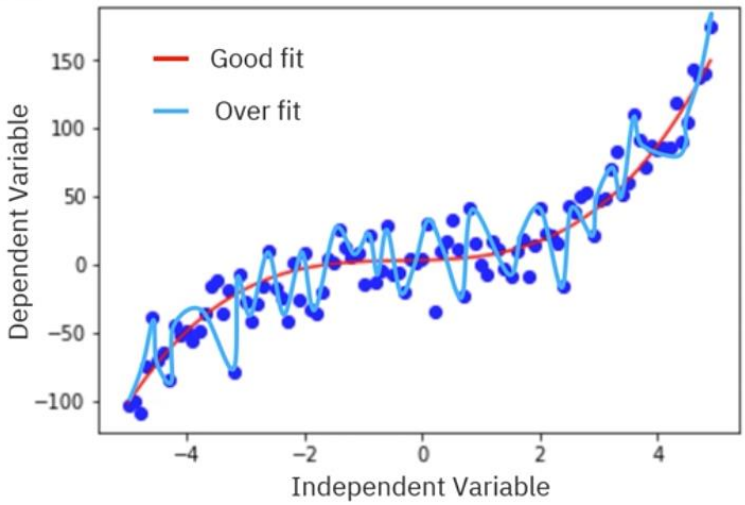


Given any finite sets of points, it's **always possible to find a polynomial of sufficiently high degree that will pass through every point**. Such a **perfect fit** amounts to **overfitting** as seen in this chart. The polynomial regression model **memorizes everything including any random noise** or large variations, rather than understanding the underlying patterns.

Overfitting with polynomials

Polynomial regression model 'memorizes' everything, including noise or variations

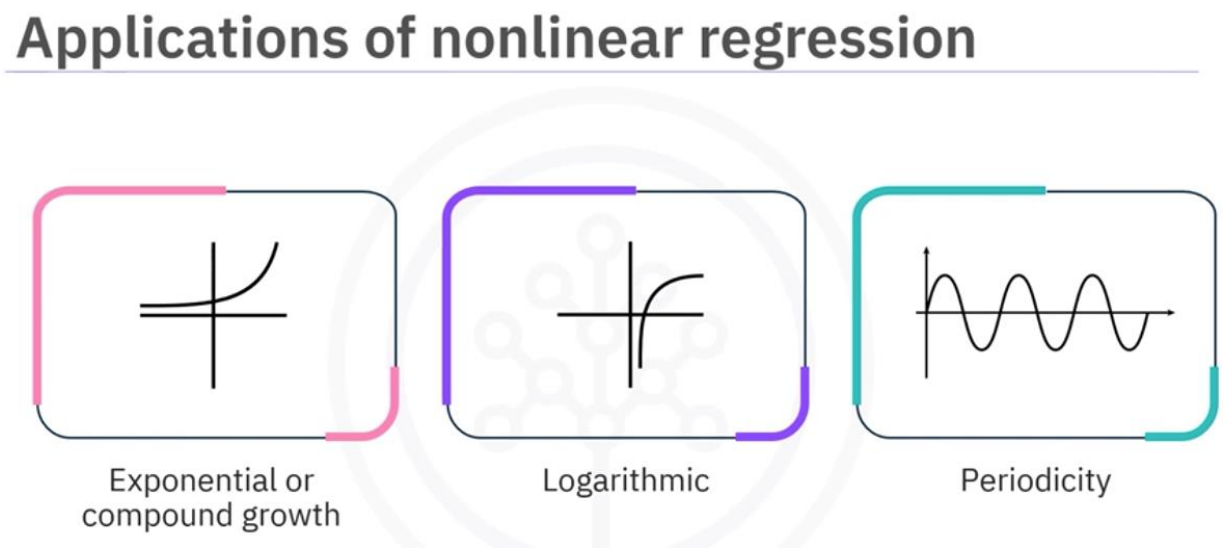
Pick a regression that fits data without overfitting



It's important to pick a regression that fits the data well without overfitting. You don't need to capture every fine detail, just the trend.

Polynomial regression is a **special form of nonlinear** regression. It expresses a nonlinear dependence on the input features, but it **has a linear dependence on the regression coefficients** because it can be transformed into a linear regression problem.

It is often simply called linear regression.



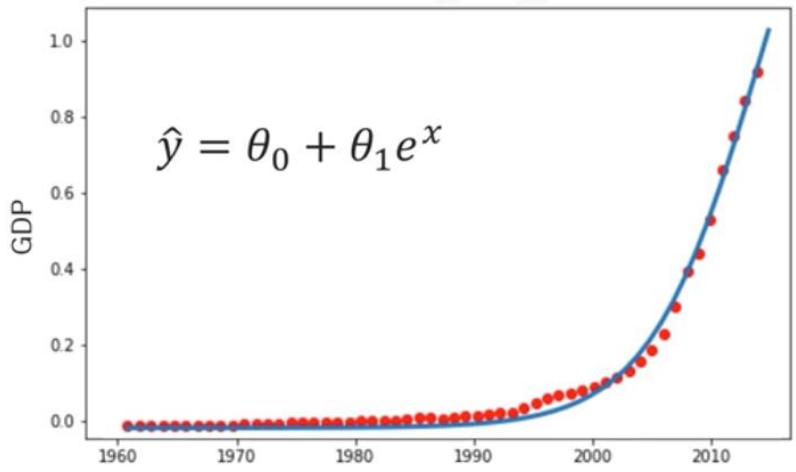
In contrast, there are many real-world complex nonlinear relationships that can't be modeled as polynomials. Such common examples of nonlinear regression include:

- E**xponential or compound growth** (how investments grow with compound interest rates.
- **Logarithmic** (.law of diminishing returns, how incremental gains in productivity or profit can reduce as investment in a production factor, such as labor increases).
- **Periodicity** (Sinusoidal seasonal variations in a quantity, such as monthly rainfall or temperature).

Let's consider this data corresponding to China's Gross Domestic Product, or GDP, from 1960 to 2014. Each row provides China's annual GDP in US dollars for the year. The scatter plot displays a strong dependence of GDP on time, but the relationship is nonlinear. As you can see, **GDP increases over time, and the rate of this growth also increases**. This increasing growth rate is characteristic of exponential growth. A reasonable regression model then uses an exponential function, like y-hat equals theta-zero plus theta-one, e-x.

Compound growth example

- Scatterplot displays strong dependence of GDP on time, but relationship is nonlinear
- GDP increasing over time, but growth rate is also increasing

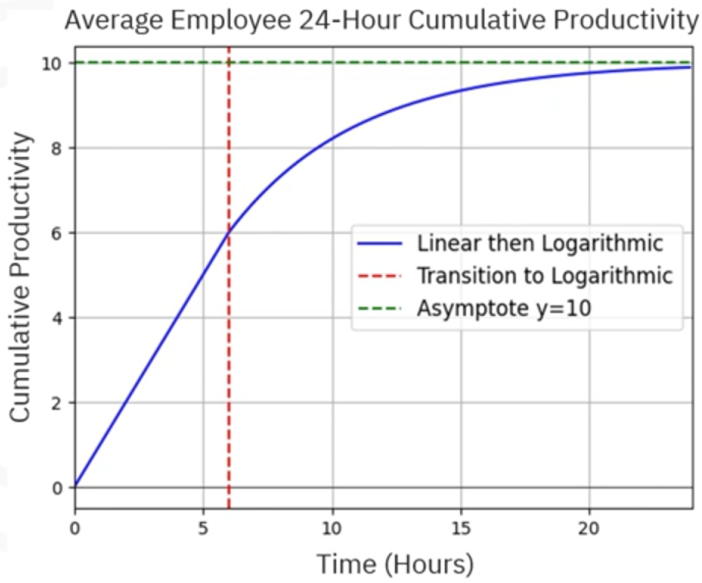


	Year	GDP (B USD)
0	1960	59.2
1	1961	49.6
2	1962	46.7
3	1963	50.1
4	1964	59.1
5	1965	69.8
6	1966	75.9
7	1967	72.1
...	.....	.....

Consider the following simulated example involving human productivity as a function of the **number of consecutive hours worked**. Working more hours per day, on average, increases your productivity. However, after a reasonable limit, say 6 hours of work, each additional work hour generates less productivity per hour than the previous hour. This is an example of **diminishing returns**. The first 6 hours of the model show a linear increase in cumulative productivity, but then, the returns slow down and become logarithmic.

Productivity by work hours example

- Working more hours per day on average increases productivity
- After a reasonable limit, each additional hour generates less productivity



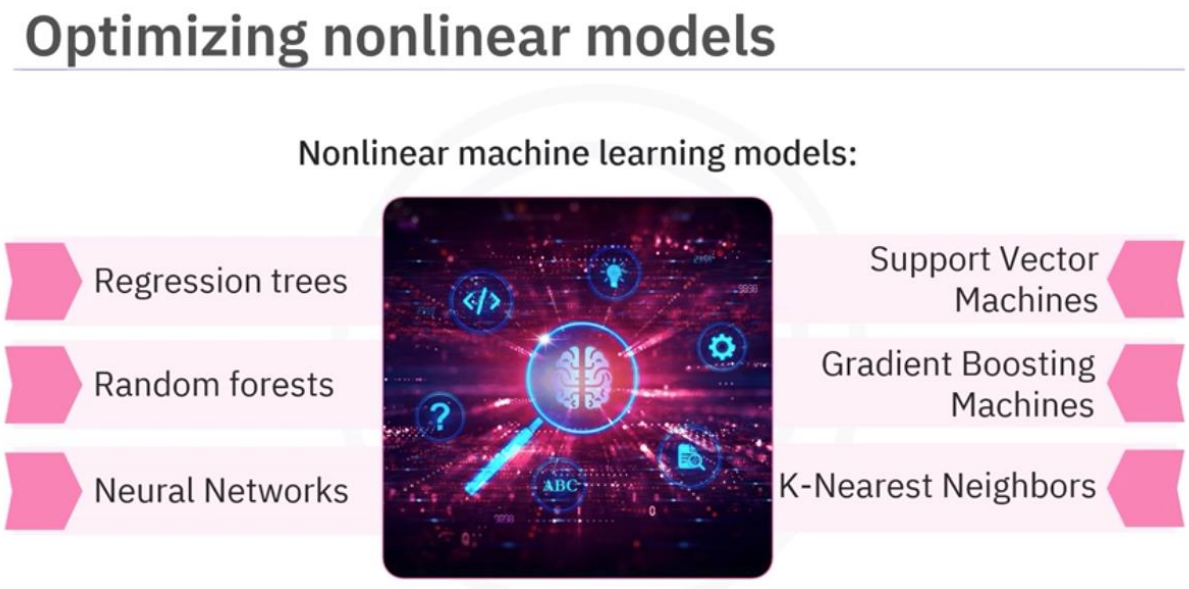
There are many methods to determine what kind of regression model you need.

One technique is to **visually determine** whether the relation is linear or nonlinear.Analyzing **scatter plots** of your target variable against each input variable can reveal patterns in the dependencies. Try to **express** these **patterns as mathematical functions** and determine if they're linear, exponential, logarithmic, or sinusoidal. Generate **models and analyze your results**. There is always a chance that your data might have no relationship with your target. You can visually interpret your model's errors by **plotting its predictions against the actual** target values.

Optimal nonlinear model

If you **have a mathematical expression** for your proposed model, you can use an optimization technique like **gradient descent to find optimal parameters**.

Otherwise, if you haven't decided on a specific regression model, you can **select** amongst many **machine learning models**.



Some options include regression trees, random forests, neural networks, support vector machines, gradient boosting machines, k-nearest neighbors.

Recap

Nonlinear regression uses polynomial, exponential, logarithmic equations to model data. It is used when the relationship between variables cannot be captured through a straight line.

In this video, you learned how you can use **polynomial regression to fit your data to polynomial expressions of the features**.

The polynomial regression model memorizes everything, including any random noise or large variations, rather than understanding the underlying patterns.

There are many real-world complex nonlinear relationships that can't be modeled as polynomials. Some common examples of nonlinear regression include exponential or compound growth, logarithmic, and periodicity.

There are many **methods to determine what kind of regression model you need**.

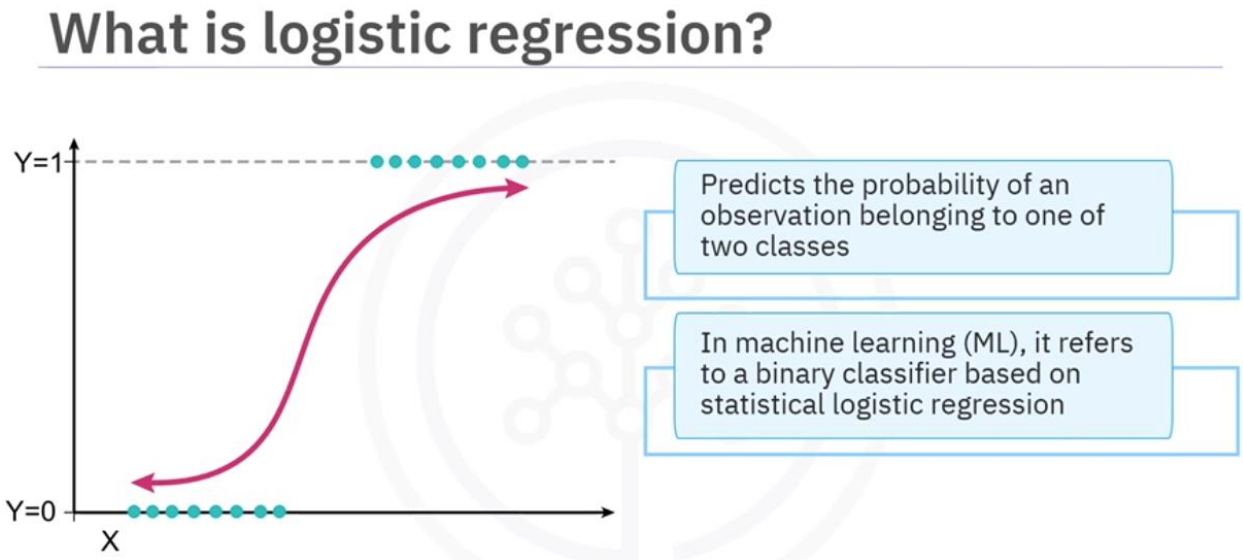
You can analyze scatter plots of your target variable against each input variable to reveal patterns in the dependencies.

To find an optimal nonlinear model, you can select amongst many machine learning models such as regression trees, random forests, and k-nearest neighbors.

Practice Quiz: Linear Regression

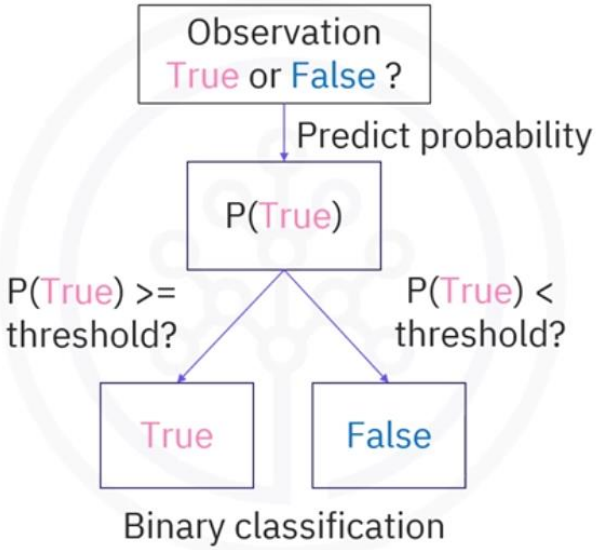
Introduction to Logistic Regression

Logistic regression is a statistical modeling technique that predicts the **probability of an observation belonging to one or two classes**, such as true or false. In machine learning, logistic regression refers to a **binary classifier based on statistical logistic regression**.



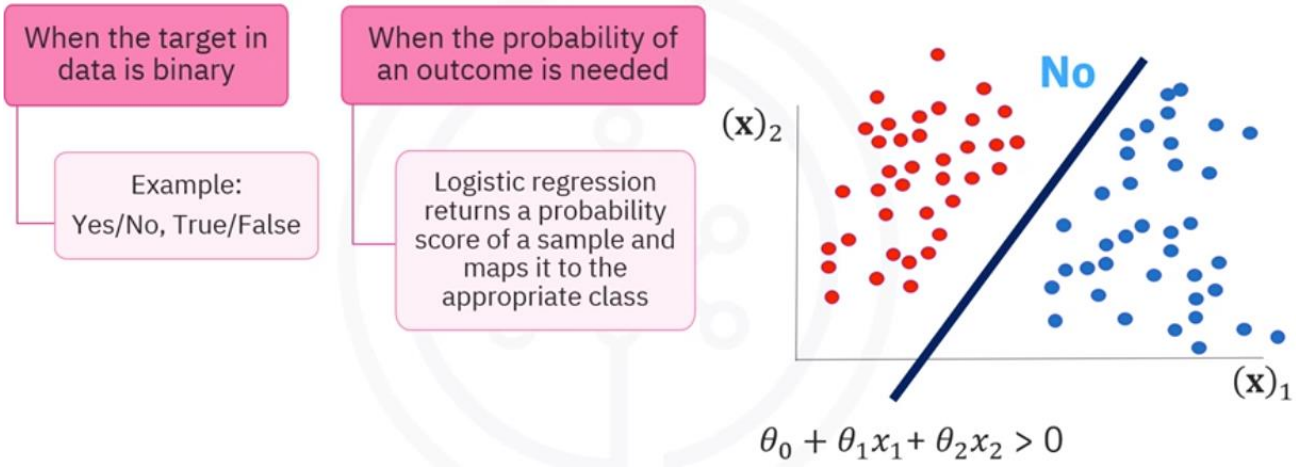
By choosing a threshold probability, the probability predictor becomes a binary classifier simply by assigning each observation to one class if its probability is greater than the threshold and to the other class if its probability is less than the threshold.

What is logistic regression?



Let's understand when logistic regression is a good choice.

When is logistic regression a good choice?

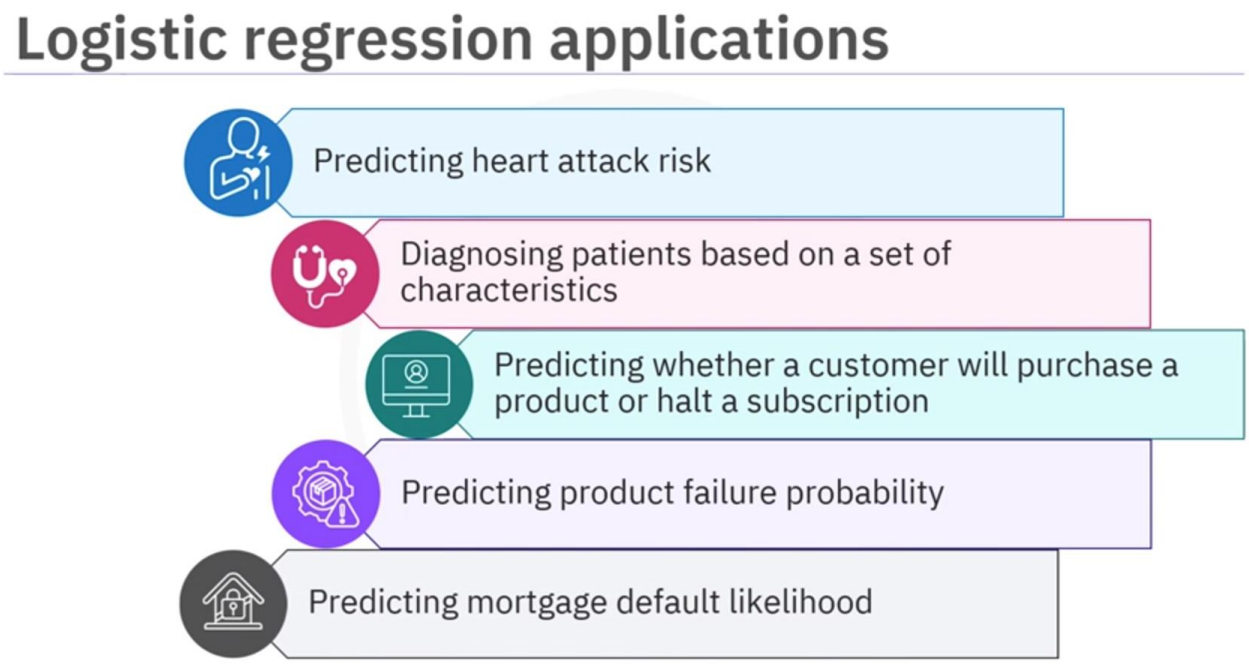


- When the target in your data is binary, indicated as 0 or 1.
- When you need the probability of an outcome, like the probability of a customer buying a product.
- When you want to understand the impact of an independent feature, it allows you to select the best features based on the size of their model coefficients or weights.

If the data is linearly separable, **the decision boundary of logistic regression is a line, a plane, or a hyperplane**. Example, theta 0 plus theta 1 x 1 plus theta 2 x 2 is greater than 0.

Logistic regression is both a **probability predictor and a binary classifier**.

Applications



- For example, you can use it to **predict the probability** that a person will have a **heart attack** within a specified time period based on knowledge of the person's age, sex, and body mass index.
- **Chance** that a specific condition or disease, such as diabetes, might **appear** based on observed characteristics of that patient, such as weight, height, blood pressure, and results of various blood tests, and so on
- **Likelihood** that a customer of a subscription-based service will **halt its subscription**.
- **Probability of failure** of a given process, system, or product.
- **Likelihood** of a homeowner **defaulting** on a **mortgage**.

Consider a telecommunication dataset that you'd like to analyze to predict which customers might leave next month. The dataset comprises historical customer data, where each row represents one customer. The data includes information about services that each customer has signed up for, customer account information, **demographic** information like gender and age range, and customers who've churned or left within the last month.

Example

Logistic regression example

Scenario

Telecommunication data set:

- Services that customers have signed up for
- Customer account information
- Demographic information
- Customers who've left in the last month

Here, the dependent variable column is called churn.

In logistics regression, you can use one or more of these features to predict whether customers will churn.

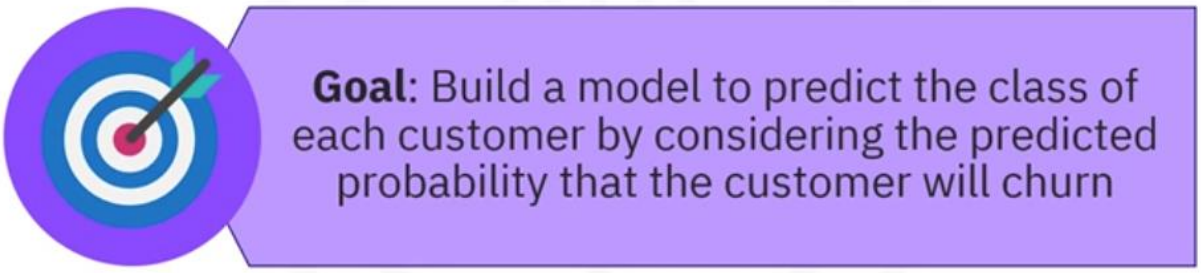
Logistic regression example

Independent variables | Dependent variable

	tenure	age	address	income	ed	employ	equip	callcard	wireless	churn
0	11.0	33.0	7.0	136.0	5.0	5.0	0.0	1.0	1.0	Yes
1	33.0	33.0	12.0	33.0	2.0	0.0	0.0	0.0	0.0	Yes
2	23.0	30.0	9.0	30.0	1.0	2.0	0.0	0.0	0.0	No
3	38.0	35.0	5.0	76.0	2.0	10.0	1.0	1.0	1.0	No
4	7.0	35.0	14.0	80.0	2.0	15.0	0.0	1.0	0.0	?

Binary Variable

Logistic regression **can predict the class**, y-hat, of each customer by **considering the predicted probability**, p-hat, that the customer will churn.



Here, p-hat is the predicted probability that the class, y is 1, given the data, x.

Suppose the goal is to predict customer churn based on their age.

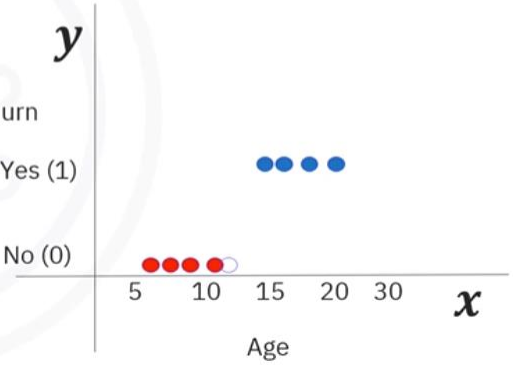
You have a feature, age, denoted as x, and a binary target, variable churn, denoted as y, with two classes, yes and no, represented by binary values 1 and 0. Graphically, you can represent the data with a scatterplot, where class 0 is denoted in red and class 1 in blue.

Predicting churn using linear regression

	tenure	age	address	income	ed	employ	equip	callcard	wireless	churn
0	11.0	33.0	7.0	136.0	5.0	5.0	0.0	1.0	1.0	1
1	33.0	33.0	12.0	33.0	2.0	0.0	0.0	0.0	0.0	1
2	23.0	30.0	9.0	30.0	1.0	2.0	0.0	0.0	0.0	0
3	38.0	35.0	5.0	76.0	2.0	10.0	1.0	1.0	1.0	0
4	7.0	35.0	14.0	80.0	2.0	15.0	0.0	1.0	0.0	0

y = Actual churn classes

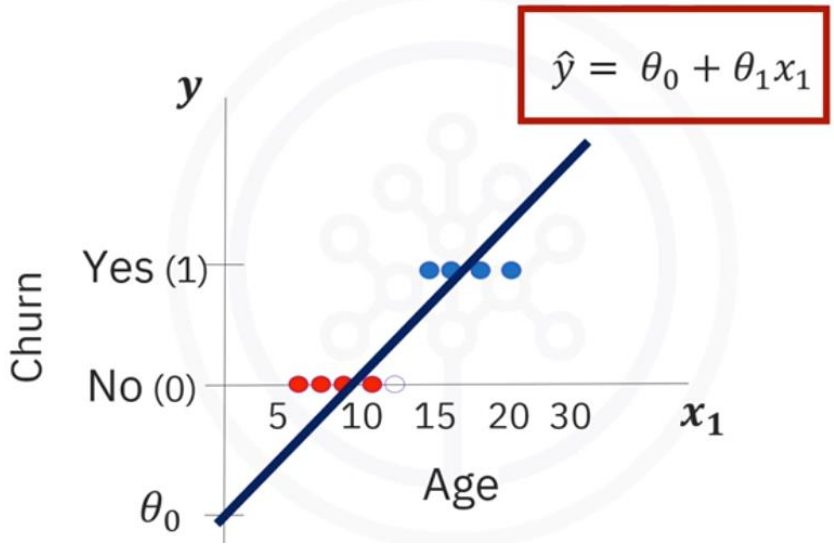
$\hat{y}$ = Predicted churn classes



With linear regression, you can fit a line through the data represented as y-hat equals theta 0 plus theta 1 x 1. This line has two parameters, where **theta 0** is the y-**intercept** of the line, and theta 1 is its slope**. Theta 1** is also called the **weight vector**, or **confidence of the equation**.

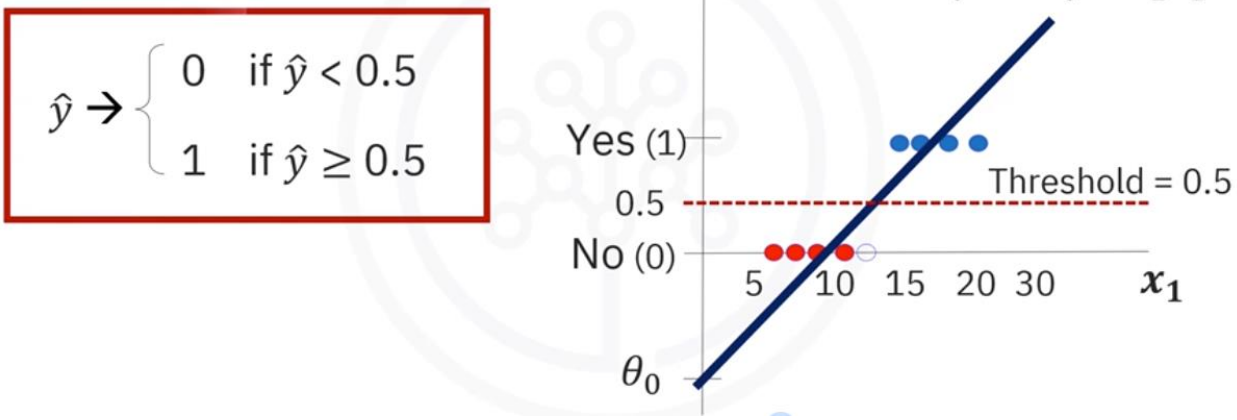
One problem is that the prediction's value y-hat increases indefinitely with age, because the line goes on forever. Obviously, **you cannot use linear regression directly to predict churn**.

Predicting churn using linear regression



Somehow, the predicted values need to be contained within the range 0 to 1. One way to keep the predicted values within the range of 0 to 1 is to **use a threshold**, like 0.5, to **differentiate the classes**.

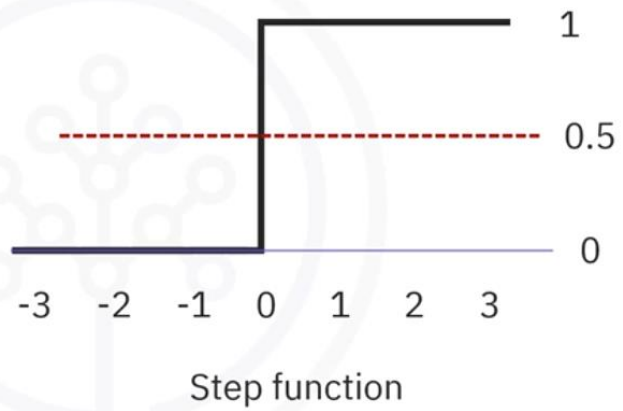
Predicting churn using linear regression



You can **write a rule to allow you to separate class 0 from class 1**. If the value of y-hat is less than 0.5, then the class is 0, otherwise the class is 1.

Challenges of linear regression

$$\hat{y} \rightarrow \begin{cases} 0 & \text{if } \hat{y} < 0.5 \\ 1 & \text{if } \hat{y} \geq 0.5 \end{cases}$$



Step function

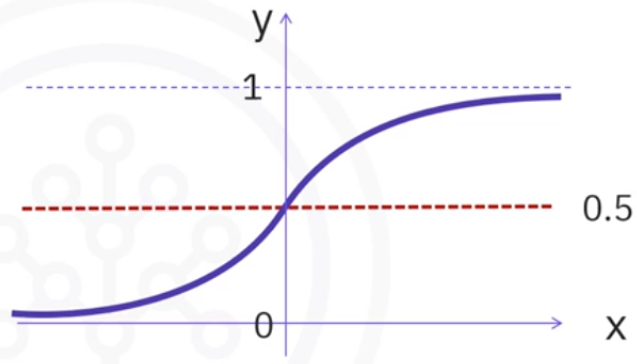
Notice that in the step function, no matter how big the value is, as long as it's greater than 0.5, it simply equals 1. And regardless of how negative the value y-hat is, the output is 0 if it is less than 0.5. In other words, there is no difference between a customer 20 years old and 100 years old. The outcome would be 1.

Wouldn't it be nice to use a smoother line that would project these values between 0 and 1? Indeed, the **existing method doesn't provide the probability of a customer belonging to a class**, which is the goal.

Towards probabilities

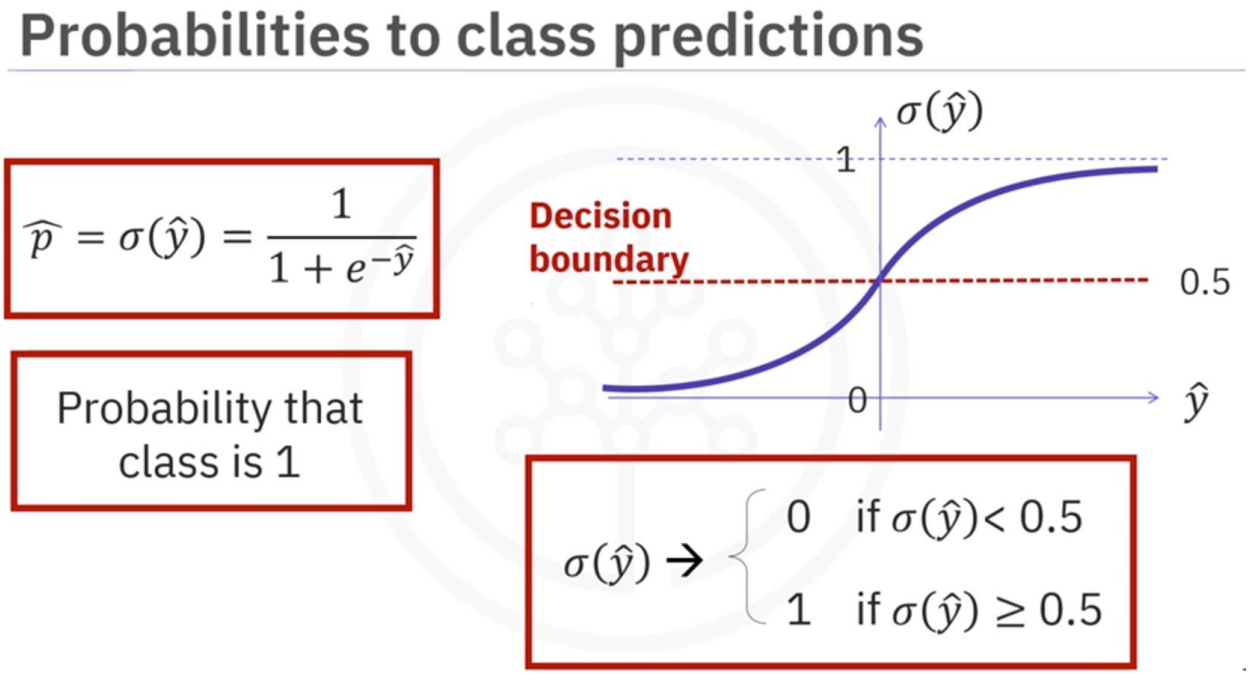
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid function



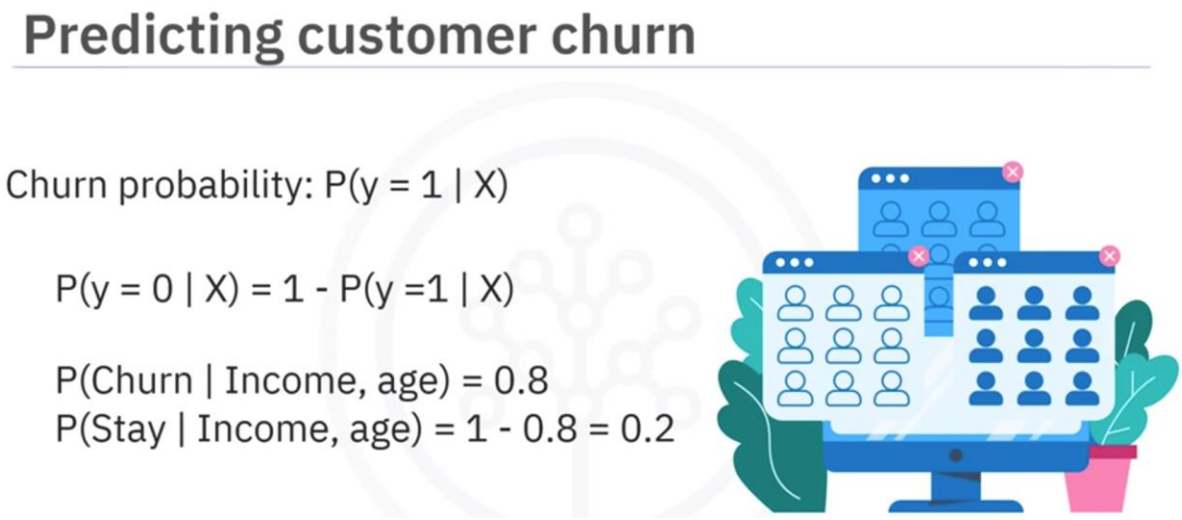
Consider the **sigmoid function**, sigma of x, also known as the **logit function**, defined as 1 over the sum of 1 and e to the minus x. This graph shows that **for x equals 0, the sigmoid function is 0.5**. As x grows, the sigmoid function approaches 1, and as x becomes more negative, the sigmoid approaches 0. This means that the sigmoid function can **take any**

continuous function of x and continuously compress it within the range 0, 1. It **defines a probability**.



Now the model is **sigma of y hat, which represents the probability p hat**, and that the output is 1 given x. Thus you can determine the chance that an observation belongs to either class.

To assign the class to the observation, simply define a threshold like 0.5 and assign the observation to 1 if its probability is greater than 0.5 and 0 otherwise. This **threshold is known as a decision boundary**.



What is the output of the customer churn model when we use the sigmoid function?

The **churn probability** is denoted as the **probability that y is 1 given the data x**. Notice that the probability that the customer won't churn is 1 minus the churn probability, as the two probabilities must add to 1.

For example, suppose the probability of a customer staying with the company can be shown as the probability of churn given a customer's income and age, which could be 0.8. Then, the probability of the same customer staying is 1 minus 0.8, which is 0.2.

Recap

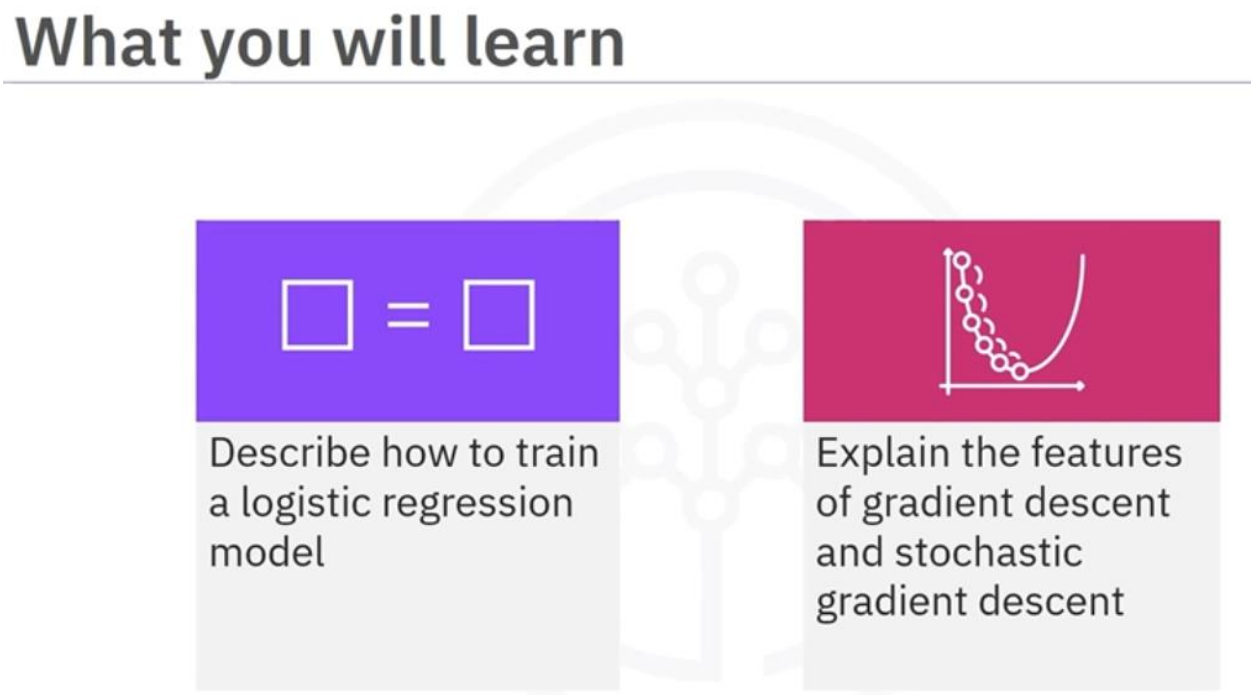
In this video, you learned that in machine learning, **logistic regression refers to a binary classifier based on** statistical logistic regression, **or probability predictor**.

Logistic regression is a good choice for a binary target, probabilistic results, and understanding the impact of a feature.

You also learned that logistics regression is both a **probability predictor and a binary classifier**.

The goal of logistic regression is to build a model to predict the class by considering the predicted probability.

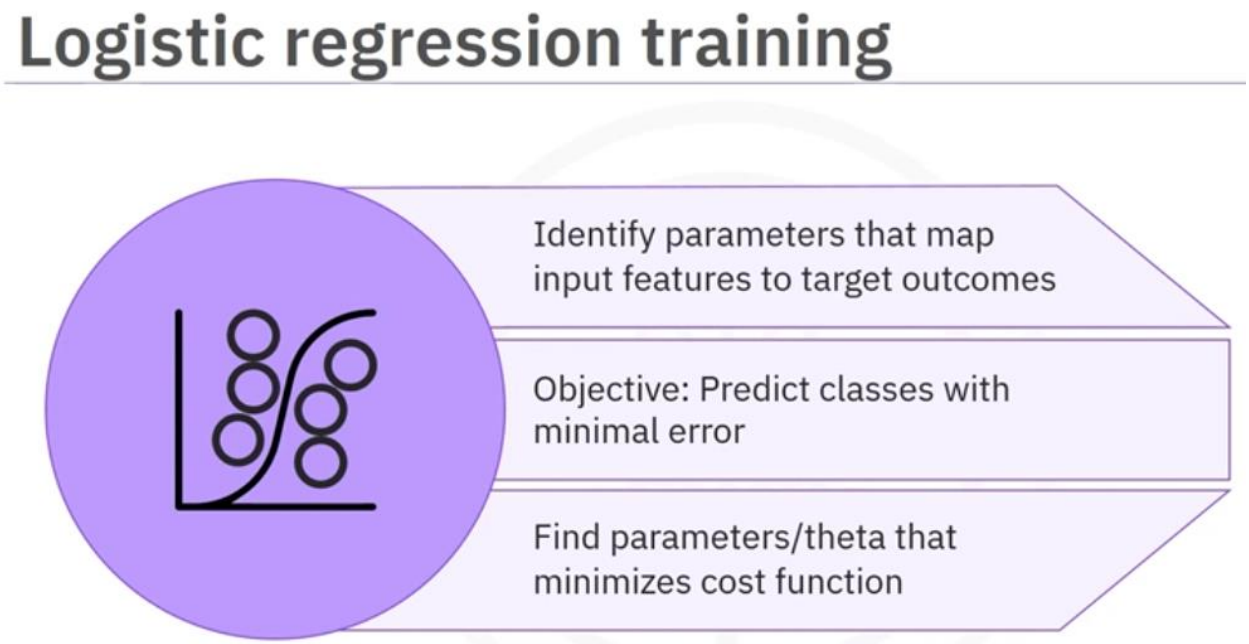
Training a Logistic Regression Model



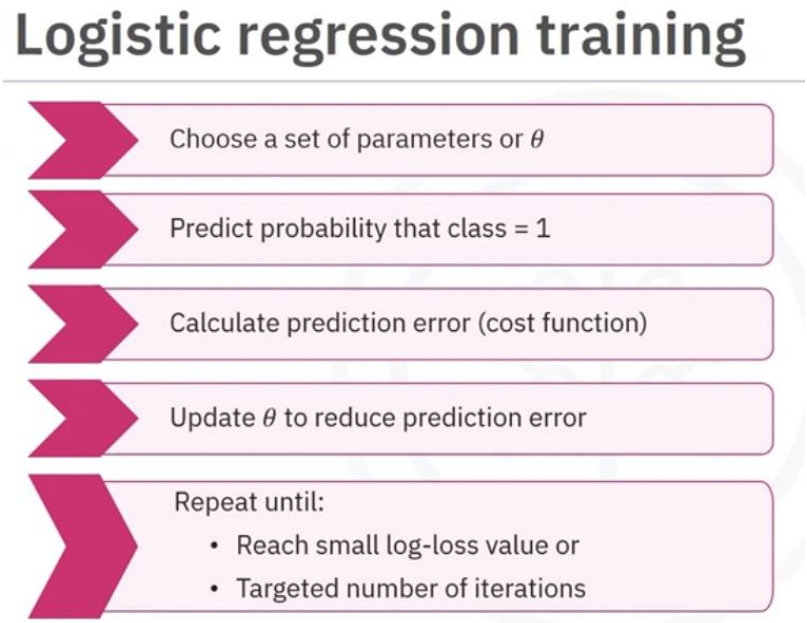
In logistical regression training, you **look for the best parameters that map the input features to the target outcomes**.

The objective is to **predict classes with minimal error**.

The **training process seeks to find a set of parameters, also known as theta, that minimizes the cost function**.



The process of training a logistical regression model comprises several steps.



First, you have to **choose a starting set of parameters** called theta. This can be a random choice.

You then **predict the probability that the class is 1 for each observation** of your data.

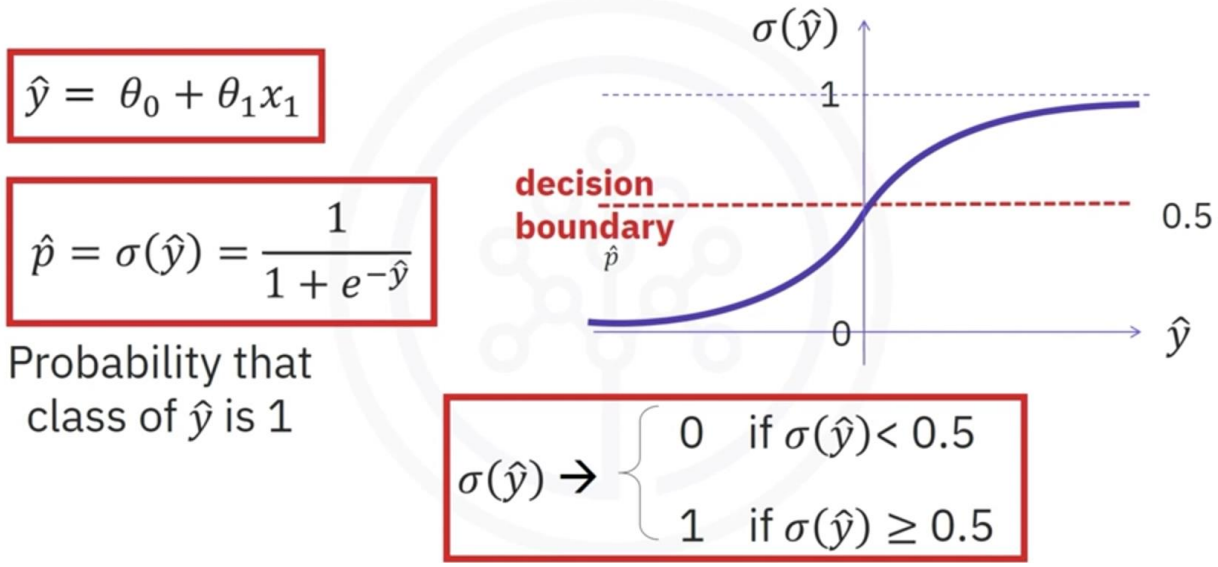
The next step is to **measure the error between the predicted classes and the actual classes**. This error is called a **cost function**.

You then **determine a new theta that reduces the prediction error**.

Finally, you need to **repeat the process** until you reach a small enough value for the log loss or a specified maximum number of iterations.

Next, let's understand optimal logistic regression.

Optimal logistic regression



The process of creating a decision boundary by **combining** a linear model y-hat in terms of parameters theta with a sigmoid function **yields** a binary classification model or what might be called a preliminary logistic regression.

The model is called preliminary because it's not necessarily the best logistic regression model. The best logistic regression model can only be achieved after the first pass. The model parameters, theta, need to be found. An optimization step finds the best parameters.

To achieve optimization, you need a **metric that determines the model's goodness** of fit for a given set of parameters. The metric for optimizing logistic regression is a cost function called log loss, which needs to be minimized. **Log loss is a cost function** that **measures** how well the **predicted probabilities, p-hat i, match the actual class's yi**.

Logistic regression seeks to minimize this cost function.

Optimizing logistic regression

$$\text{Log–loss} = -\frac{1}{N}\sum_{i-1}^{N} y_i \log(\hat{p}_i) + (1 - y_i)\log(1 - \hat{p}_i)$$

- Cost function or log-loss needs to be minimized
- Measures how well ($\hat{p}_i$) matches y_i

Here, i refers to the ith observation of the data, which is n rows.

Log loss is defined as minus the average over i of two terms.

The actual class times the logarithm of the predicted probability that the class is 1 plus 1 minus the actual class times the log of the probability that the class is 0. The negative sign exists because the logarithm is negative for arguments between 0 and 1.

Log loss favors confident classifications that are correct.

Understanding log-loss

$$\text{Log–loss} = -\frac{1}{N}\sum_{i-1}^{N} y_i \log(\hat{p}_i) + (1 - y_i)\log(1 - \hat{p}_i)$$

Confident and correct: Predicted probability of class 1 is high and correct => log-loss is small

Confident and incorrect: Predicted probability of class 0 is high and incorrect => log-loss is large

For instance, when the predicted probability of class 1 is high and correct, p-hat i is close to 1 for an observation, and yi is equal to 1. You can convince yourself by inspecting the formula

that the log loss is small. Indeed, the first term vanishes because the log term tends to 0 as the probability approaches 1, while the second term vanishes because the factor 1 minus yi is 0.

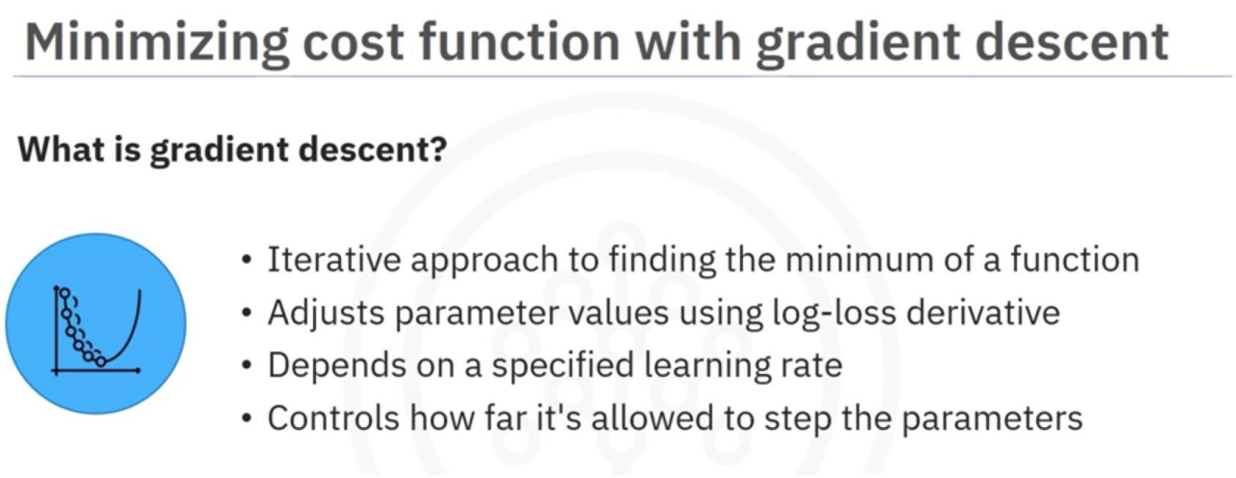
In this way, **log loss penalizes confident, incorrect predictions**.

When the predicted probability of class 0 is high and incorrect, that is, when p-hat is close to 1 for an observation, and the actual class is 0, the log loss is very large.

Gradient descent and stochastic gradient descent.

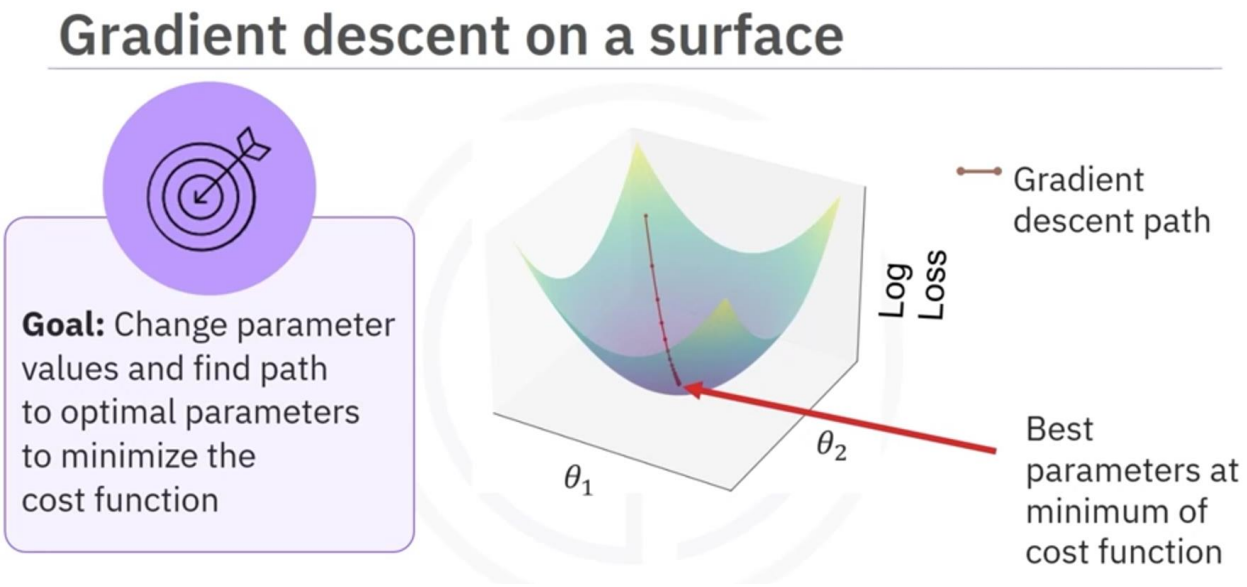
There are various ways to stop iterations, but essentially, **you stop training when your model's log loss is satisfactory**. Different techniques can be used to change the values of theta, and one of the most popular methods is gradient descent.

Gradient descent is a clever iterative approach to finding the minimum of a function. It adjusts the parameter values in the **direction of the steepest descent using the derivative of the log loss function**.



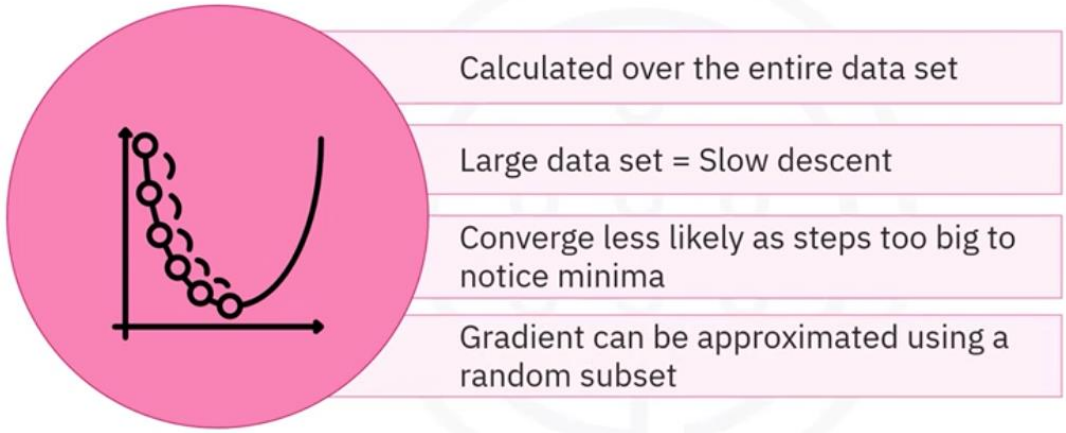
Gradient descent depends on a **specified learning rate**, which controls how **far it's allowed to step the parameters on each iteration**.

The main objective of gradient descent is to **change the parameter values and find a path to the optimal parameters to minimize the cost function**.



Consider the plot, which simulates a **parabolic log loss cost function** of the trial parameters theta1, theta2. This **surface represents the error** for different values of parameters. The gradient of the surface points in the direction of the steepest ascent. Thus, **the negative of the gradient points** in the direction of the steepest descent, hence the **name gradient descent**. The steeper the slope, the greater the magnitude of the gradient, and thus, the greater the step toward the minimum. You can **control the size of each step** by scaling the gradient by a factor called **the learning rate**. As the lowest point is reached, the slope diminishes to zero. This lowest point of the path occurs at the optimum theta1, theta2.

Let's explore some additional features of gradient descent.

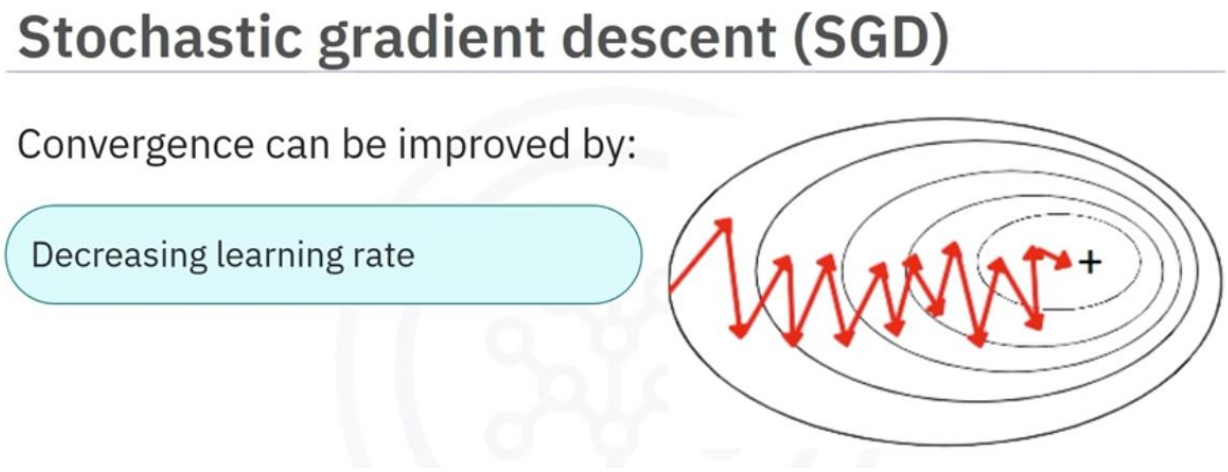


The gradient of the cost function is **calculated over the entire data set on each iteration**. When the data set is large, gradient descent becomes very **slow**.

You could try **speeding up** the convergence **by increasing the learning rate**, but convergence becomes less likely as the **steps might be too big to notice the minima**.

Instead of using the whole, the cost function gradient can be approximated by choosing a **random subset of the data to calculate it on**.

A variation of the gradient descent algorithm is stochastic gradient descent, or SGD.



It's **faster, but can be less accurate**. It uses a **random subset** of training data and scales well. SGD is more likely to **overlook local minima and find global minima** of the cost function. It **converges quickly** toward a global minimum, but **can wander around it** for some time.

The convergence can be **improved** by **slowing down** as the **algorithm gets closer to a global minimum**. You can home in on the minimum by **decreasing the learning rate as you get closer**, or you can **gradually increase the size of the random data** sample used to calculate the gradient of the cost function.

Recap

In this video, you learned that the objective of **logistical regression training is to predict classes with minimal error**.

The training process consists of key steps created **to find a set of parameters**, or theta, that **minimize the cost function**.

An optimization step is used to find the best parameters.

The metric for optimizing logistic regression is **a cost function called log loss**, which needs to be minimized.

Log loss favors confident classifications that are correct and penalizes confident, incorrect predictions.

Gradient descent is a clever, iterative approach to **finding the minimum of a function**.

Stochastic gradient descent is a scalable variation of the gradient descent algorithm, which uses **a random subset of training data**.

Favorece las predicciones correctas y seguras ✅

El log loss recompensa a las predicciones que son correctas y muy seguras. Si tu modelo predice que una imagen es un gato con un 99% de probabilidad y la imagen realmente es un gato, la penalización es muy, muy pequeña. La pérdida es mínima, casi cero. Esto indica que el modelo está funcionando de manera óptima y es muy fiable.

Penaliza las predicciones incorrectas y seguras ❌

Aquí es donde el log loss se vuelve muy estricto. Si tu modelo predice con un 99% de probabilidad que una imagen es un gato, pero en realidad es un perro, la penalización es enorme. La pérdida es altísima. Esto se debe a que el modelo no solo se equivocó, sino que lo hizo con una confianza extrema, lo que es un indicio de que su entrenamiento o su entendimiento de los datos es defectuoso.

App Item: Lab: Logistic Regression

It is a churning exercise.

The dependent variable must be an integer.

At preprocessing we convert the features extracted from the DataFrame into an NumPy array X. the same with the dependent variable.

Data normalization is done with this sentence

X_norm = StandardScaler().fit(X).transform(X)

StandardScaler()

This part of the code creates an **instance** of the StandardScaler class from the scikit-learn library. This instance is a tool that we'll use to perform the scaling. The StandardScaler is a powerful and popular **scaling method** that is a part of the sklearn_preprocessing module. It's particularly **useful when your data has different units or scales**, as many machine learning algorithms perform better when all features are on a similar scale.

.fit(X)

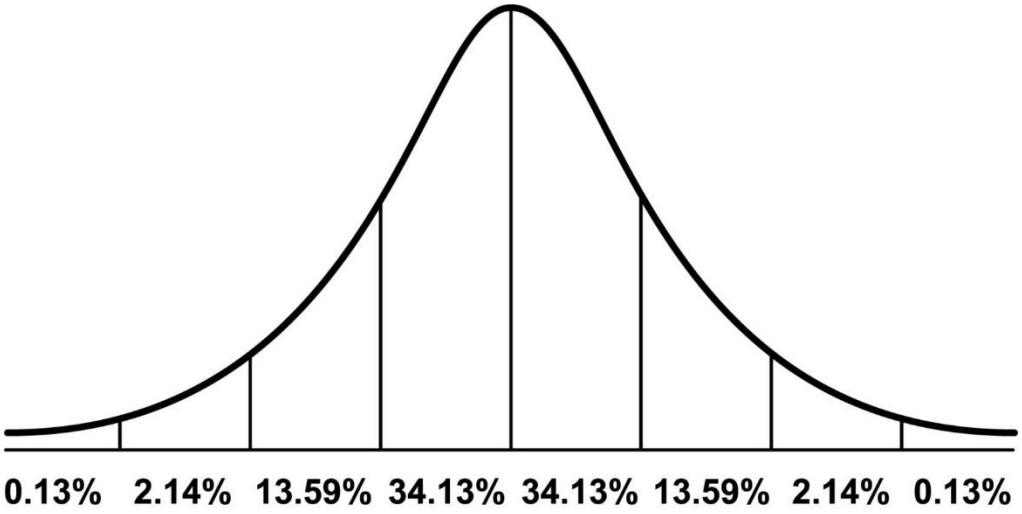
The .fit() method calculates the **mean** and **standard deviation for each feature** (column) in the dataset X. It's essentially "learning" the parameters needed for scaling from the provided data. This step does **not** modify the data itself; it **just computes the necessary statistics**.

.transform(X)

The .transform() method then **uses the mean and standard deviation calculated in the .fit()** step to **scale** the data in X. For each feature, it subtracts the mean and then divides by the standard deviation. The result is a new array, X_norm, where each feature has a mean of 0 and a standard deviation of 1.

Why It's Done

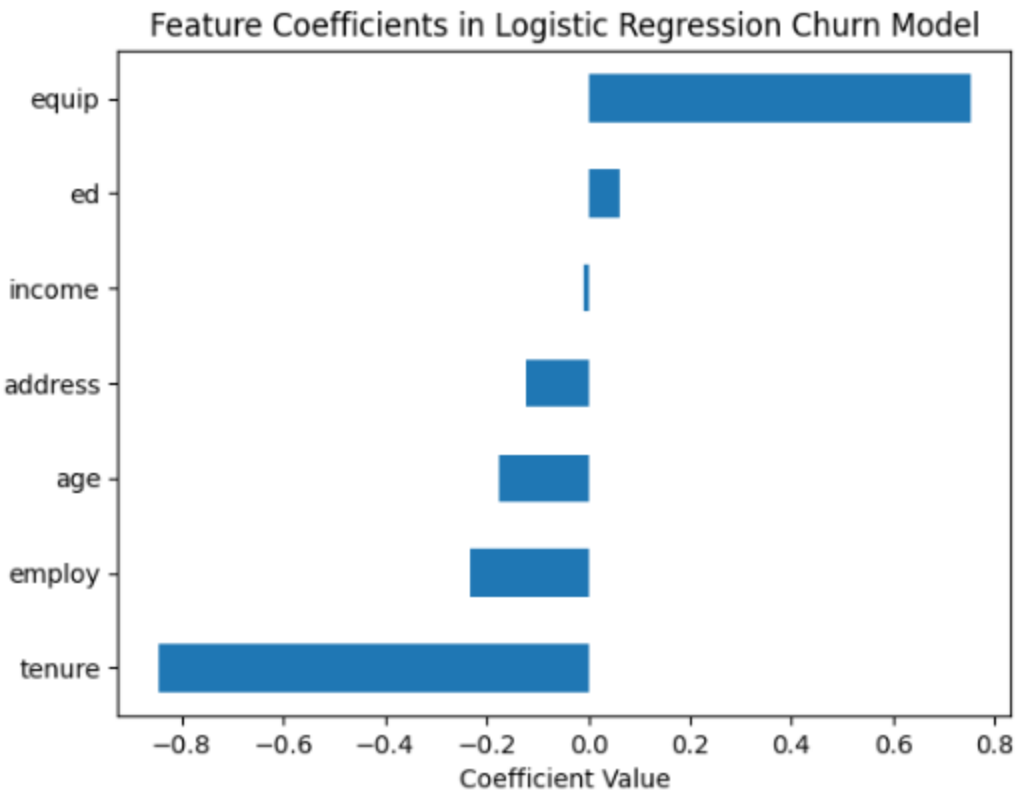
This normalization process, often called **Z-score normalization** or **standardization**, is crucial for many machine learning algorithms, especially those that rely on distances between data points, such as **Support Vector Machines (SVMs)**, **K-Nearest Neighbors (KNNs)**, and **Principal Component Analysis (PCA)**.



Without scaling, features with larger ranges could disproportionately influence the model's performance. For example, if one feature ranges from 0 to 1,000, and another from 0 to 1, the algorithm might incorrectly give more weight to the first feature.

To calculate the probability of each record belong to class 0 or to class 1 we use yhat_prob = LR.predict_proba(X_test).

It is interesting to graphically see the weight for each feature. How they contribute to the dependet variable.



Graded Assignment: Practice Quiz: Logistic Regression

Module 2 Summary and Highlights

- Regression models relationships between a **continuous target variable and explanatory features**, covering **simple** and **multiple** regression types.
- Simple regression uses a single independent variable to estimate a dependent variable, while multiple regression involves more than one independent variable.
- Regression is widely applicable, from forecasting sales and estimating maintenance costs to predicting rainfall and disease spread.
- In simple linear regression, a best-fit line minimizes errors, measured by Mean Squared Error (MSE); this approach is known as Ordinary Least Squares (OLS).
- OLS regression is easy to interpret but **sensitive to outliers, which can impact accuracy**.
- Multiple linear regression extends simple linear regression by using multiple variables to predict outcomes and analyze variable relationships.

- Adding **too many variables can lead to overfitting**, so **careful variable selection** is necessary to build a balanced model.
- **Nonlinear regression** models complex relationships using **polynomial**, **exponential**, or **logarithmic** functions when data does not fit a straight line.
- **Polynomial regression** can fit data but **may overfit by capturing random noise rather than underlying patterns**.
- **Logistic regression is a probability predictor and binary classifier**, suitable for binary targets and assessing feature impact.
- Logistic regression **minimizes errors using log-loss** and optimizes with gradient descent or stochastic gradient descent for efficiency.
- Gradient descent is an iterative process to minimize the cost function, which is crucial for training logistic regression models.

Cheat Sheet: Linear and Logistic Regression

Graded Quiz: Linear and Logistic Regression:7.42 > 7. OK

1.Question 1

A company wants to forecast CO2 emissions based on engine size, number of cylinders, and fuel consumption. Which regression techniques should a company use?

- Simple regression
- Polynomial regression
- Logistic regression
- Multiple regression

2.Question 2

In which of the following scenarios would simple regression be an appropriate analysis method?

- Classifying customer feedback as positive, neutral, or negative based on text data.
- Predicting annual rainfall based on average temperature.
- Grouping customers into segments based on purchase behavior.
- Predicting sales based on advertising budget, season, and product price.

3.Question 3

A multinational company is trying to predict employee productivity based on several factors, including hours of training, years of experience, and the number of completed projects. What regression technique would allow them to consider all these variables in their model?

- Simple logarithmic regression
- Simple regression
- Multiple linear regression
- Simple polynomial regression

4.Question 4

A company is analyzing its production costs against the number of workers it employs. It observes diminishing returns in its cost savings as it hires more workers.

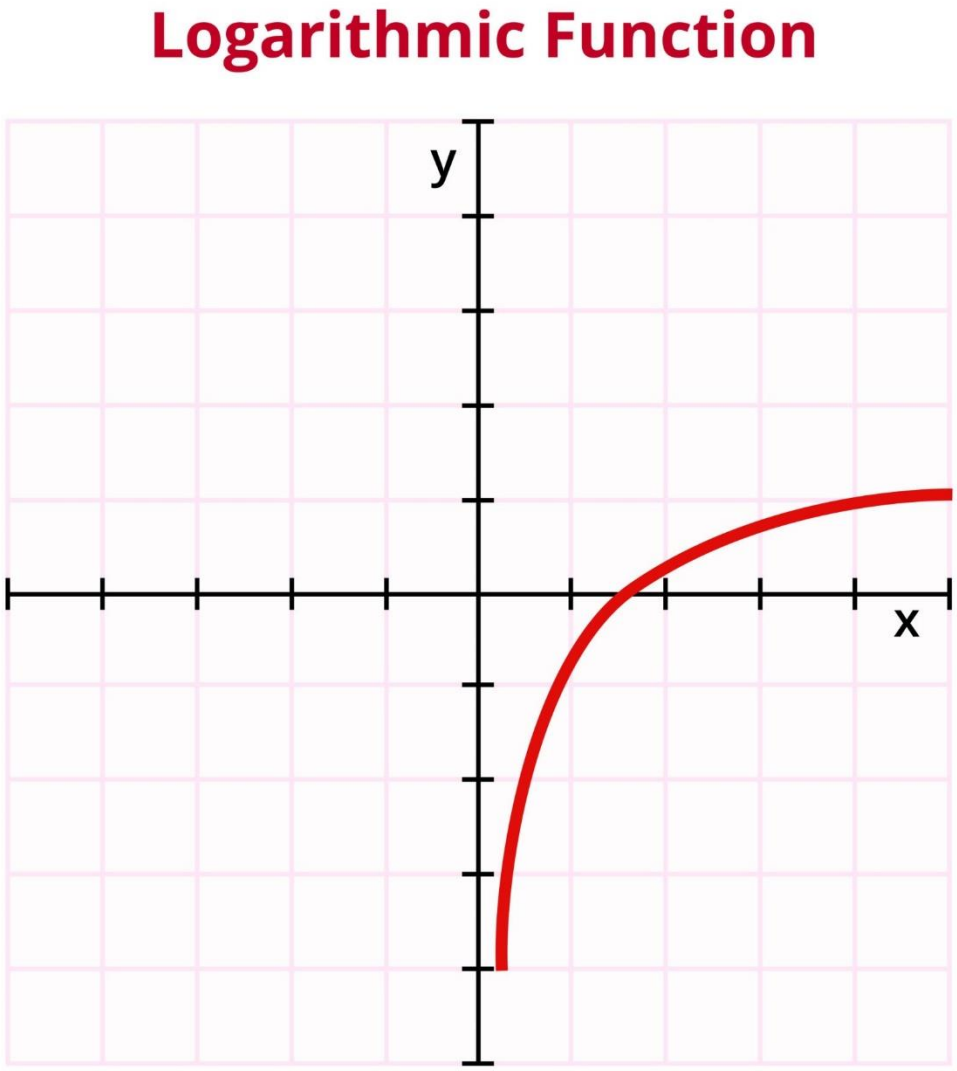
Which type of regression would be most appropriate for modeling this relationship?

- Exponential regression
- Linear regression
- Polynomial regression
- Logarithmic regression

Como se reducen los ahorros con el numero de trabajadores , aumentan los costes de producción.

Por qué la regresión logarítmica es la correcta

El concepto de **rendimientos decrecientes** (o retornos decrecientes) significa que, a medida que una variable de entrada (en este caso, el número de trabajadores) aumenta, el beneficio o resultado (ahorro de costes) que se obtiene de cada unidad adicional de esa variable es cada vez menor.



Con licencia de Google

- La **regresión logarítmica** se utiliza para modelar este tipo de relaciones no lineales, donde la variable dependiente (costes de producción) cambia a una tasa que disminuye a medida que la variable independiente (número de trabajadores) aumenta.
- La curva de una función logarítmica crece rápidamente al principio y luego se aplana, lo que representa perfectamente el escenario descrito: los primeros trabajadores contratados tienen un gran impacto en la reducción de costes, pero a medida que se añaden más, el ahorro adicional es cada vez más pequeño.

Por qué las otras opciones no son correctas

- **Regresión lineal:** Asume una relación constante, donde el ahorro de costes por cada nuevo trabajador sería el mismo. Esto no modela la idea de rendimientos decrecientes.
- **Regresión exponencial:** Se utiliza para modelar un crecimiento o decrecimiento que se acelera con el tiempo, lo contrario a lo que se describe en el problema.
- **Regresión polinomial:** Puede modelar curvas complejas, pero una regresión polinomial simple (por ejemplo, de grado 2) modelaría un efecto de rendimientos decrecientes solo hasta un punto, y luego podría comenzar a crecer en la dirección opuesta, lo que no encajaría con el escenario de costes. La forma más natural y comúnmente utilizada para este tipo de relación es la regresión logarítmica.

5.Question 5

A healthcare provider uses logistic regression to predict whether a patient will develop a certain condition based on features like age, weight, and medical history. The model correctly identifies most high-risk patients but often misclassifies low-risk patients as high-risk. What method could help the provider reduce false positives?

- Increase the complexity of the logistic regression model.
- Implement principal component analysis (PCA) in the model.
- Increase the dataset size with more samples of low-risk patients.
- Tune the decision threshold of a logistic regression model.

6.Question 6

A ride-sharing company uses logistic regression to predict whether a user will cancel a ride booking based on features like wait time and trip distance. For one booking, the model predicts a cancellation probability of 0.4. What does this probability indicate?

- There is no chance that the ride will be canceled.
- The model predicts a 100% likelihood that the ride will be canceled.
- The model predicts a 60% likelihood that the ride will be canceled.
- The model predicts a 40% likelihood that the ride will be canceled.

7.Question 7

You are evaluating a binary classification model using log loss. Which of the following scenarios would result in the highest log loss?

- The model predicts a probability of 0.1 for the correct class and 0.9 for the incorrect class.
- The model predicts a probability of 0.9 for the correct class and 0.1 for the incorrect class.
- The model predicts a probability of 0.7 for the correct class and 0.3 for the incorrect class.
- The model predicts a probability of 0.5 for both the correct and incorrect classes.

Explicación Detallada

El *log loss* (pérdida logarítmica) es una función de coste que penaliza las predicciones incorrectas, especialmente aquellas que se hacen con una alta confianza. Cuanto mayor sea el *log loss*, peor será el rendimiento del modelo.

La fórmula del *log loss* para una sola observación en un modelo de clasificación binaria es:

Donde:

- es la etiqueta real (1 para la clase correcta, 0 para la incorrecta).
- es la probabilidad predicha por el modelo para la clase 1.

En los escenarios que se presentan, el modelo siempre predice una probabilidad para la clase correcta y para la incorrecta. Esto significa que podemos simplificar la fórmula del *log loss* a solo , donde es la probabilidad asignada por el modelo a la clase que realmente es la correcta.

Veamos cómo se aplica esto a cada opción:

1. **Modelo predice 0.1 para la clase correcta:** El *log loss* sería , lo que es aproximadamente 2.3.
    - **Interpretación:** El modelo no solo se equivocó (le dio una baja probabilidad a la clase correcta), sino que también lo hizo con una confianza muy alta, ya que le asignó una probabilidad de 0.9 a la clase incorrecta. Esta es la peor situación, por lo que el *log loss* es el más alto.
2. **Modelo predice 0.9 para la clase correcta:** El *log loss* sería , lo que es aproximadamente 0.1.
    - **Interpretación:** El modelo predijo la clase correcta con una confianza muy alta. Esto es lo ideal, y por lo tanto, la penalización (el *log loss*) es muy baja.

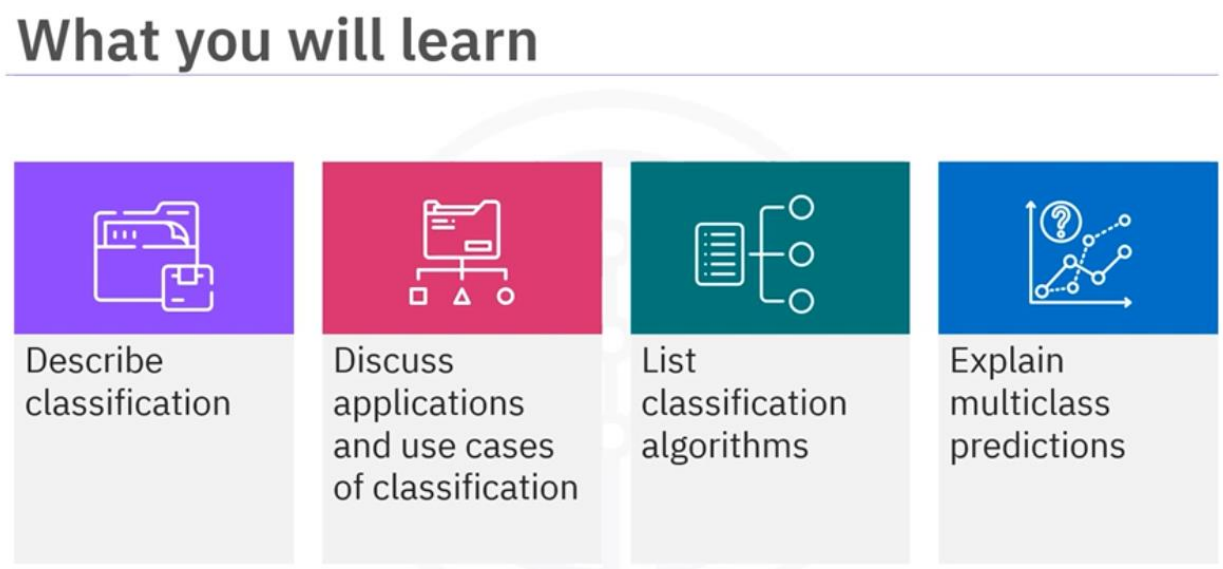
3. **Modelo predice 0.7 para la clase correcta:** El *log loss* sería , lo que es aproximadamente 0.36.
    - **Interpretación:** El modelo predijo la clase correcta con una confianza razonable. El *log loss* es bajo, pero no tan bajo como el del escenario anterior, ya que la confianza no fue tan alta.
4. **Modelo predice 0.5 para la clase correcta:** El *log loss* sería , lo que es aproximadamente 0.69.
    - **Interpretación:** El modelo no tiene idea de cuál es la clase correcta; su predicción es equivalente a un volado. La penalización es mayor que en los escenarios 2 y 3 porque el modelo no tiene confianza. Sin embargo, no es la más alta, ya que no se equivocó con una confianza extrema.

Conclusión: El *log loss* se dispara cuando un modelo predice una baja probabilidad para la clase correcta y una alta probabilidad para la incorrecta. Este es el caso del primer escenario, donde la probabilidad para la clase correcta es solo 0.1, resultando en la mayor penalización.

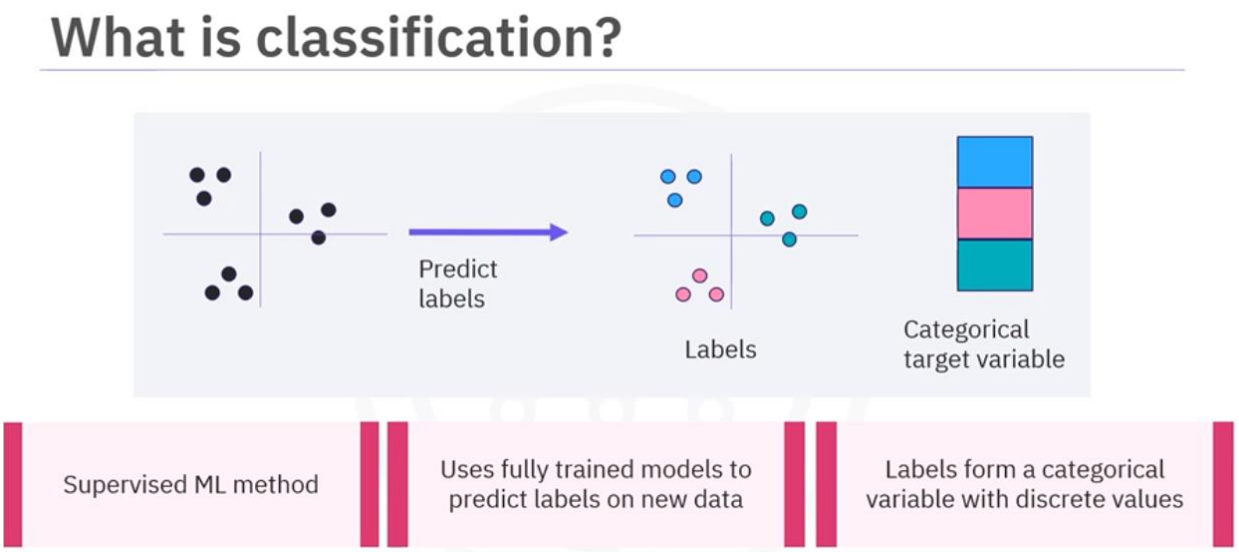
Module 3 Building Supervised Learning Models

In this module, you will build and evaluate a range of supervised machine learning models to solve both classification and regression problems. You'll start by describing how classification models predict categorical outcomes, and implement multi-class classification strategies using real-world data. You'll then explore how decision trees make predictions and apply them to both classification and regression tasks. The module also covers using support vector machines (SVM) for fraud detection, applying K-Nearest Neighbors (KNN) for customer classification, and training ensemble models like Random Forest and XGBoost to improve accuracy and efficiency. You'll differentiate bias and variance in model performance and explore how ensemble methods help balance this tradeoff. To support your learning, you'll receive a Cheat Sheet: Building Supervised Learning Models with key terms, model types, and evaluation tips.

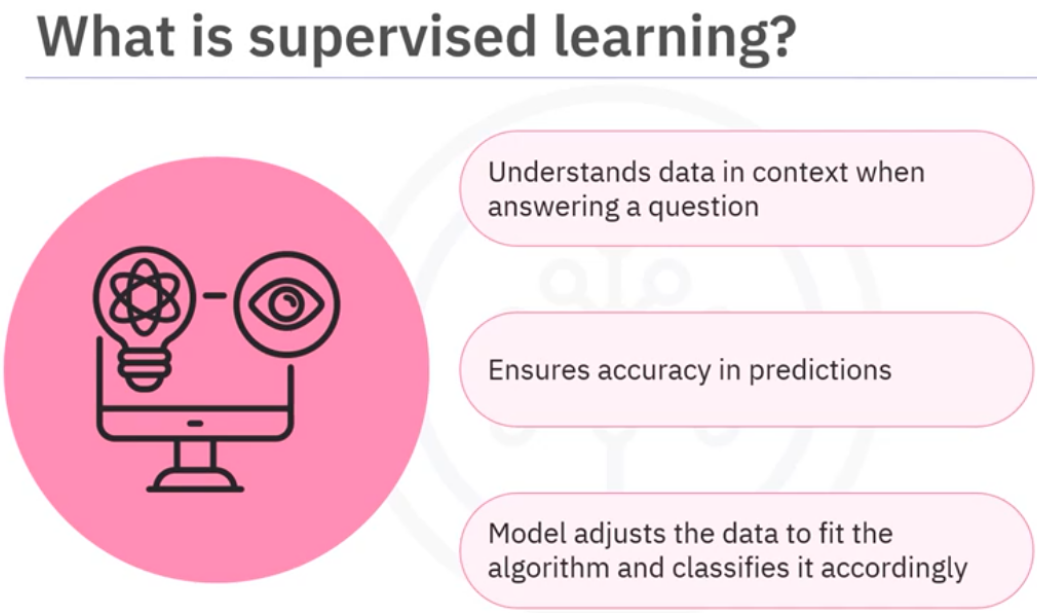
Classification



Classification.

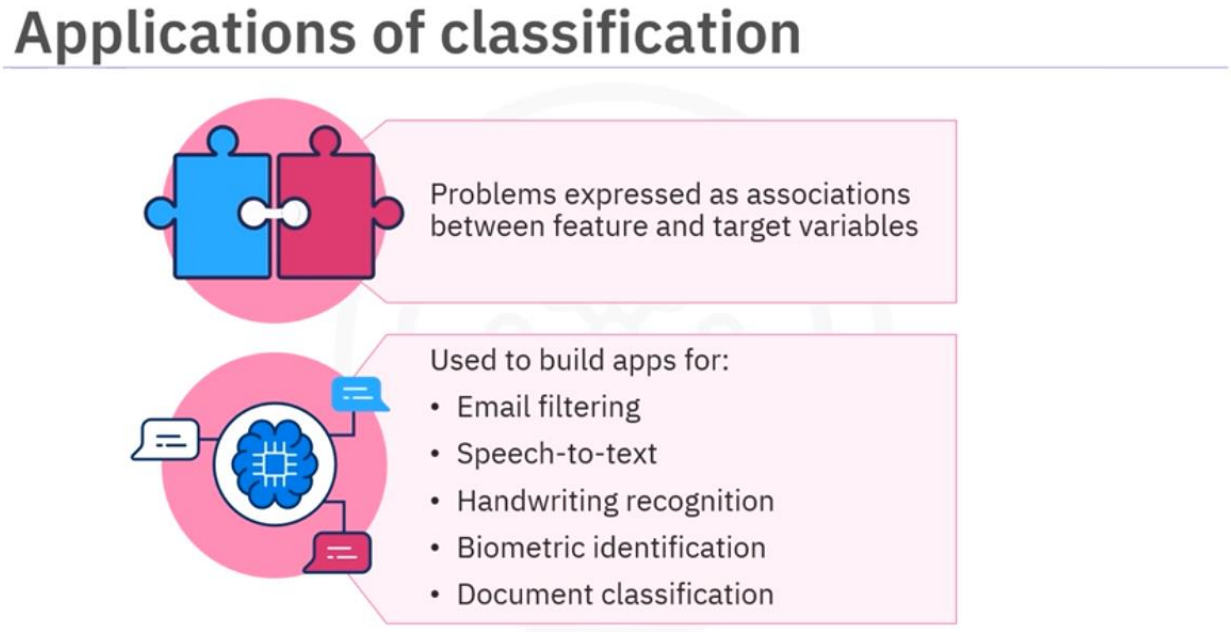


Classification is a supervised machine learning, or ML method, that uses **fully trained models to predict labels on new data**. The labels in classification form a categorical variable with **discrete values**.



Supervised learning aims to **understand data in the correct context when answering a specific question**. This ensures data accuracy when making predictions. As data is input into the model, the **model adjusts the data to fit the algorithm and classifies it** accordingly, defining the input and the predicted output.

Classification use cases.

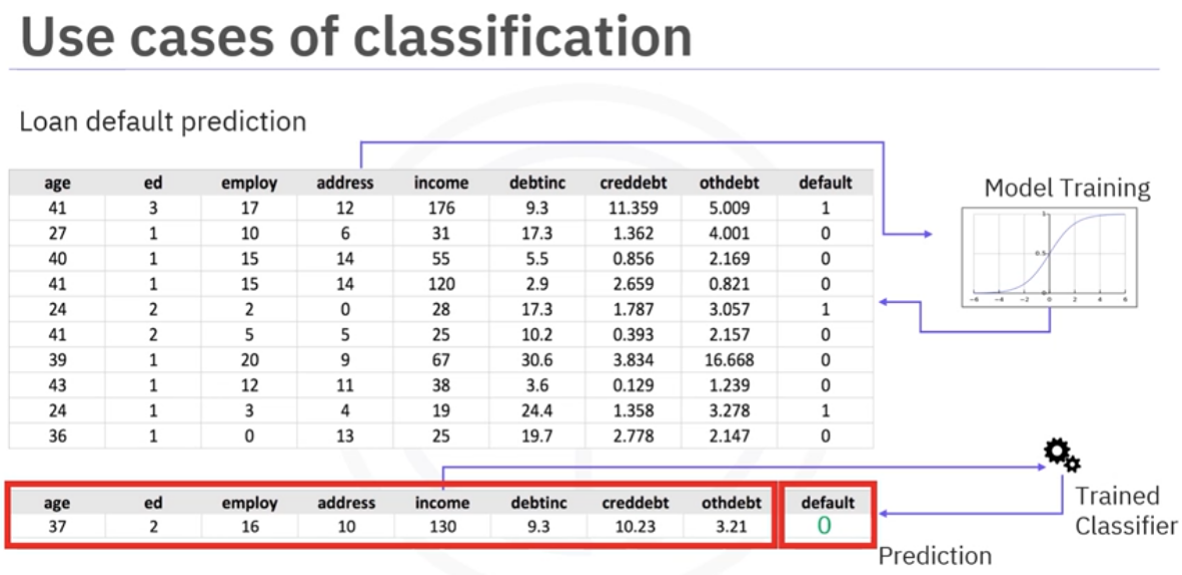


Classification has several applications in a wide variety of industries. Many problems can be expressed as **associations between feature and target variables**, particularly when labeled data is available. Classification can be used to build applications for **email filtering, speech-to-text, handwriting recognition, biometric identification, document classification**, and much more.

- **Churn prediction** is when you use machine learning classification to predict whether a customer will discontinue a service.
- **Customer segmentation** is when you use classification to predict the category to which a customer belongs.
- You can also use classification to predict whether a customer will likely respond to an **advertising campaign**.

Let's explore additional use cases of classification.

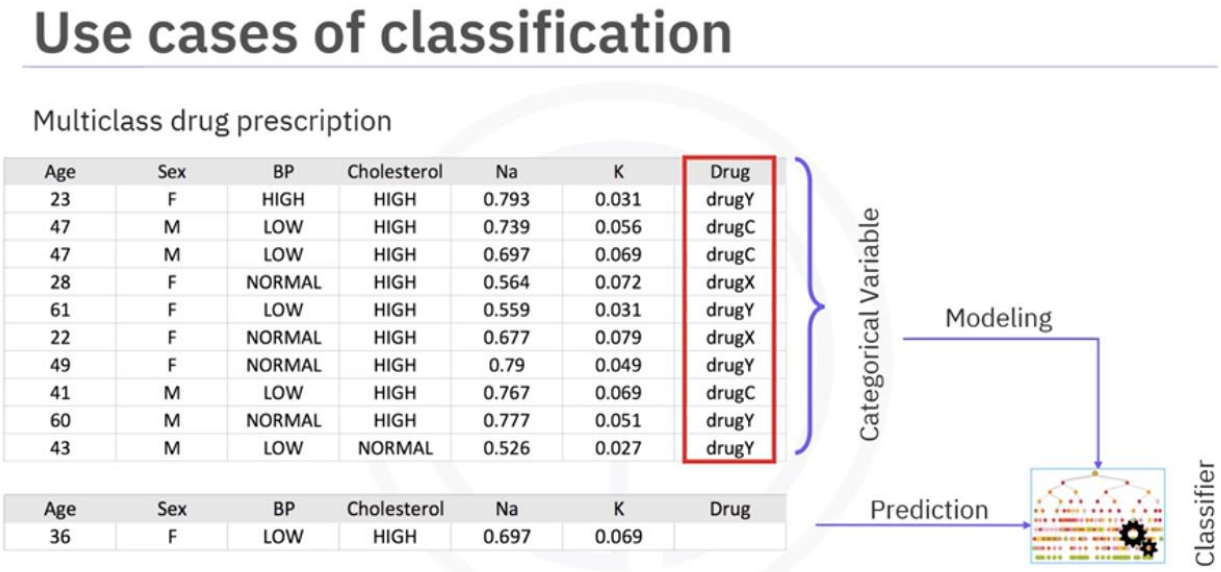
Suppose a bank is concerned that some of their **loan applicants may be unable to repay** their loan. The bank uses historical loan default data to predict which customers are likely to default. Here, a **classifier can be trained** to use customer information like age, income, and credit debt levels to learn whether they will default. Given a new customer and the same information, but without the knowledge of the likelihood of defaulting, the trained classification model predicts whether the customer is likely to default. This is an example of a **binary classifier**, as its predictions are limited to **two possible classes**.



age	ed	employ	address	income	debtinc	creddebt	othdebt	default
41	3	17	12	176	9.3	11.359	5.009	1
27	1	10	6	31	17.3	1.362	4.001	0
40	1	15	14	55	5.5	0.856	2.169	0
41	1	15	14	120	2.9	2.659	0.821	0
24	2	2	0	28	17.3	1.787	3.057	1
41	2	5	5	25	10.2	0.393	2.157	0
39	1	20	9	67	30.6	3.834	16.668	0
43	1	12	11	38	3.6	0.129	1.239	0
24	1	3	4	19	24.4	1.358	3.278	1
36	1	0	13	25	19.7	2.778	2.147	0

age	ed	employ	address	income	debtinc	creddebt	othdebt	default
37	2	16	10	130	9.3	10.23	3.21	0

Now, consider the following example of a **multi-class classifier** used to help prescribe various drugs. Imagine that you collected data about a set of patients, all of whom suffered from the same illness. During their course of treatment, each patient **responded positively to one of three medications**. You can use this labeled dataset with a classification algorithm to build a classification model that can predict which drug might be appropriate for a future patient with the same illness.



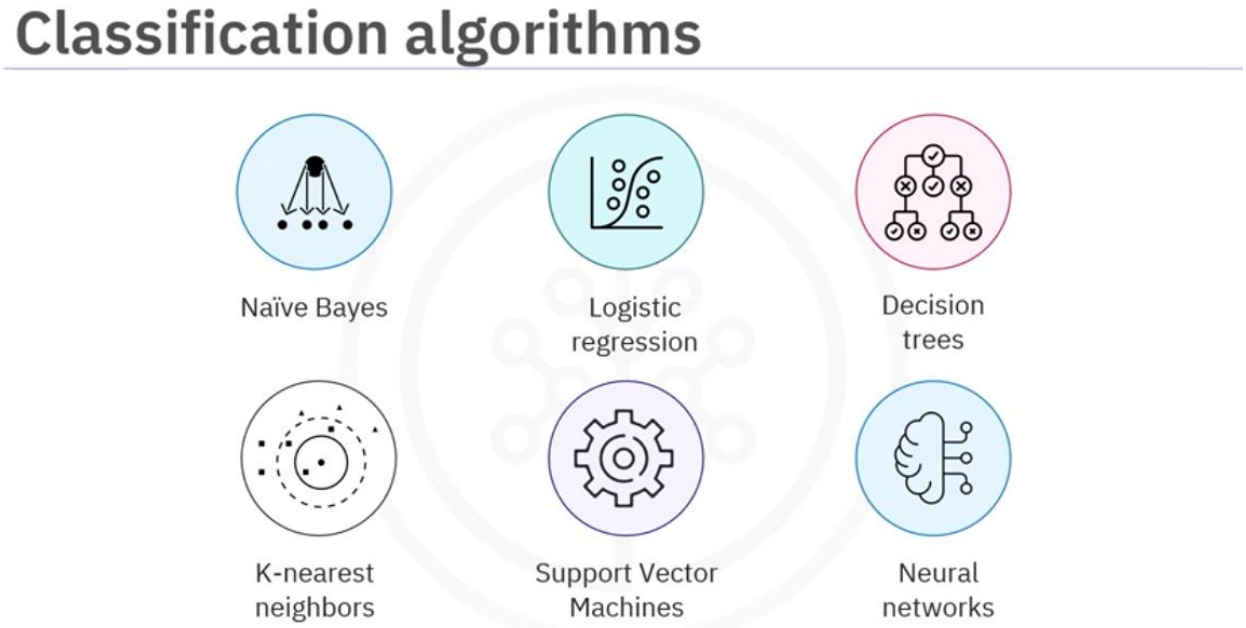
Age	Sex	BP	Cholesterol	Na	K	Drug
23	F	HIGH	HIGH	0.793	0.031	drugY
47	M	LOW	HIGH	0.739	0.056	drugC
47	M	LOW	HIGH	0.697	0.069	drugC
28	F	NORMAL	HIGH	0.564	0.072	drugX
61	F	LOW	HIGH	0.559	0.031	drugY
22	F	NORMAL	HIGH	0.677	0.079	drugX
49	F	NORMAL	HIGH	0.79	0.049	drugY
41	M	LOW	HIGH	0.767	0.069	drugC
60	M	NORMAL	HIGH	0.777	0.051	drugY
43	M	LOW	NORMAL	0.526	0.027	drugY

Age	Sex	BP	Cholesterol	Na	K	Drug
36	F	LOW	HIGH	0.697	0.069	

Classification algorithms.

You can use many different types of algorithms to build your classification model. Some common machine learning classification algorithms include **Naive Bayes, Logistic**

Regression, Decision Trees, K-Nearest Neighbors, Support Vector Machines, and Neural Networks.

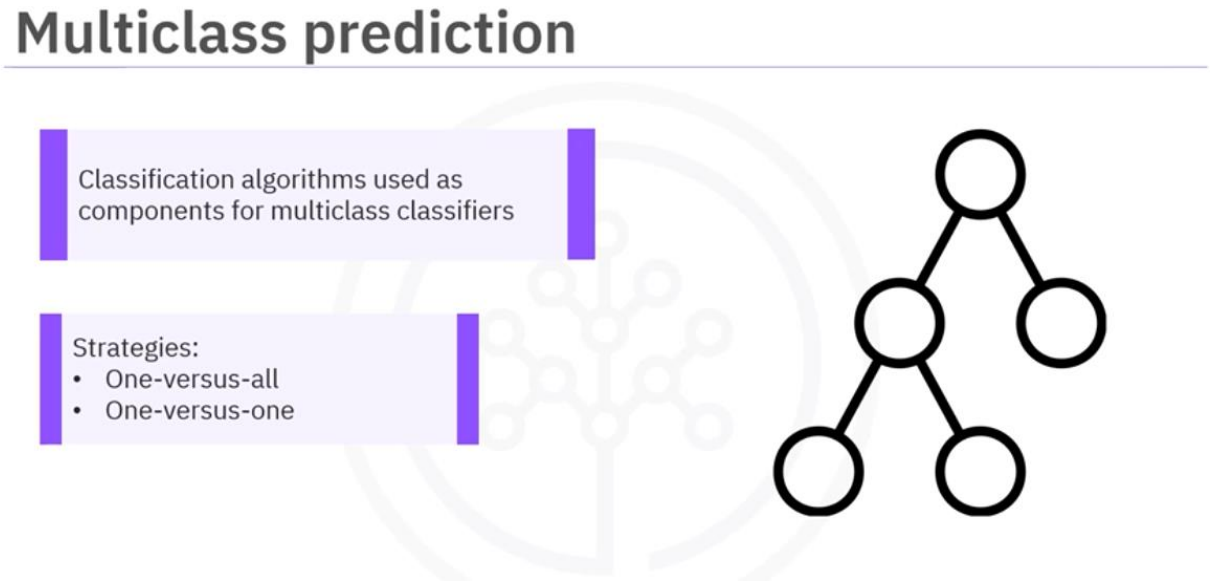


Algorithms like Logistic Regression, KNN, and Decision Trees can learn **how to distinguish multiple classes**.

Multiclass Predictions

Many classification **algorithms are not able to make distinctions between more than two classes**, but you can **use them as components for multi-class classifiers**.

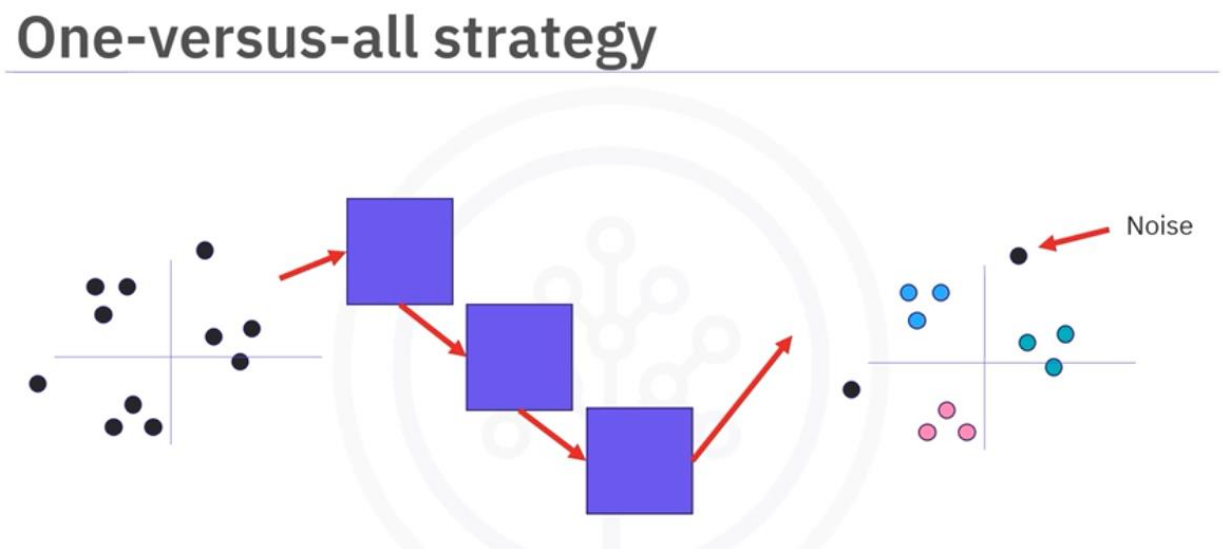
Strategies for extending binary classifiers to handle multiple classes include one-versus-all classification and one-versus-one classification.



One-versus-all

The one-versus-all scheme implements a **set of independent binary classifiers, one for each class label in the dataset**. Each classifier is assigned a single label that defines its target class. Each classifier's task is to make a binary prediction for every data point about **whether it has the given label, a one-versus-the-rest classifier**. As you can see, if there are k classes, there will be exactly k binary classifiers contributing.

Let's understand how the one-versus-all strategy decomposes a set of data points. The algorithm **works on each class label, one at a time**, with a trained outcome showing the points predicted to have a given color. Notice also **that a given data point might not belong to any of the classes**, as it might not get picked up by any of the individual classifiers.

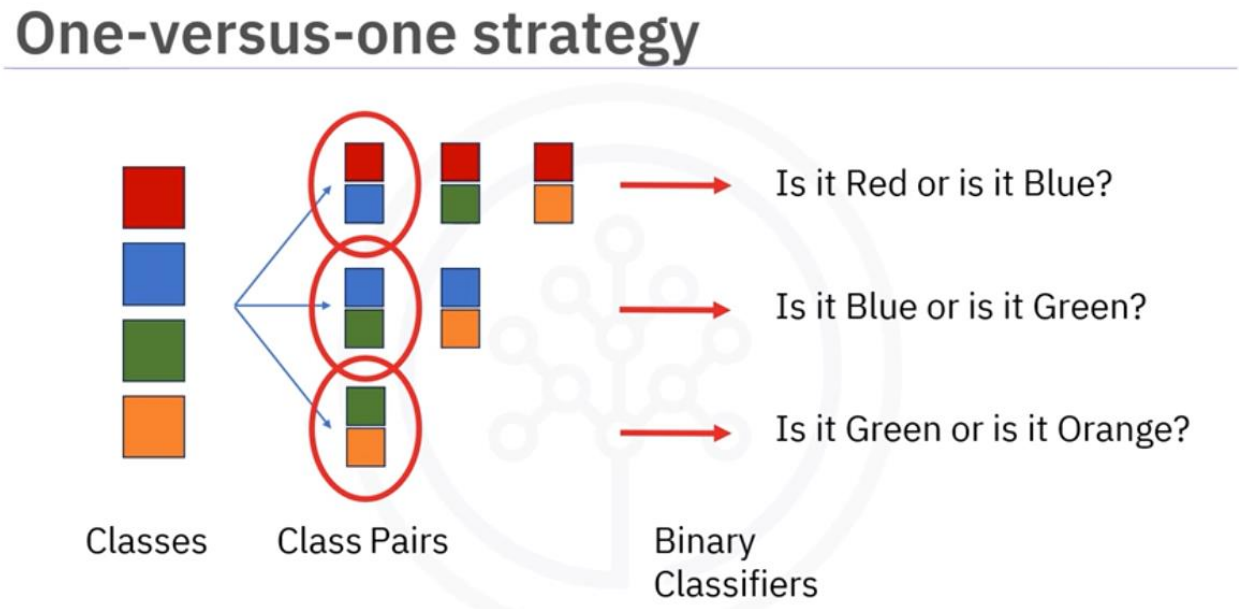


Such **unclassified points fall into another class**. This property might be useful for **identifying outliers or noise**.

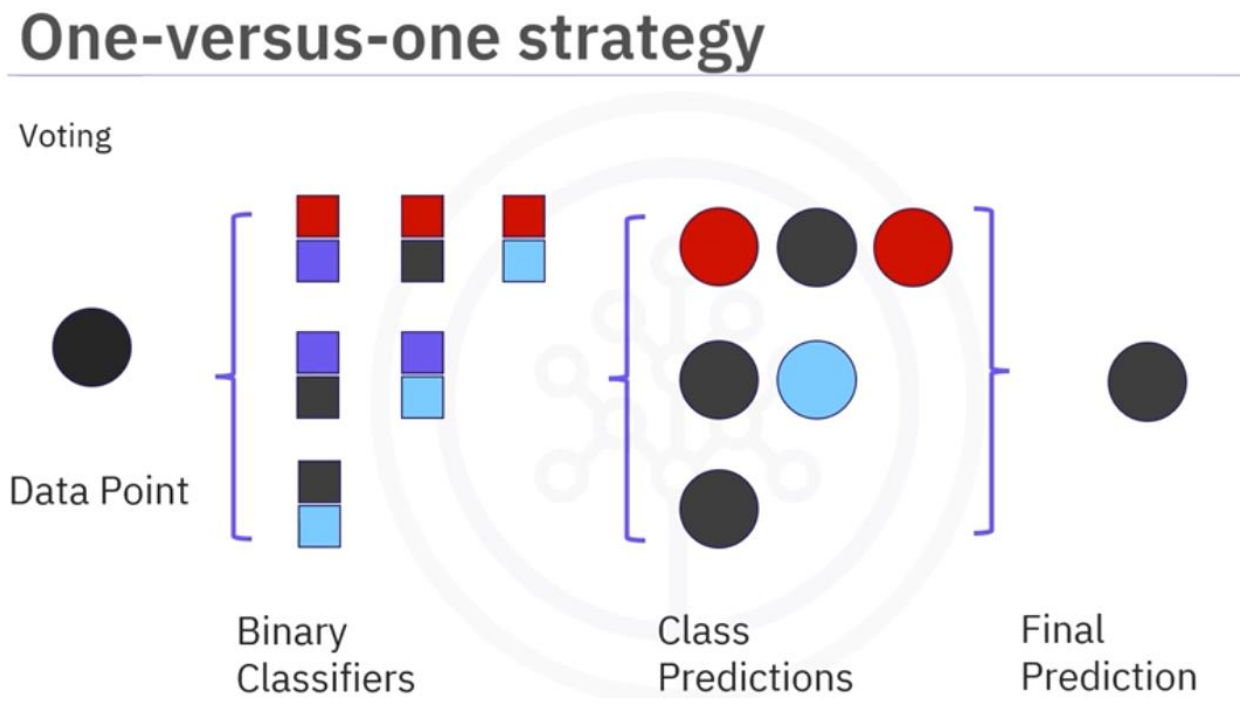
One-versus-one

With the one-versus-one strategy, instead of each classifier deciding whether a point belongs to a class, **the question changes from "Is it this?" to "Is it this or is it that?"**

Given a set of classes, consider all possible **pairs of classes**. **For each pair of labels, a classifier is trained on the subset of the data corresponding to the two labels** and decides which class each point belongs to. The process continues until all classifiers are trained.



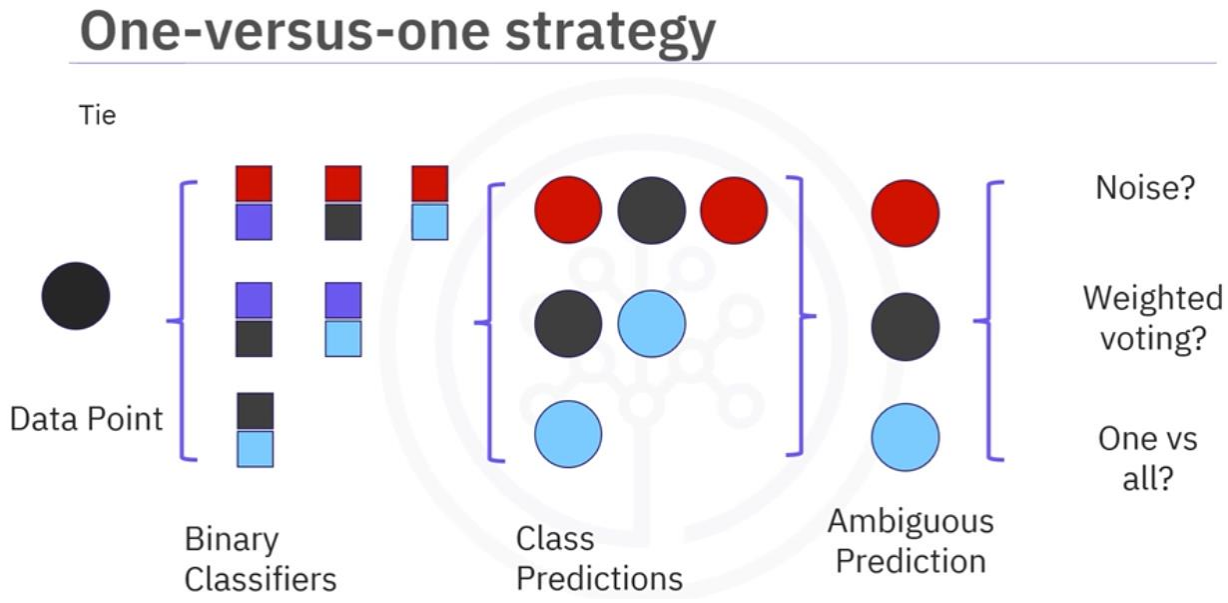
The final class label assigned to each point may be decided **by a voting scheme**. The **simplest scheme is by popularity**, meaning the class predicted by the most binary classifiers wins. Here, black is the winner.



What if there is a tie?

Here, we have three classes with the same number of votes. In a scenario where that is possible, it would be better to use an improved scheme, **weighing each vote by the confidence level or probability assigned to that class for each classifier**.

Alternatively, you could try **using one-versus-all classification instead.**



Recap

- **Classification** is a **supervised** ML method that uses **fully trained models** to predict labels on new data.
- Classification can be used for churn prediction, customer segmentation, and predicting advertising campaign responsiveness.
- Use cases of classification also include loan default prediction and **multi-class** drug **prescription**.
- Classification has several algorithms, which also include multi-class classifiers.
- Binary classifiers can be **extended to handle multiple classes** by using certain strategies.
    - The one-versus-all scheme implements independent binary classifiers, one for each class label.
    - The one-versus-one strategy answers the question, "Is it this or is it that?"

Multi-class Classification

Obesity risk prediction.

In 2111 samples, there are 7 obesity levels fairly balanced into requiring special attention in terms of biased training.

It is interesting how to extract continuous variables:

```
continuous_columns = data.select_dtypes(include=['float64']).columns.tolist()
```

```
['Age', 'Height', 'Weight', 'FCVC', 'NCP', 'CH2O', 'FAF', 'TUE']
```

```
scaled_features = scaler.fit_transform(data[continuous_columns])
```

With scaled features creates a scaled_df that joins to the original one dropped out from unscaled continuous columns

Converting to a DataFrame

```
scaled_df = pd.DataFrame(scaled_features, columns=scaler.get_feature_names_out(continuous_columns))
scaled_data = pd.concat([data.drop(columns=continuous_columns), scaled_df], axis=1)
```

Usa la misma técnica para las variables categoricas pero aplicando un encoder

```
encoder = OneHotEncoder(sparse_output=False, drop='first')
encoded_features = encoder.fit_transform(scaled_data[categorical_columns])
```

El **OneHotEncoder** se utiliza para convertir variables categóricas (como "rojo", "verde", "azul") en un formato numérico que los modelos de machine learning pueden procesar. Los argumentos sparse_output y drop controlan el formato de la salida y la estructura de las columnas resultantes.

Argumento sparse_output

El argumento **sparse_output** (salida dispersa) determina el tipo de estructura de datos que generará el OneHotEncoder.

- **sparse_output=True** (valor por defecto): El encoder crea una **matriz dispersa** (sparse matrix). Este formato es extremadamente eficiente en memoria cuando las variables tienen muchas categorías, ya que solo almacena las ubicaciones de los valores 1 (los valores 0 se omiten). Es ideal para conjuntos de datos grandes.
- **sparse_output=False** (como en tu código): El encoder devuelve un **array de NumPy** denso y completo. Este formato es más fácil de usar y manipular para la mayoría de las operaciones y para visualización, pero consume mucha más memoria si hay muchas categorías.

Argumento drop

El argumento **drop** se usa para evitar la **multicolinealidad**, un problema estadístico donde una variable puede ser predicha linealmente por otras. Esto es crucial para modelos como la regresión lineal que asumen que las variables de entrada son independientes.

- **drop=None** (valor por defecto): El encoder crea una columna binaria para cada categoría única. Esto genera redundancia, ya que si tienes N categorías, conocer el valor de N-1 columnas te permite inferir el valor de la columna restante. Por ejemplo, en las categorías ['rojo', 'verde', 'azul'], si las columnas is_verde y is_azul son 0, sabes que la categoría es "rojo" sin necesidad de la columna is_rojo.
- **drop='first'** (como en tu código): El encoder elimina la primera categoría de cada variable. Al codificar ['rojo', 'verde', 'azul'], el encoder solo crearía las columnas is_verde y is_azul. Esto elimina la multicolinealidad, ya que la información de la categoría "rojo" queda implícita (cuando is_verde y is_azul son ambos 0).

To encode the target variable, convert it to type Category.

```
prepped_data['NObeyesdad'] = prepped_data['NObeyesdad'].astype('category').cat.codes
```

La instrucción se puede entender en tres partes:

1. **prepped_data['NObeyesdad'].astype('category')**:
    - Esta parte selecciona la columna NObeyesdad y convierte su tipo de dato a **category**.

- Este tipo de dato de pandas **es más eficiente en memoria** para columnas con un número limitado de valores únicos (como las etiquetas de categorías) y es el paso necesario para usar el siguiente método.

2. **.cat.codes**:
    - Una vez que la columna es de tipo category, el atributo .cat permite acceder a las propiedades categóricas.
    - El método **.codes** extrae los códigos numéricos asociados a cada categoría. Por defecto, pandas asigna un código entero a cada valor único, generalmente en orden alfabético.
3. **prepped_data['NObeyesdad'] = ...**:
    - Finalmente, el resultado de los pasos anteriores (la serie de números enteros) se asigna de vuelta a la misma columna NObeyesdad, **reemplazando** los valores de texto originales con sus equivalentes numéricos.

Logistic Regression with One-vs-All

- If there are k classes, k classifiers are trained.
- During prediction, the algorithm evaluates all classifiers on each input, and selects the class with the **highest confidence score as the predicted class**.

Advantages:

- Simpler and more efficient in terms of the number of classifiers (k)
- Easier to implement for algorithms that naturally provide confidence scores (e.g., logistic regression, SVM).

Disadvantages:

- Classifiers may struggle with class imbalance since each binary classifier must distinguish between one class and the rest.
- Requires the classifier to perform well even with highly imbalanced datasets, as the "all" group typically contains more samples than the "one" class.
-

```
model_ova = LogisticRegression(multi_class='ovr', max_iter=1000)
```

one versus rest

Y seguimos el procedimiento establecido de entrenar, predecir, medir

```
model_ova.fit(X_train, y_train)
y_pred_ova = model_ova.predict(X_test)
accuracy_score(y_test, y_pred_ova)
```

Logistic Regression with OvO

- If there are k classes, this results in classifiers.

- During prediction, all classifiers are used, and a "voting" mechanism decides the final class by selecting the class **that wins the majority of pairwise comparisons**.

Advantages:¶

- Suitable for algorithms that are computationally expensive to train on many samples because each binary classifier deals with a smaller dataset (only samples from two classes).
- Can be more accurate in some cases since classifiers focus on distinguishing between two specific classes at a time.

Disadvantages:

- Computationally expensive for datasets with a large number of classes due to the large number of classifiers required.
- May lead to ambiguous predictions if voting results in a tie.

```
model_ovo = OneVsOneClassifier(LogisticRegression(max_iter=1000))
```

model_ovr.coef_ tiene los coeficientes de cada uno de los 7 clasificadores de este ejercicio. Por ese motivo para dibujar el peso específico de cada atributo trabajamos con la media.

```
feature_importance = np.mean(np.abs(model_ova.coef_), axis=0)
plt.barh(X.columns, feature_importance)
```

Los valores que ves en el array son los **coeficientes (o pesos)** que el modelo de regresión logística ha aprendido durante su entrenamiento.

Estos coeficientes representan la "importancia" y la dirección (positiva o negativa) de cada característica (variable independiente) para la predicción de la variable dependiente.

Explicación del Array

Dado que tu modelo se entrenó con la estrategia **multi_class='ovr'** (One-vs-Rest) y tu variable dependiente tenía siete categorías, el array resultante tiene la siguiente estructura:

- **Filas:** Hay **siete filas**, una para cada una de las siete categorías de tu variable dependiente. Cada fila corresponde a un clasificador binario separado que fue entrenado para esa clase en particular contra todas las demás.
- **Columnas:** Hay **veintitrés columnas**. Cada columna representa un coeficiente para una característica de tu conjunto de datos.

¿Qué significan los números?

- Un **coeficiente positivo** grande indica que el valor de esa característica aumenta la probabilidad de que una instancia pertenezca a la clase correspondiente a esa fila.
- Un **coeficiente negativo** grande indica que el valor de esa característica disminuye la probabilidad de que una instancia pertenezca a esa clase.
- Los valores cercanos a **cero** sugieren que la característica tiene poco o ningún impacto en la predicción de esa clase específica.

Por ejemplo, el valor -4.88338559e-01 en la primera fila, primera columna, es el coeficiente de la primera característica (la primera columna de tu conjunto de datos) para la predicción de la primera clase de tu variable dependiente. Este valor negativo sugiere que, para esa clase, un aumento en el valor de esa característica reduce la probabilidad de pertenencia.

En resumen, este array es el **corazón de tu modelo entrenado**. Contiene toda la información que el modelo utiliza para hacer predicciones, codificada en la relación que cada característica tiene con cada una de las siete categorías de la variable objetivo.

el clasificador **OneVsOneClassifier** por sí mismo **no tiene** un atributo **coef**_.

La razón es que **OneVsOneClassifier** es un **meta-estimador**. Esto significa que es un contenedor que utiliza múltiples modelos de base (en este caso, **LogisticRegression**) para resolver un problema de clasificación de múltiples clases.

El atributo **coef**_ pertenece a los modelos de **LogisticRegression individuales** que se entrenan dentro del contenedor OneVsOneClassifier.

Cómo acceder a los coeficientes

El OneVsOneClassifier almacena todos los clasificadores binarios que ha entrenado en el atributo **.estimators**_. Este atributo es una lista que contiene una instancia de LogisticRegression para cada par de clases.

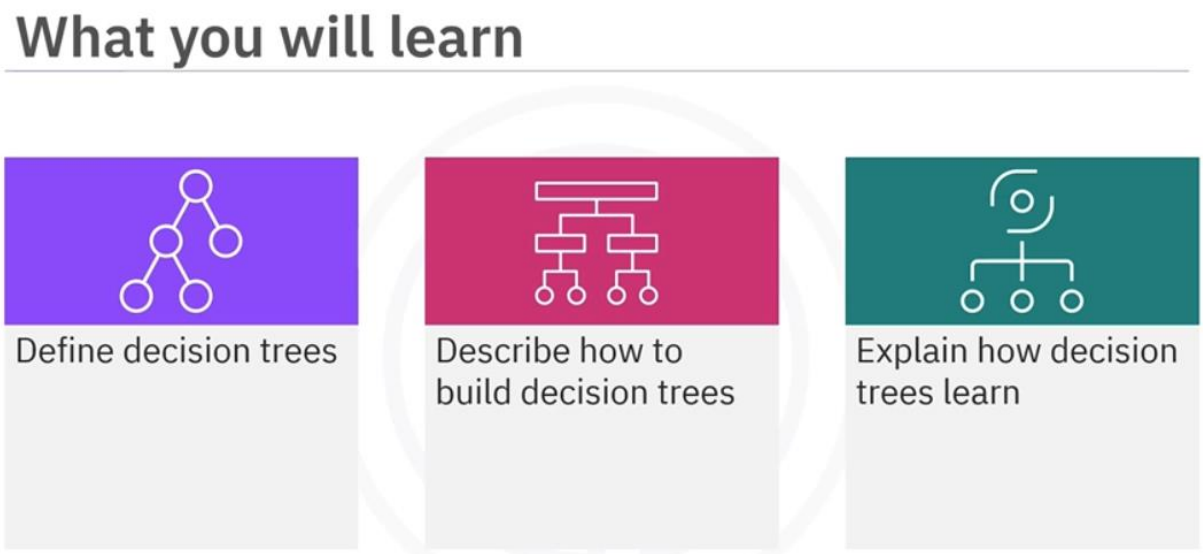
Para acceder a los coeficientes, debes iterar sobre esta lista. El número de clasificadores binarios entrenados es N * (N-1) / 2, donde N es el número de clases.

Python

```
# Accediendo a los coeficientes de cada clasificador
for i, estimator in enumerate(model_ovo_03.estimators_):
    print(f"Coeficientes para el clasificador binario {i+1}:")
    print(estimator.coef_)
```

En resumen, los coeficientes existen, pero están contenidos dentro de los modelos individuales que forman el OneVsOneClassifier, no en el modelo principal directamente.

Decision Trees

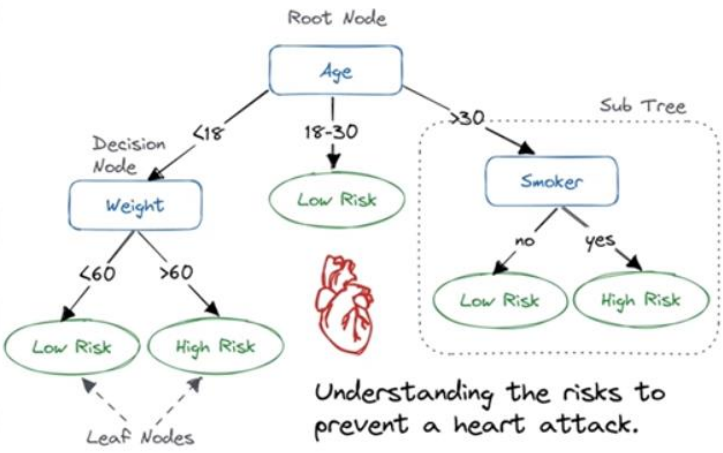


Definition

A Decision Tree is an algorithm that can be **visualized** as a flowchart for **classifying** data points.

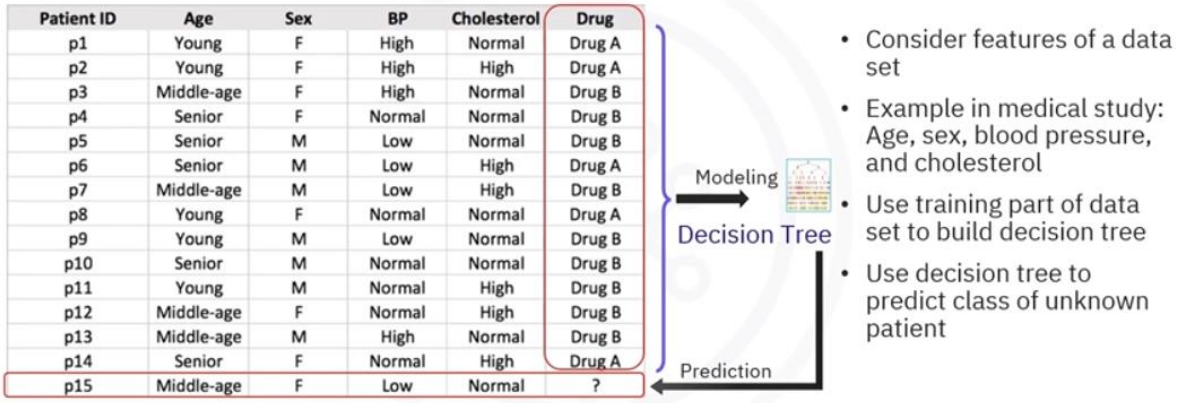
- Each **internal node** corresponds to a **test**.
- Each **branch** corresponds to the **result of the test**.
- Each terminal or **leaf** node assigns its **data to a class**.

- Each internal node corresponds to a test
- Each branch corresponds to the result of the test
- Each terminal, or leaf node, assigns its data to a class



Construction

A Decision Tree can be built by **considering the features of a dataset one by one**. Imagine that you are a researcher compiling data for a medical study. You would already have collected data about a set of patients who suffered from the same illness. During their course of treatment, each patient responded to one of two medications.

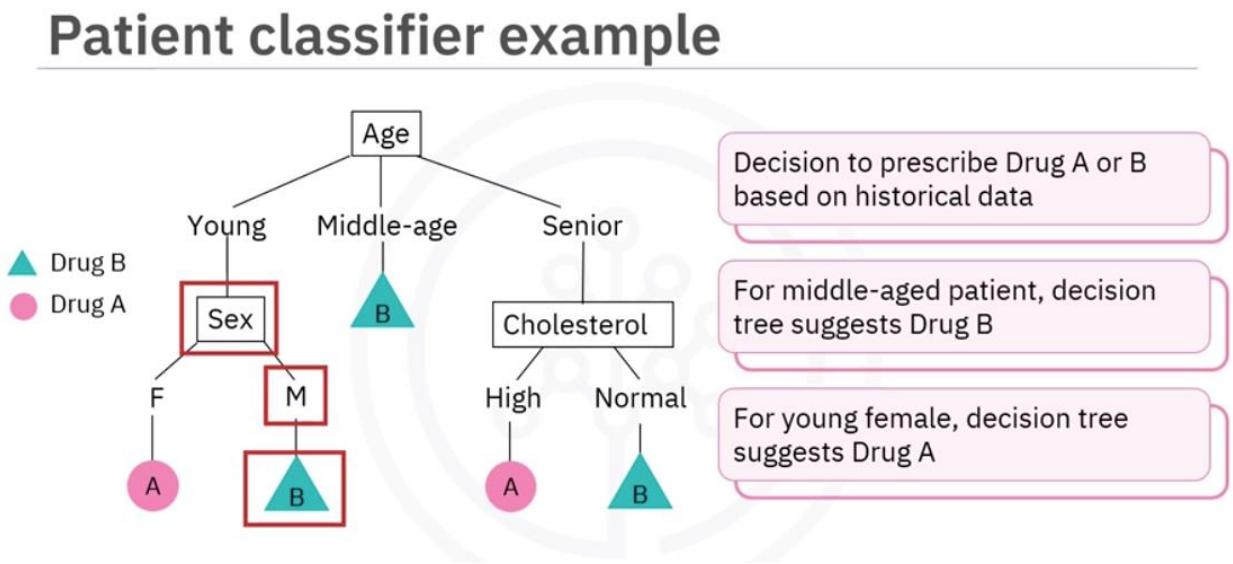


Patient ID	Age	Sex	BP	Cholesterol	Drug
p1	Young	F	High	Normal	Drug A
p2	Young	F	High	High	Drug A
p3	Middle-age	F	High	Normal	Drug B
p4	Senior	F	Normal	Normal	Drug B
p5	Senior	M	Low	Normal	Drug B
p6	Senior	M	Low	High	Drug A
p7	Middle-age	M	Low	High	Drug B
p8	Young	F	Normal	Normal	Drug A
p9	Young	M	Low	Normal	Drug B
p10	Senior	M	Normal	Normal	Drug B
p11	Young	M	Normal	High	Drug B
p12	Middle-age	F	Normal	High	Drug B
p13	Middle-age	M	High	Normal	Drug B
p14	Senior	F	Normal	High	Drug A
p15	Middle-age	F	Low	Normal	?

- Consider features of a data set
- Example in medical study: Age, sex, blood pressure, and cholesterol
- Use training part of data set to build decision tree
- Use decision tree to predict class of unknown patient

Let's call them drug A and drug B. Suppose you want to build a model to predict which drug might be appropriate for a future patient with the same illness. The **features** of this dataset are age, gender, blood pressure, and cholesterol of our group of patients, and the **target** is the drug that each patient responded to. You would use the training part of the dataset to build a Decision Tree and then use it to predict the class of an unknown patient. In essence, to produce a decision on which drug the patient is likely to respond to. The decision to prescribe drug A or B will be based on historical data for a large set of patients diagnosed with the same disease.

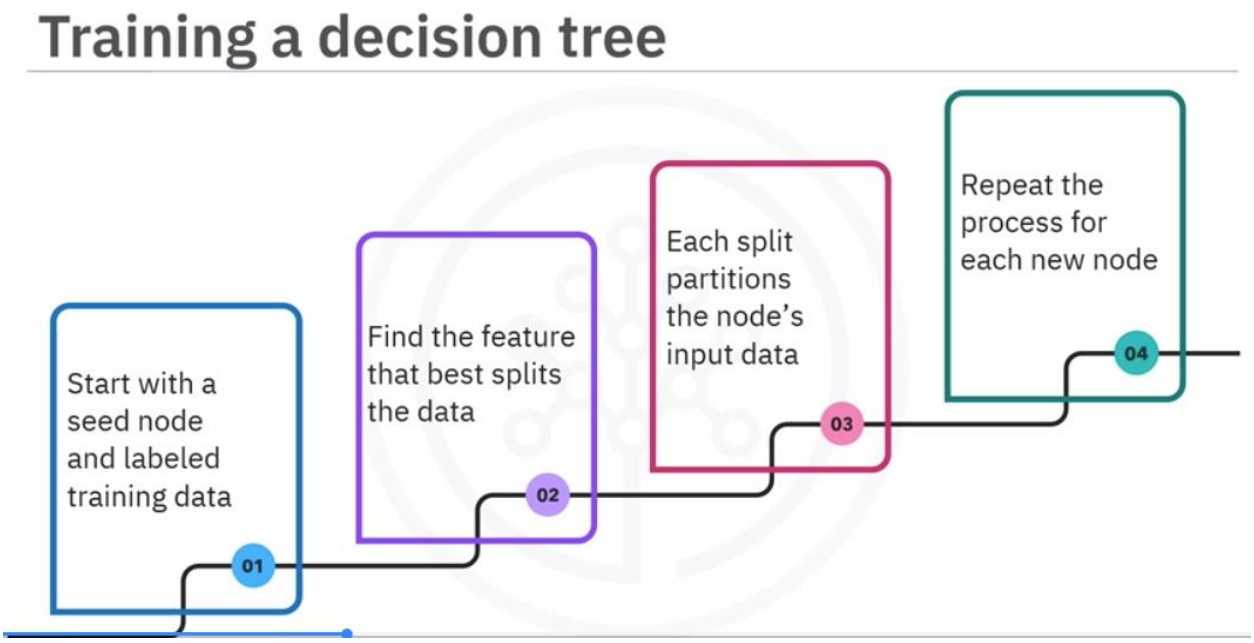
The tree starts by assigning a diagnosed patient to their age category, which can be young, middle-aged, or senior.



If the patient is middle-aged, the Decision Tree suggests drug B. It also suggests drug B if the patient is young and male, or a senior with normal cholesterol. On the other hand, if the patient is a young female or a senior with high cholesterol, the tree's branches lead to a prescription for drug A.

Learnt

A Decision Tree is trained by growing it as follows**. Start with a seed node** and labeled training data. **Train the node** on its assigned data by **finding the feature that best splits the data into its pre-labeled classes**, according to a pre-selected splitting criterion. Each such split partitions the node's input data, and each partition is passed along its branch to a new node.



Repeat the process for each new node, **using each feature only once**. **The tree grows until all nodes contain a single class each**, or **you run out of features** to select, or a pre-selected stopping criterion is met.

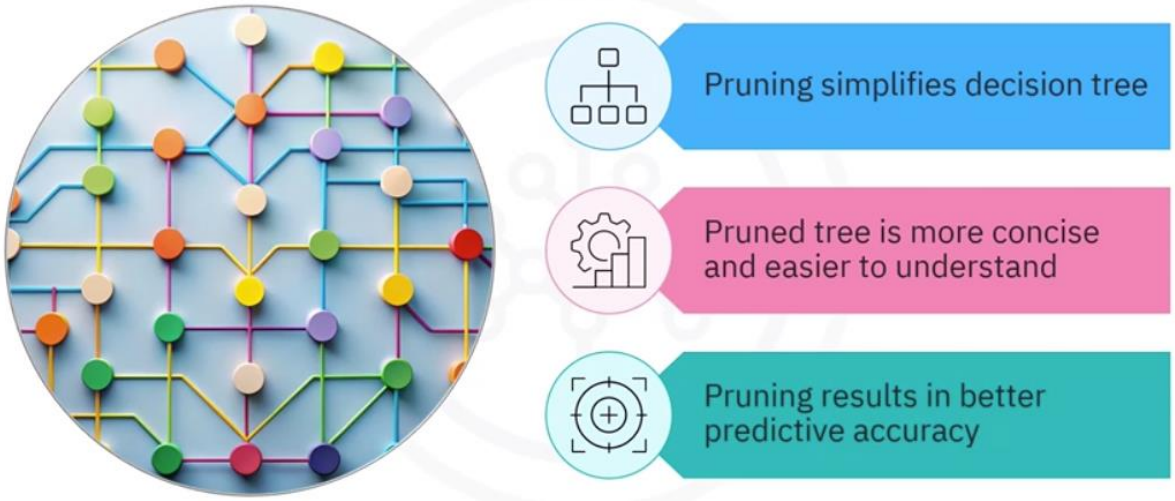
Prunning

A Decision Tree stops growing when a **stopping criterion** is met. This is also known as pre-emptive tree pruning. For instance, you can set stopping criteria for your model when the following criterion is met.

- Maximum tree **depth** is reached.
- Minimum **number of data points** in a node have been exceeded.
- Minimum number of samples in a leaf have been exceeded.
- Decision Tree has reached **the maximum number of leaf nodes**.
- Alternatively, you can also stop a tree from growing by **cutting branches that don't significantly improve system performance**.

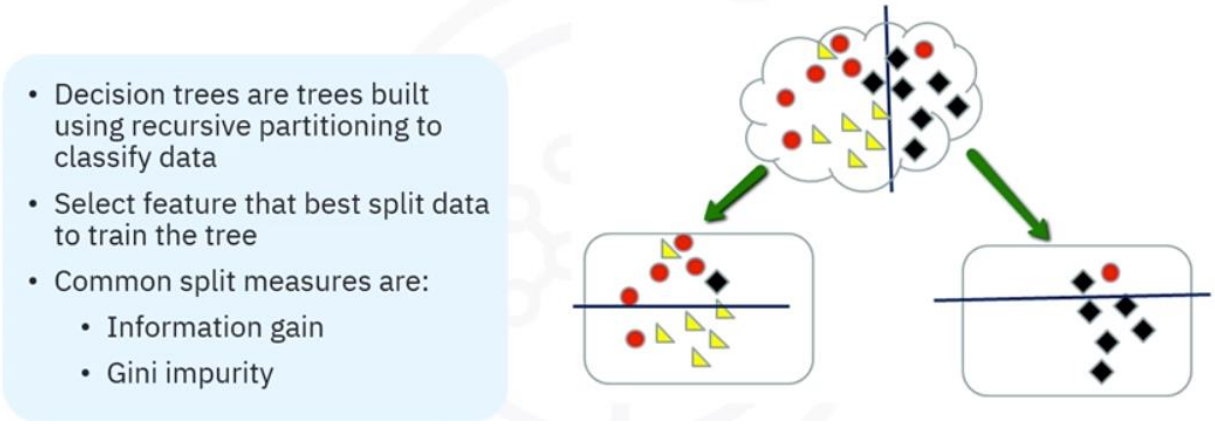
There are several reasons why you might want to prune a Decision Tree. If the tree is too complex, you might **be overfitting** it to the training data. If you have too many classes and features, the tree might **be capturing noise and irrelevant details**.

Why prune?



- Pruning simplifies your Decision Tree model and makes it **amenable to generalization**.
- A pruned tree is more **concise and easier to understand**.
- Pruning also results **in better predictive accuracy**.

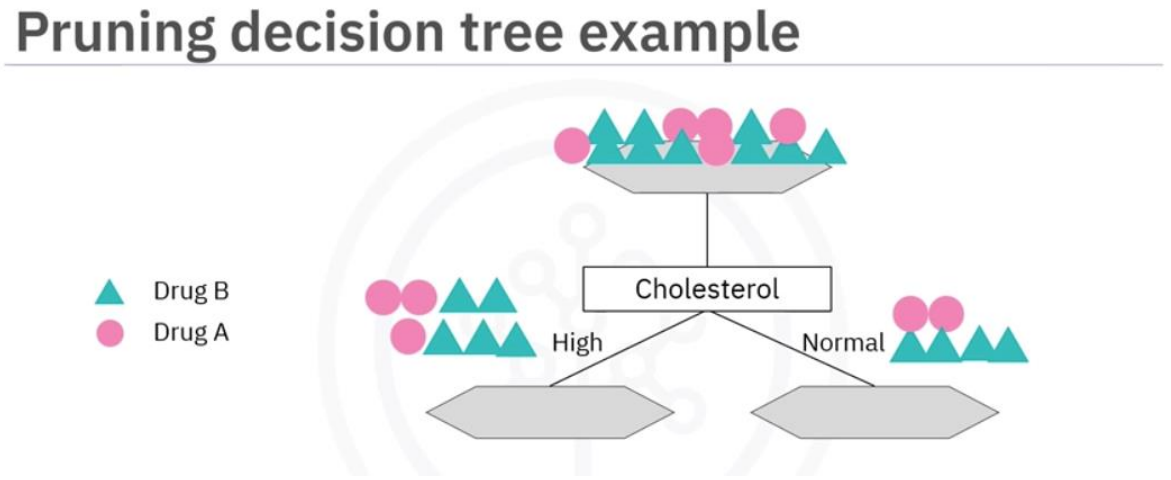
Which is the best feature?



Decision Trees are built using recursive partitioning to classify the data. The Decision Tree algorithm must **select a feature that best splits the data at each node to train a tree**. To do this, you must select a splitting criterion **to measure the split quality** for determining the best split. Two common split measures are information gain, which is also called entropy reduction, and Gini impurity.

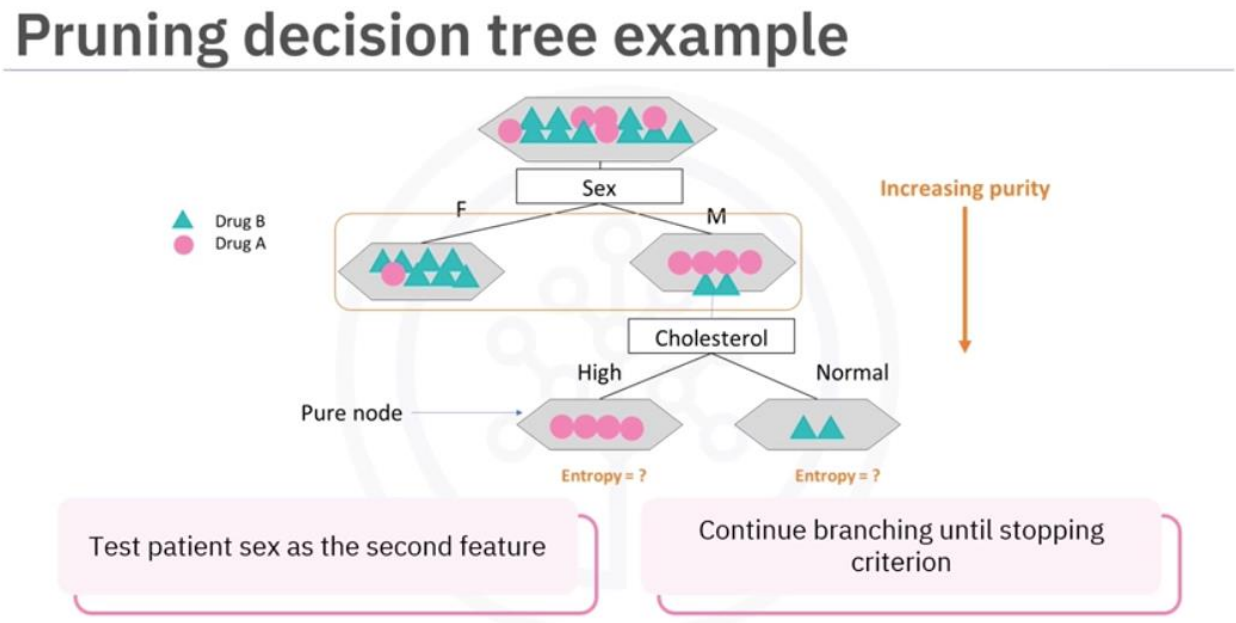
Consider the 14 patients in our data set.

The Decision Tree algorithm chooses the most predictive feature to split the data on, for example, the **feature that best distinguishes the patient classes it assigns**.



Suppose it starts by testing cholesterol as the first feature to split on. The tree assigns patients to two nodes, high and normal. As you can see, if the patient has high cholesterol, **we cannot say with high confidence** that drug B might be suitable for him. Also, even if the patient's cholesterol is normal, we still don't have sufficient evidence or information to determine whether either drug A or drug B is suitable. Cholesterol might not be the best attribute to split on.

We're looking for the best feature **to decrease impurity of patients in the leaves**, so let's try another feature.



This time, we pick the sex feature of patients. The Decision Tree splits patients into two branches, male and female. For females, the sex split classifies most patients as drug B. For males, the distinction between drug A and B diagnoses is less clear. Further splitting the male node using the cholesterol feature results in **two pure nodes**: **the terminal leaves in which all patients fall into a single class or prescription**. The algorithm continues branching until it reaches a stopping criterion.

Entropy

Entropy is the **measure of information disorder**, or randomness in a data set.

It measures how random the classes in a node are, or how uncertain the feature split result is. In Decision Trees, **you look for trees that have the smallest entropy in their nodes**. You can calculate the entropy of a node using the entropy formula, where pA and pB are, respectively. The proportions of drug A and drug B patients in the node. If the classes are completely homogenous, the entropy is 0, and if they are equally divided, the entropy is 1. For example, if pA equals pB equals 1 half, then the formula yields 1 half times negative 1, minus 1 half times negative 1 equals 1. You don't need to calculate these, of course, as it's calculated by the libraries or packages that you use.

En el contexto de **árboles de decisión** (por ejemplo ID3, C4.5 o CART con criterio de entropía), la entropía mide la impureza de un nodo, es decir, cuán mezcladas están las clases de los ejemplos que caen en ese nodo.

La fórmula general es:

$$H(S) = -\sum_{i=1}^{k} p_i \log_2(p_i)$$

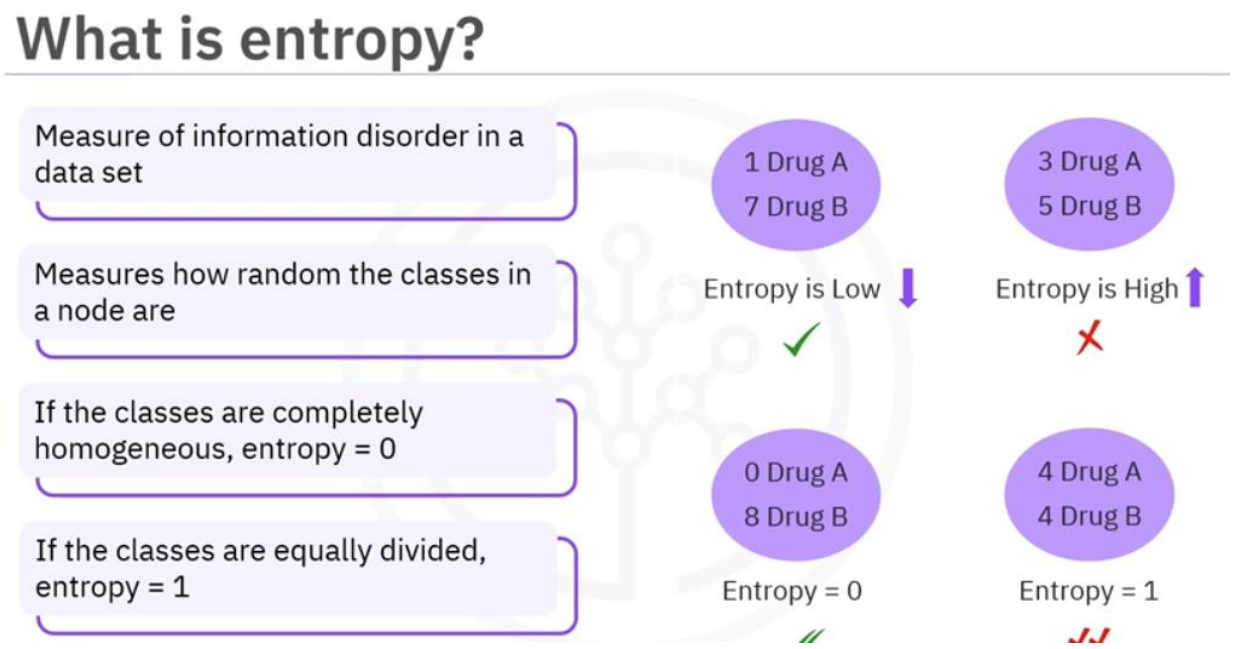
donde:

- Ses el conjunto de ejemplos en el nodo,
- $k$es el número de clases,
- $p_i$es la proporción de ejemplos de la clase $i$en el nodo, es decir:

$$p_i = \frac{\text{número de ejemplos de clase } i}{\text{total de ejemplos en el nodo}}$$

Observaciones

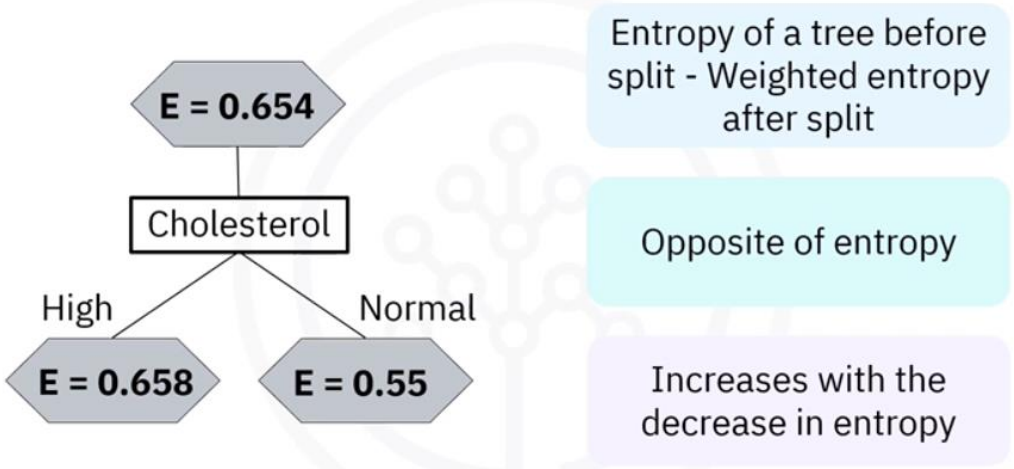
- Si todos los ejemplos del nodo pertenecen a una misma clase, entonces $p_i = 1$para esa clase y 0 para las demás, con lo que $H(S) = 0$(nodo puro).
- Si las clases están distribuidas de manera uniforme, la entropía es máxima. Por ejemplo, con dos clases y $p_1 = p_2 = 0.5$, se obtiene $H(S) = 1$.
- Se utiliza la base 2 porque se mide en **bits**, aunque también puede usarse logaritmo natural.



Information gain

Information gain is the **entropy of a tree before the split minus the weighted entropy after the split by a feature**. For example, using cholesterol as the feature to split on for all patients yields an information gain of 0.042.

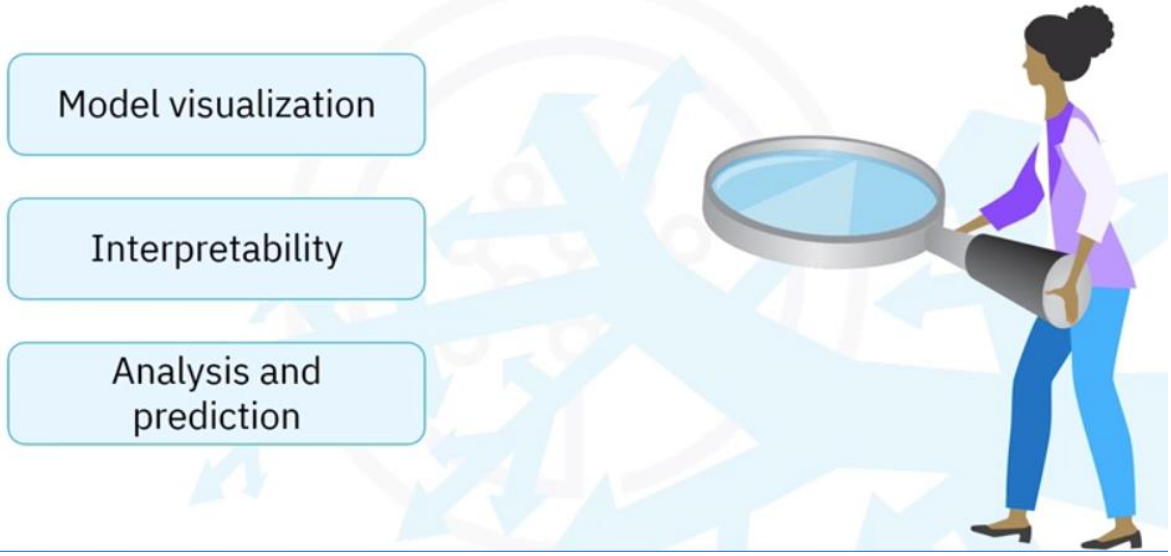
What is information gain?



You can consider **information gain and entropy as opposites**. As entropy decreases, the information gain, or amount of certainty, increases. **Constructing a decision tree is all about finding features that return the highest information gain**.

Decision trees are advantageous because they can be **visualized**. This means you can see exactly how it makes decisions, which makes them highly interpretable.

Advantages of decision trees



Since the tree grows by gradually selecting the next best feature to split on, you can gain **insights about how important or predictive each feature is**.

Recap

- A decision tree is an algorithm for classifying data points
- Decision trees are built by considering data set features
- In a decision tree:
  - Each internal node corresponds to a test
  - Each branch corresponds to the result of the test
  - Each terminal, or leaf node, assigns its data to a class

In this video, you learned how to train a decision tree, how to prune a decision tree, and how to select the features that best splits the data at each node when you're training a tree. You also learned about the information gain and Gini impurity split measures.

Decision trees help in visualizing a data model and predicting outcomes based on the information in a dataset.

1. Índice de Gini de un nodo

El **índice de Gini** mide la probabilidad de clasificar incorrectamente un ejemplo si se asigna una clase de manera aleatoria siguiendo la distribución de clases del nodo.

La fórmula es:

$$Gini(S) = 1 - \sum_{i=1}^{k} p_i^2$$

donde:

- Ses el conjunto de ejemplos en el nodo,
- $k$es el número de clases,
- $p_i$es la proporción de ejemplos de la clase $i$en el nodo.

Características:

- Si el nodo es puro (todos los ejemplos de una sola clase), $Gini(S) = 0$.
- Si las clases están distribuidas de manera uniforme (ej. en dos clases, 50%-50%), el índice Gini es máximo.

---

2. Medida de división (Gini split)

Cuando evaluamos un **atributo candidato para dividir el nodo**, se calcula un **Gini ponderado** de los subconjuntos resultantes:

$$Gini_{split}(S, A) = \sum_{j=1}^{m} \frac{|\ S_j\ |}{|\ S\ |} Gini(S_j)$$

donde:

- Aes el atributo que divide Sen subconjuntos $S_1, S_2, \dots, S_m$,
- $|\ S_j\ |$es el número de ejemplos en el subconjunto S_j.

Se selecciona el atributo (y la división) que **minimiza** $Gini_{split}$.

---

3. Uso en algoritmos

- El índice de Gini es el criterio estándar usado en **CART (Classification And Regression Trees)**.
- Es conceptualmente similar a la entropía, pero más simple de calcular.
- Mientras la entropía proviene de la teoría de la información, el índice de Gini es más directamente una medida de "impureza de clasificación".

Lab Decision Trees

Categorial features are converted into numericals one with LabelENcoder.

Strictly speaking, the need to transform **categorical features into numerical ones** depends on the implementation of the decision tree, not on the theory of decision trees themselves. Let me explain in detail:

---

1. Theoretical perspective

Decision trees split data based on feature values to maximize some criterion (e.g., information gain, Gini index).

- If a feature is **categorical**, in principle the tree can partition the dataset by grouping categories in different subsets (e.g., splitting on {red, blue} vs. {green, yellow}).
- If a feature is **numerical**, the tree usually splits on thresholds (e.g., $x > 3.5$).

So, from theory alone, decision trees can naturally handle categorical features.

---

2. Practical perspective (implementations in software)

Most **machine learning libraries** (like scikit-learn in Python) expect input features to be **numerical arrays (float or int)**. This has two consequences:

- The implementation does not store categorical metadata. It only works with numbers and assumes comparisons like $x < t$make sense.
- If you pass raw categories as strings (e.g., "red", "blue"), the algorithm cannot compute splits because it doesn't know how to compare "red" < "blue" in a meaningful way.

That is why **encoding is required**:

- **Label encoding** (maps each category to an integer) is simple, but can be misleading, because it introduces an artificial order.
- **One-hot encoding** (binary dummy variables per category) avoids artificial ordering and is widely used with decision trees.

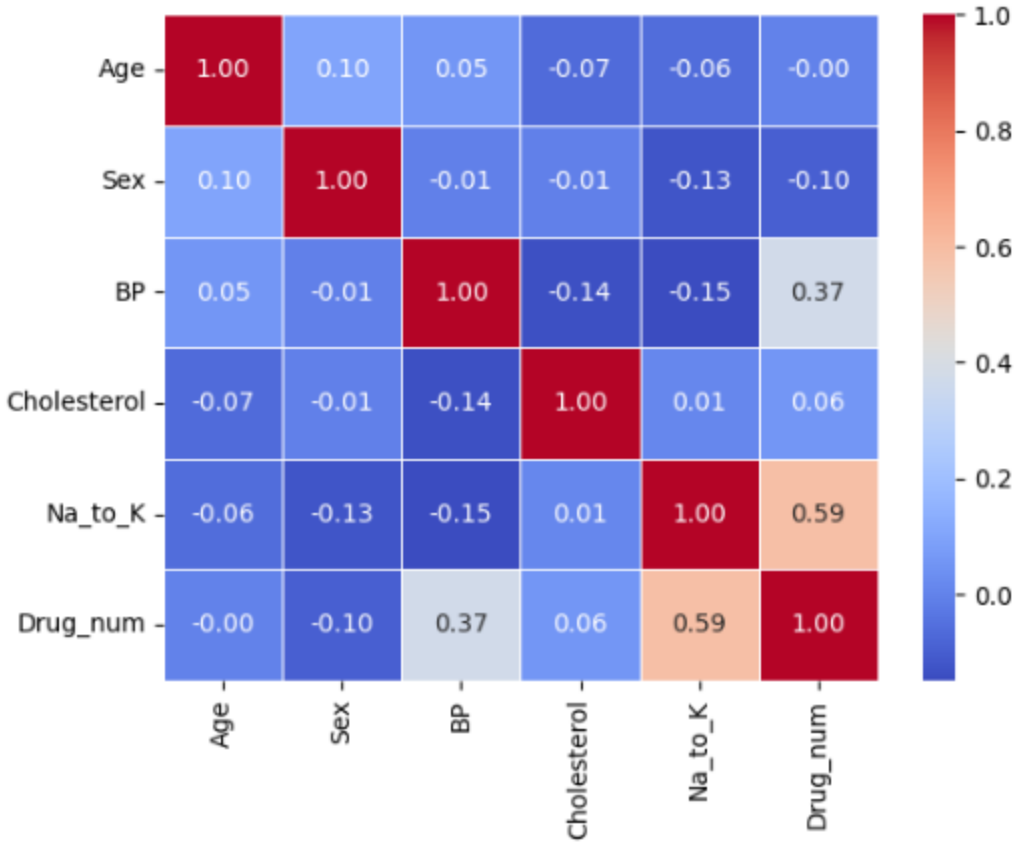
---

3. Exceptions

Some implementations **natively support categorical features** without transformation:

- **LightGBM** and **CatBoost** can handle categorical variables directly.
- Newer versions of scikit-learn (from 1.1 onward) have limited support for categorical features in trees, but it's still less common.

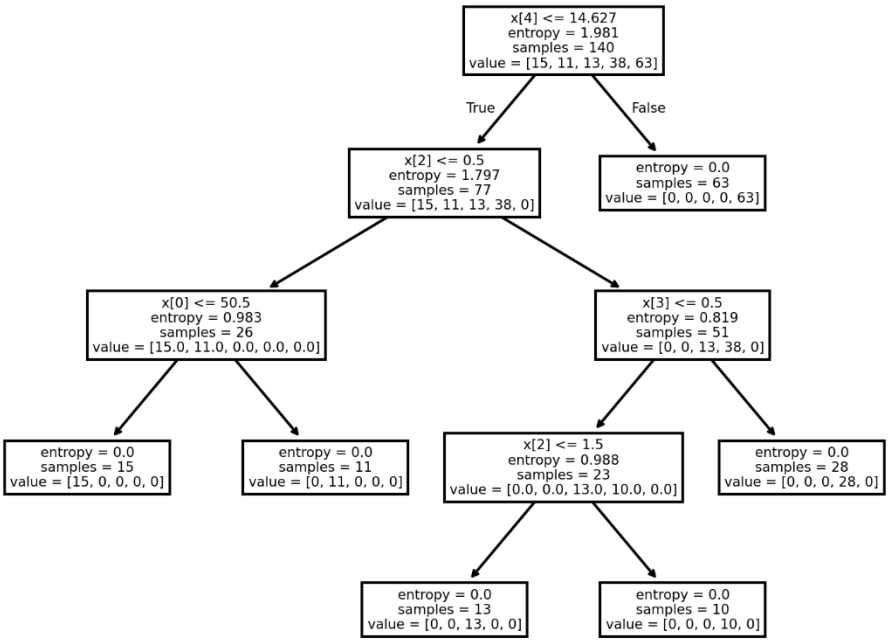
Convierte las categorias de la variable dependiente en números para ver la correlación matemática entre las variables independientes y la variable dependiente,



La presion sanguínea y la relacion sodio potasio son las variables independientes que mas se relacionan con la variable dependiente.

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
drugTree = DecisionTreeClassifier(criterion="entropy", max_depth = 4)
```

El criterio por defecto es Gini en el Decision Tree de Sklearn.



In the tree visualization, x[i] refers to **the i-th column (feature)** of your X_train matrix. However, by default, scikit-learn does not know the feature names — it only stores their **column indices**.

The **value** array corresponds to the **number of samples of each target class** at that node. The order of classes is given by: drugTree.classes_

array(['drugA', 'drugB', 'drugC', 'drugX', 'drugY'], dtype=object)

A medida que aumenta la profundida del arbol aumenta la precision hasta que estabiliza

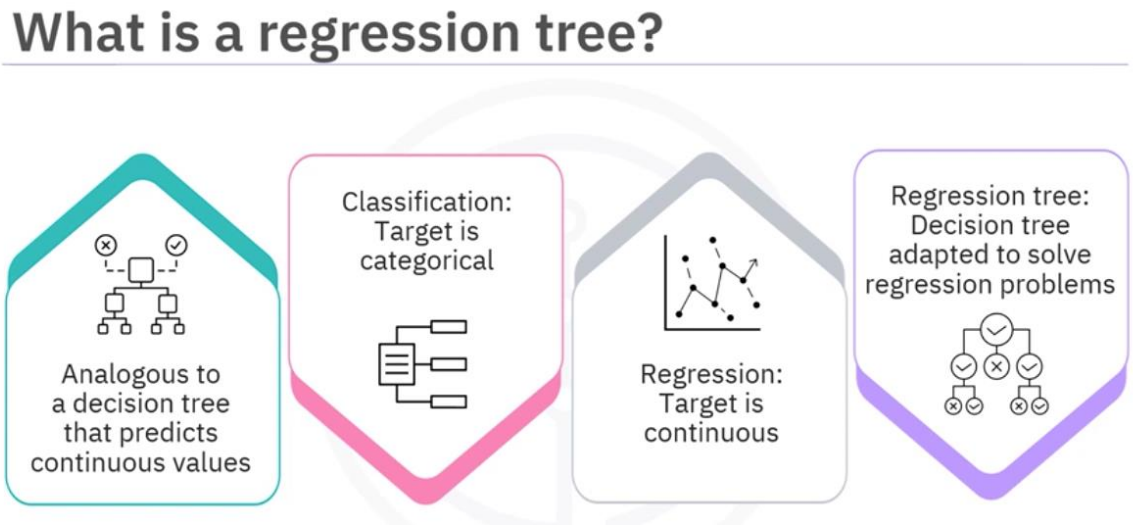
Depth 1 Decision Trees's Accuracy: 0.7166666666666667
Depth 2 Decision Trees's Accuracy: 0.85
Depth 3 Decision Trees's Accuracy: 0.8166666666666667
Depth 4 Decision Trees's Accuracy: 0.9833333333333333
Depth 5 Decision Trees's Accuracy: 0.9833333333333333

Regression Trees

Welcome to Regression Trees. After watching this video, you will be able to describe a regression tree and recognize how it is different from classification. You will also be able to explain how to create a regression tree.

Description

A regression tree is analogous to a decision tree that **predicts continuous values** rather than discrete classes. The distinguishing feature between classification and regression is the characteristic of the target or labeled data. In classification, the target variable is categorical, such as true or false. In regression, the target is a continuous value, such as temperature or salary.



When a decision tree is adapted to solve regression problems, it is called a regression tree.

Difference between classification and regression trees

Let's compare classification trees with regression trees to understand how they're different.

The target of a classification tree is to **classify data into discrete sets**, whereas a regression tree aims **at predicting continuous target variables**.

Hence, the target variable for a classification tree is categorical but floating for a regression tree.

The **prediction** at leaf nodes for a classification tree is a **class-labeled majority vote**, whereas for a regression tree, it is the **average value of target values**.

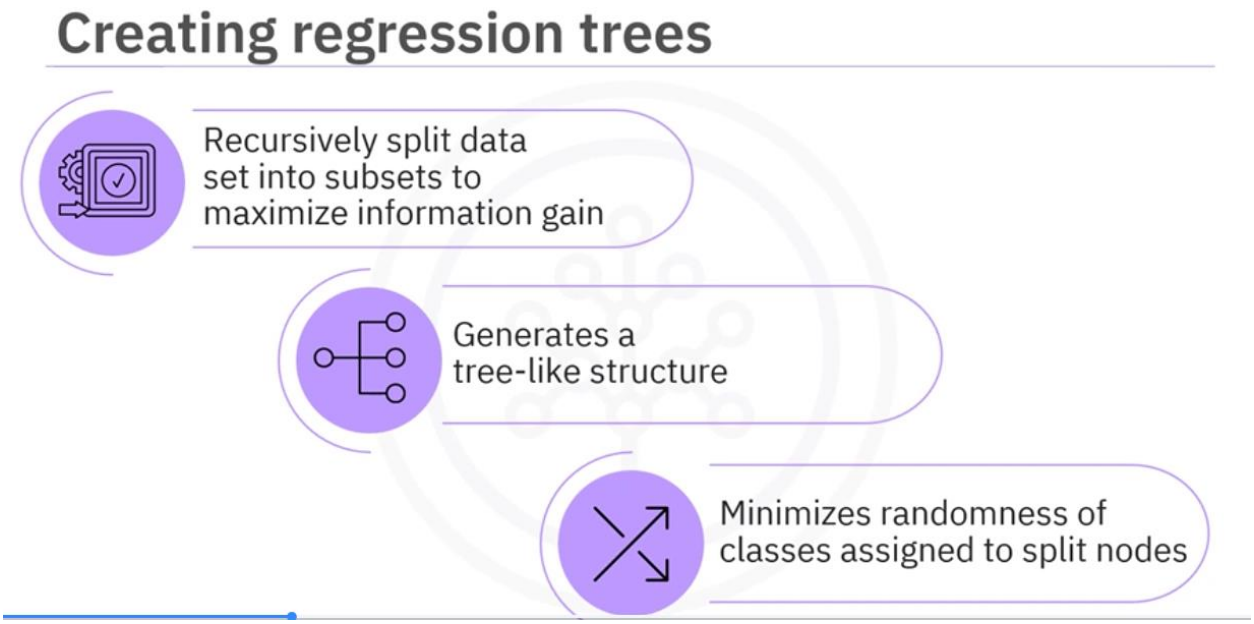
Classification versus regression trees

	Classification Trees	Regression Trees
Objective	Classify data into discrete sets	Predict continuous target variable
Target Variable	Categorical	Float
Splitting Criterion	Gini impurity or entropy	Variance reduction
Prediction at Leaf Nodes	Class label majority vote	Average value of target values
Example Use Cases	Spam detection, image classification, medical diagnosis	Predicting revenue, temperatures, wildfire risk

Some used cases of classification trees are spam detection, image classification, medical diagnosis. Regression trees are used for predicting revenue, temperatures, and wildfire risk.

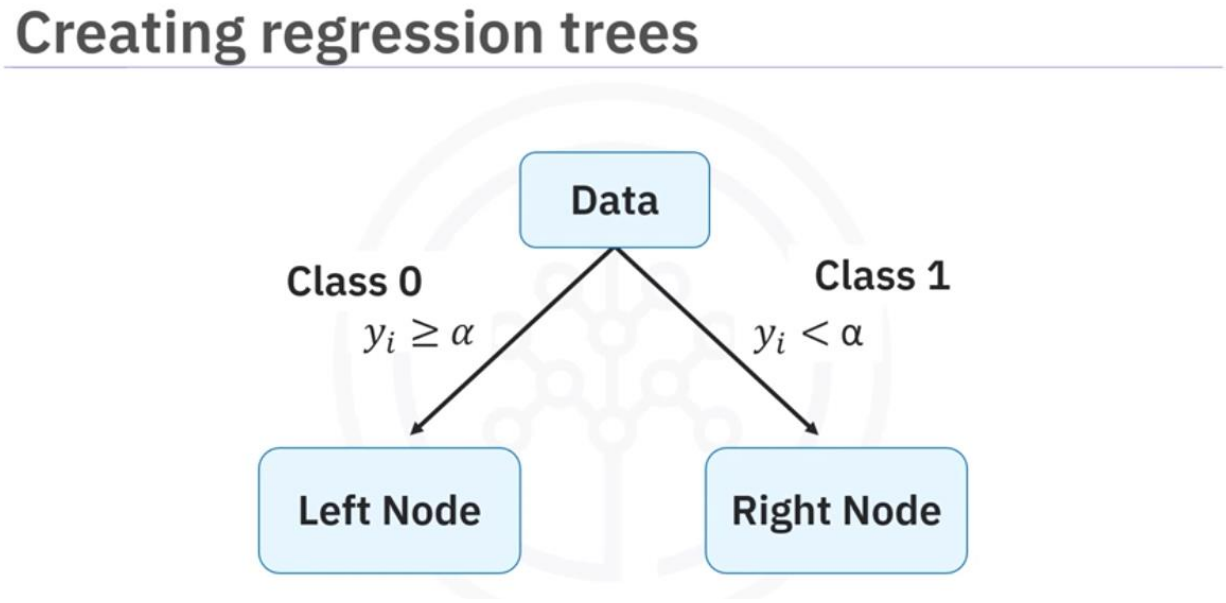
Creation

Regression trees are created by recursively splitting the dataset into subsets **to maximize information gained from data splitting**. This process generates a tree-like structure and **minimizes the randomness of the classes assigned to the split nodes**.



Let's consider this example. Given a continuous feature from a dataset and a trial **threshold value, alpha**, the data in a node is split into two subsets according to whether the data is

greater than or less than alpha, and the corresponding points are assigned to the **left and right nodes**. If the feature is binary, consisting of two classes, then the split is according to the two classes.



You make a **prediction at each node** based either on a class voting scheme as with decision trees or by using the **average of the target values in the node**. The predicted value, y-hat, for a given node is defined as the average of the actual target values, y, i of the data points in the node.

You could use other statistics, like the **median value**, to assign the prediction. This would be **preferable when your data is skewed**. For normally distributed data, the median is comparable to the mean, but **the median is more expensive to compute**.

Predicting values

Predicted value:

- Mean of target values

$$\hat{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$$

Alternative value:

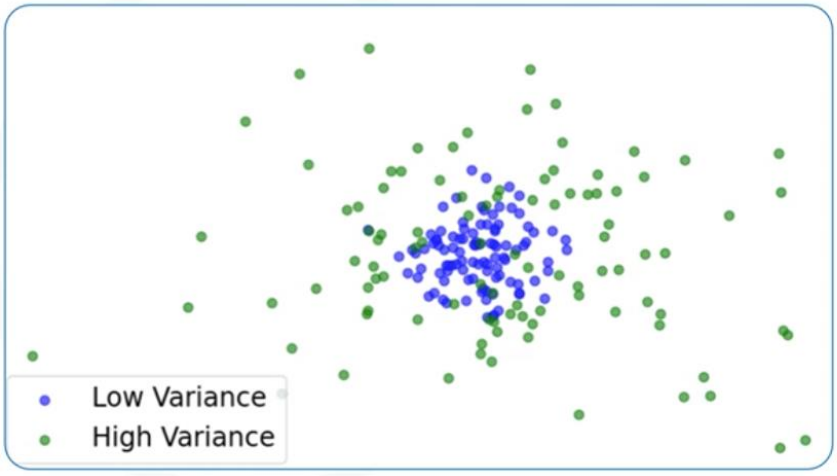
- Median of target values
- Better for skewed data
- More expensive

Instead of using the entropy or information gained criteria to measure the quality of a split, as for decision trees, **regression trees select features that minimize the error** between **the actual values**, y, i in the resulting nodes, and **the predicted value**, y-hat.

Splitting criterion

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y})^2$$

A measure of target variance



A natural criterion for measuring the split quality of a given feature uses the mean-squared error, or MSE. Notice that this amounts to measuring the variance of the target values **within each node**, which **gauges how spread out the values are**. The smaller the variance is, the more closely the values agree.

Quality of split

Weighted average of MSEs of each split:

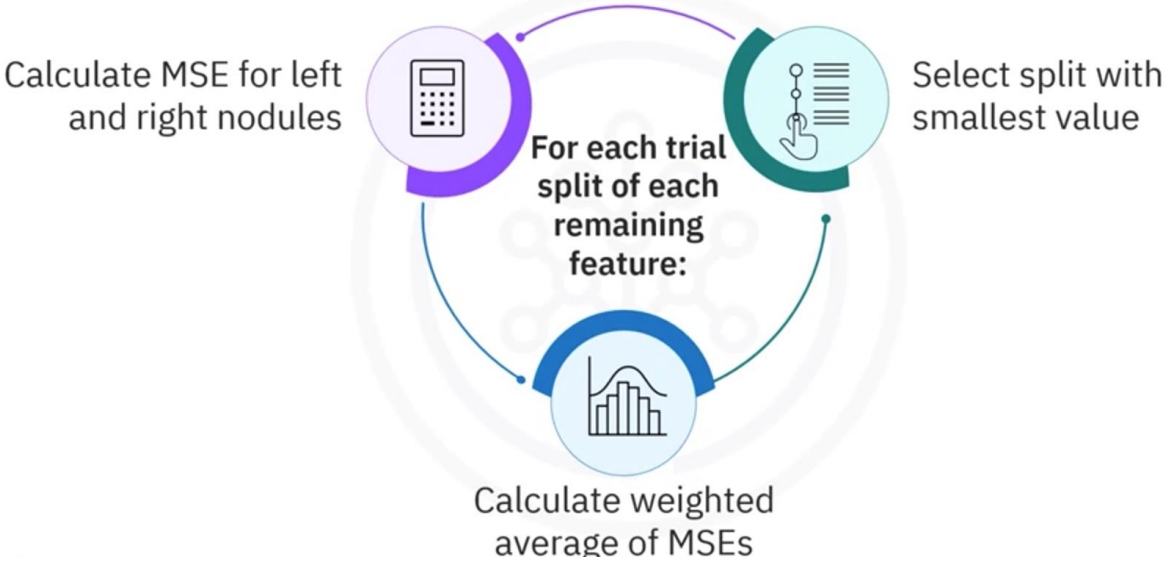
$$\mathrm{MSE}_{Avg} = \frac{1}{\mathrm{N}_{Total}} \left(\mathrm{N}_{Left} * \mathrm{MSE}_{Left} + \mathrm{N}_{Right} * \mathrm{MSE}_{Right} \right)$$

To measure the quality of a split, the weighted average of the MSEs of each split node can be used.

The weighted average is calculated as average MSE equals one over the **number of observations in the two split nodes**, times the **sum** of the **number of observations in the left split times the MSE of the left split**, and the **number of observations in the right split times the MSE of the right split**.

The **lower this value**, the lower the variance, and thus, the **higher the quality of the split**.

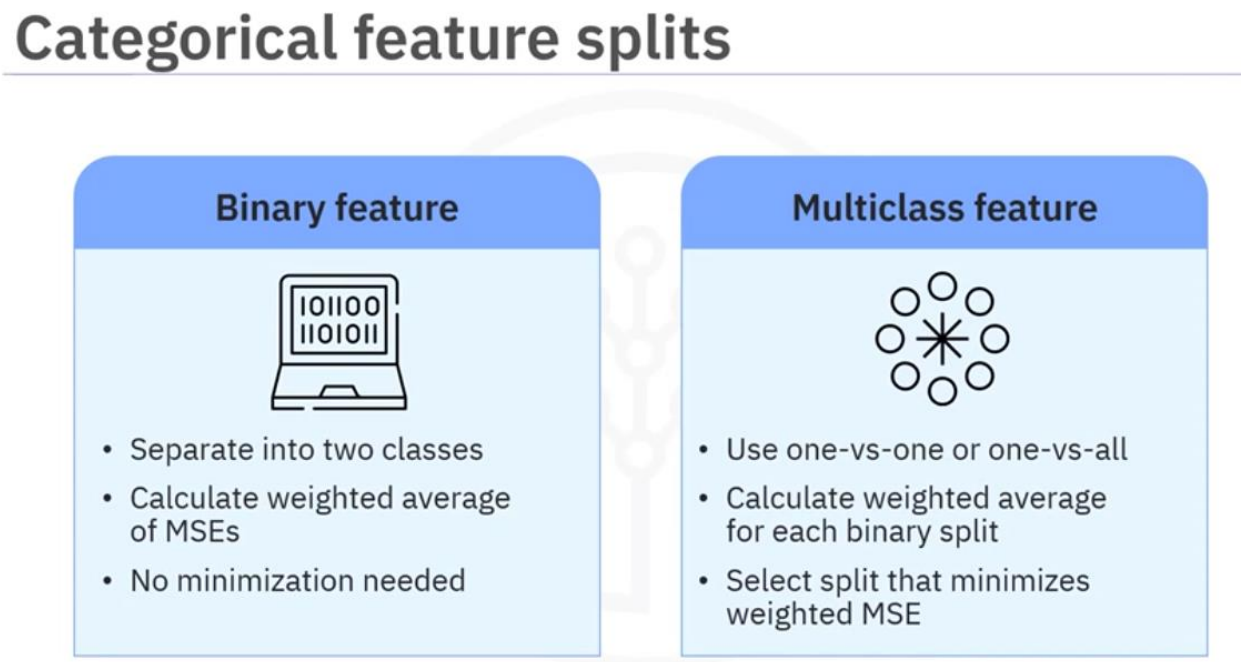
Choosing the best split



During training, the **tree finds the feature and threshold that best splits each node**. **For each potential split of a feature**, the tree **calculates the MSE for the left and right**

subsets. The MSE of the split is a weighted average of the MSEs of the subsets. **The split with the lowest-weighted MSE is chosen**.

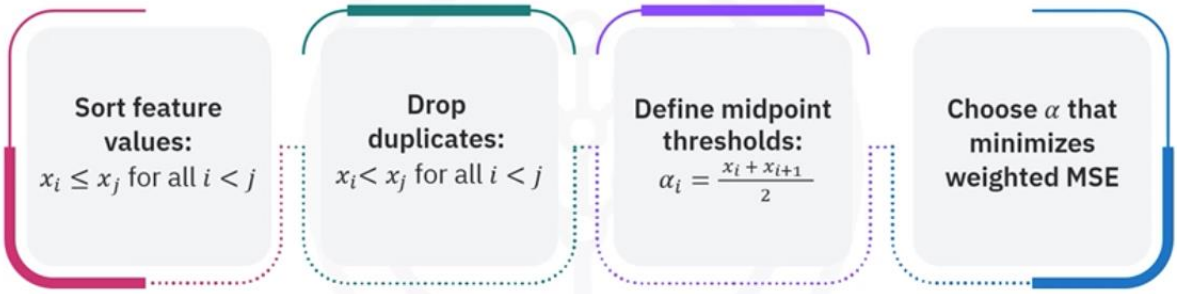
This process minimizes the variance in the predicted values and improves the accuracy of the regression tree.



For **a binary feature**, instead of using thresholds, the **data** is simply **separated** into its **two classes**, and the split quality is just the weighted average of the class MSEs. The weighted MSE has only one possible result, so **it is already optimized**.

For a **multi-class feature**, you can use a strategy like **one-versus-one or one-versus-all** to generate a set of possible binary splits. Then, for **each binary split, calculate the weighted average of the MSEs. Select the split that minimizes the weighted MSE**, and thus, the lowest prediction variance.

Continuous feature trial thresholds



How can you choose a set of trial thresholds to split a continuous feature on? There are many ways. Here's one strategy.

Start by **sorting the feature's values** so that Xi is less than or equal to Xj, for all indexes i less than j.

Drop any duplicated values so that Xi is strictly less than Xj, for all i less than j.

Define your candidate thresholds, αi as the **midpoints between each pair** of consecutive values, αi is half of Xi, plus Xi plus 1.

Choose the threshold that minimizes the weighted MSE for its data split.

This is an exhaustive search method that **doesn't scale well to big data**.

For very large datasets, selecting a **sparse subset of these thresholds can improve efficiency at the cost of accuracy**.

The method also assumes the **target values, X, are uniformly distributed**.

For efficiency, you should **consider the distribution when sampling** the thresholds.

Recap

Regression tree is analogous to a decision tree that **predicts continuous values**.

In **classification**, the target variable is **categorical**, and in **regression**, the target is a **continuous** value.

Regression trees are created by **recursively splitting the dataset into subsets to maximize information gained from data splitting**.

MSE is a natural criterion for measuring the split quality of a given feature.

The regression tree **finds** the **feature and threshold** that best splits each node during **training**.

The feature can be binary or multi-class.

Finally, you learned that you can choose continuous feature trial thresholds in multiple ways depending on data size.

App Item: Lab: Regression Trees

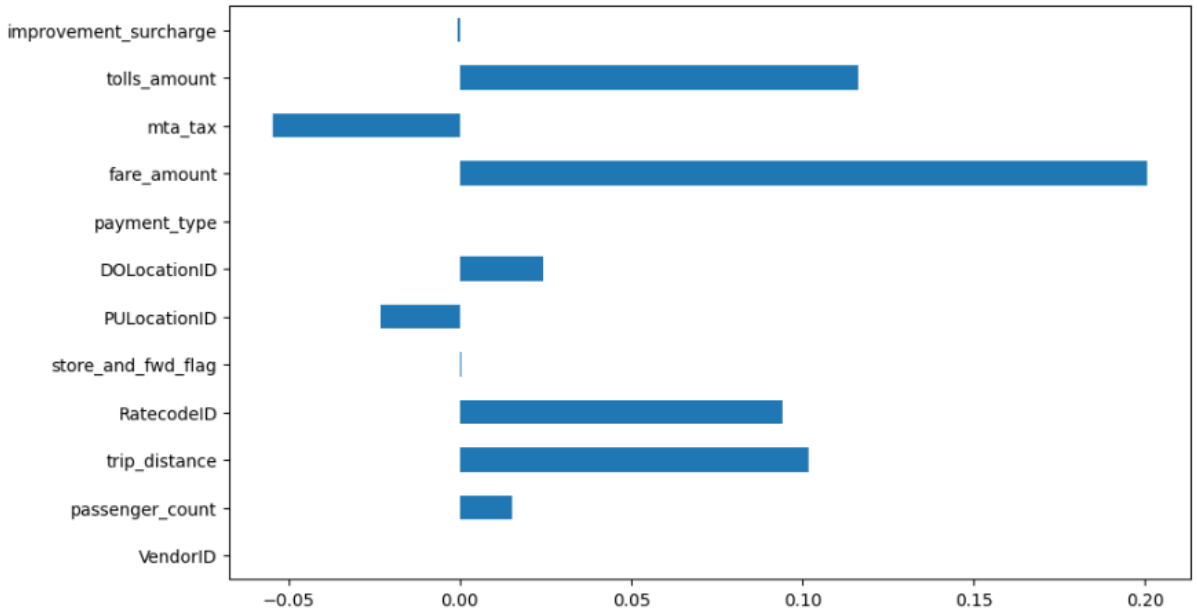
Taxi tips data set with 12 numerical features and one continuous target variable and 41000+ facts.

Field Name	**Description**
VendorID	Identifier for the TPEP/LPEP provider
tpep_pickup_datetime	Timestamp when the meter was activated
tpep_dropoff_datetime	Timestamp when the meter was turned off
passenger_count	Number of passengers
trip_distance	Distance of the trip in miles
RateCodeID	Rate code used for the trip
store_and_fwd_flag	Whether the trip record was stored before transmission

Field Name	Description
PULocationID	Pickup location ID (TLC Taxi Zone)
DOLocationID	Drop-off location ID (TLC Taxi Zone)
payment_type	Payment method (e.g., credit card, cash)
fare_amount	Metered fare based on time and distance
extra	Additional charges (e.g., rush hour, overnight)
mta_tax	$0.50 tax added to all trips
improvement_surcharge	$0.30 surcharge for accessibility funding
tip_amount	Tip paid via credit card (cash tips not recorded)
tolls_amount	Total tolls paid
total_amount	Total charge to passenger (excluding cash tips)

```
correlation_values = raw_data.corr()['tip_amount'].drop('tip_amount')
correlation_values.plot(kind='barh', figsize=(10, 6))
```

To see which features contribute more to the target variable, calculates correlation coeficients.



Remark dropping the target variable here.

```
# drop the target variable from the feature matrix
proc_data = raw_data.drop(['tip_amount'], axis=1)
# get the feature matrix used for training
X = proc_data.values
# normalize the feature matrix
X = normalize(X, axis=1, norm='l1', copy=False)
```

.values is a DataFrame attribute: Accessing the .values attribute of a pandas DataFrame or Series returns the **raw data** in the form of a **NumPy ndarray** (N-dimensional array).

◆ `axis=1`

Specifies **which axis** to normalize along.

axis=1 means **normalize each row** (i.e., each sample).

If you used axis=0, it would normalize each column (i.e., each feature).

◆ **`norm='l1'`**

Defines the **type of norm** used for normalization.

'l1' means each row will be scaled so that the **sum of absolute values** equals 1.

Other options include:

'l2': scales so that the **Euclidean norm** (square root of sum of squares) equals 1.

'max': scales so that the **maximum absolute value** in each row equals 1.

Cuando la profundidad aumenta se produce overfitting y el error aumenta. Incluso el

```
Depth :  12 =>MSE score : 26.459R^2 score : -0.047
Depth :  11 =>MSE score : 26.245R^2 score : -0.039
Depth :  10 =>MSE score : 25.937R^2 score : -0.027
Depth :   9 =>MSE score : 25.452R^2 score : -0.008
Depth :   8 =>MSE score : 24.555R^2 score : 0.028
Depth :   7 =>MSE score : 24.818R^2 score : 0.017
Depth :   6 =>MSE score : 24.563R^2 score : 0.028
Depth :   5 =>MSE score : 24.464R^2 score : 0.031
Depth :   4 =>MSE score : 24.412R^2 score : 0.034
```

Graded Assignment: Practice Quiz: Classification and Regression

Question 1 What type of machine learning method is classification?

Semi-supervised

Unsupervised

Reinforcement

Supervised

Question 2 What is the purpose of using a one-versus-all classification strategy?

To classify data into one or more classes

To extend binary classifiers to handle multiple classes

To improve the performance of neural networks

To handle binary classification

Question 3 In a decision tree, what does each leaf node represent?

A class label

A feature of the data

The result of a test

The split criterion used at that level

Question 4 Why might you want to prune a decision tree?

To maximize the number of features used in the tree

To ensure the tree has no leaf nodes

To increase the complexity of the model

To avoid overfitting the training data

Question 5 What is the primary goal of a regression tree?

To split data based on information gain and entropy

To minimize the number of leaf nodes

To predict continuous values based on features

To classify data into discrete categories

Supervised Learning with SVMs

Description

Support Vector Machines, or SVMs, are used for **classification**.

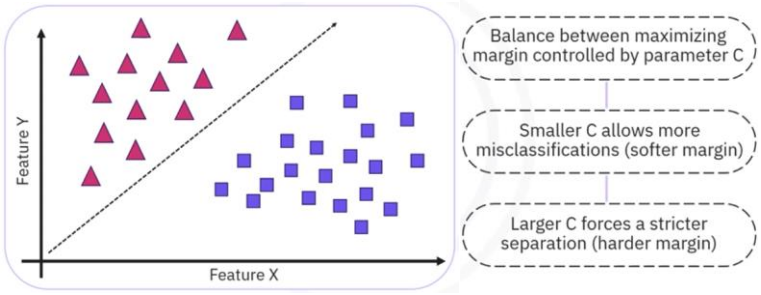
Support Vector Machines, or SVM, is a **supervised learning** technique for building classification and regression models.

What are support vector machines?

- Supervised learning technique
- Used for building classification and regression models
- Classifies input by identifying hyperplane

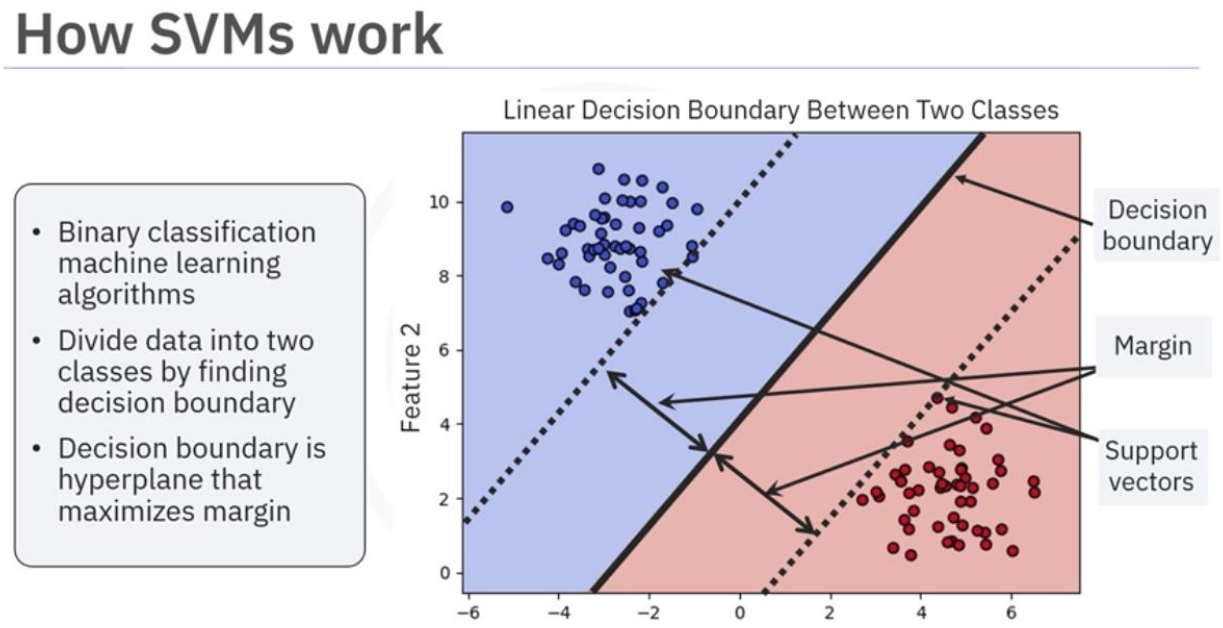
It maps each data instance as a **point in multidimensional space** where the input **features** are **represented as a value** for a specific coordinate. SVM classifies input data by identifying the **hyperplane**, which distinctly **differentiates two classes**. Thus, these points are fundamental to the dataset. In a classification task comprising two features, the hyperplane is a linear line that segregates and classifies the dataset.

Every time new inputs are added to the task, their classification depends on which side of the hyperplane they land.



The primary **goal** of SVM is to **create a hyperplane** that segregates a dataset into two parts and finds the largest margin. In this example, the dataset comprises a collection of two classes, red triangle and blue square. **The larger the margin, the better the model's accuracy** on new, unseen data. Data is often noisy and **overlapping** in real-world scenarios, **making perfect separation impossible**. SVM can incorporate a **soft margin**, which allows it to **tolerate misclassifications while maximizing the margin**. The balance between maximizing the margin and minimizing the number of misclassifications is controlled by a parameter, C. A **smaller C allows more misclassifications** – softer margin, while **a larger C forces a stricter separation** – harder margin.

Rudimentary **SVMs are binary classification machine** learning algorithms. Binary SVMs assume that two classes are linearly separable. However, **SVMs can also be adapted to solve regression problems**. SVMs try to divide data into two classes by **finding a decision boundary**. For example, this chart shows two classes as red and blue data points based on two features. The **decision boundary is a hyperplane that maximizes the ma**rgin. In a two-dimensional feature space, the decision boundary is a line.



The **margin is the distance from the hyperplane to the closest points from each class**. **These nearest-point representatives from each class are support vectors**.

Without getting into the details of the mathematics involved in the derivation of the optimization problem, consider this 2D example. Using the training data, and **assuming the data has been normalized**, the objective is to find a weight vector and a value b, called the bias term, such that

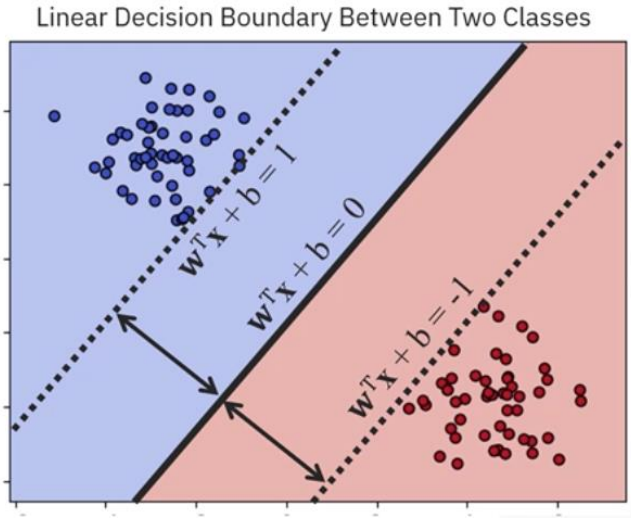
A: the **inner product of w** with itself is **minimized**, which amounts to minimizing the length of w.

B: For every observation, or data point x, and target value y, the product of y and w transported x plus b is greater than or equal to 1.

Therefore, the algorithm's output is the line's values, w and b.

You can make classifications using this estimated line. Adding input values into the line equation lets you calculate whether an unknown point is above or below the line.

SVM training and prediction



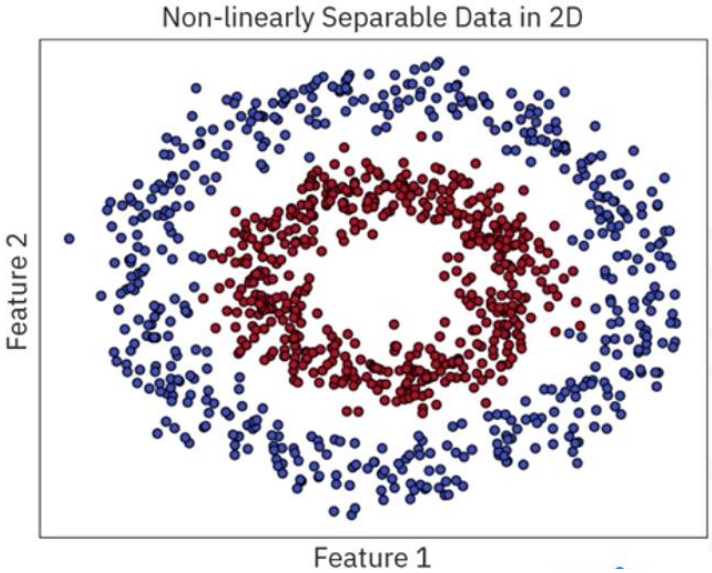
- 2D example
- Find a weight vector and value b, called the bias term

If equation > 0 :	Else:
• Point belongs to the blue class	• Point belongs to the red class

If the equation returns a value greater than 0, the point belongs to the first class, which is above the line, and vice versa.

Consider a 2D object of a **non-linearly separable pair of classes**. As you can see from the 2D chart, there are two non-overlapping classes with concentrically circular shapes. You can imagine these points as analogous to map contours, representing their heights. Clearly, these classes are not linearly separable.

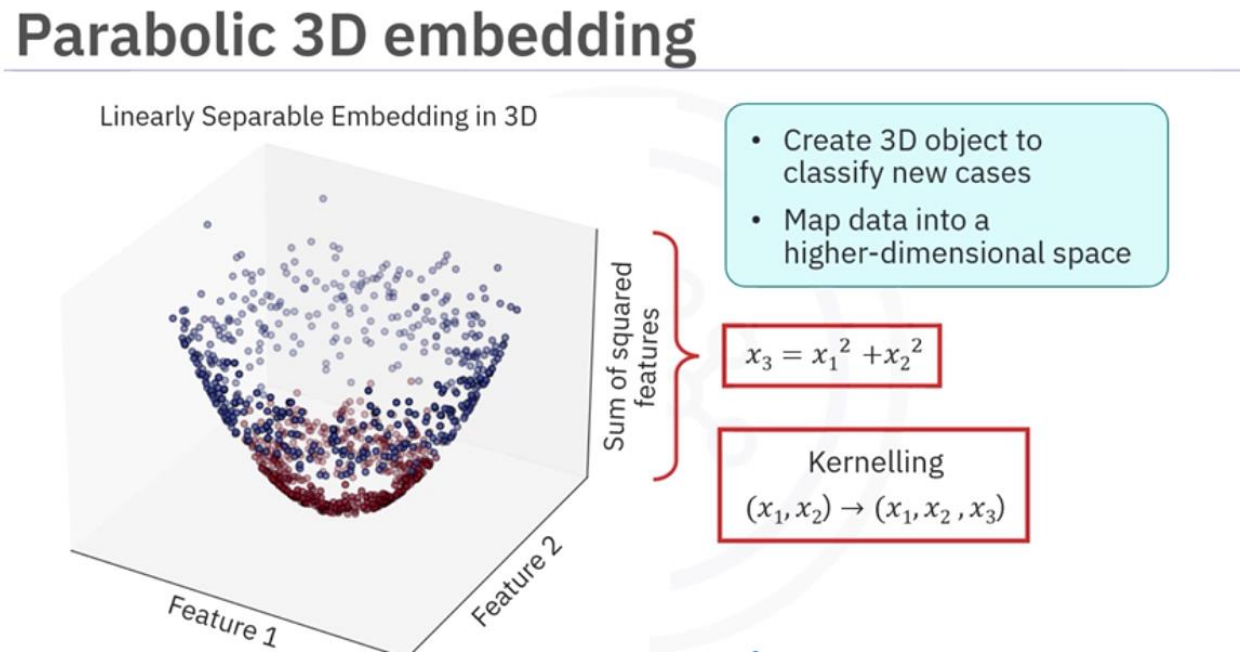
Nonlinearly separable classes



- 2D object
- Two non-overlapping classes
- Imagine these points as map contours

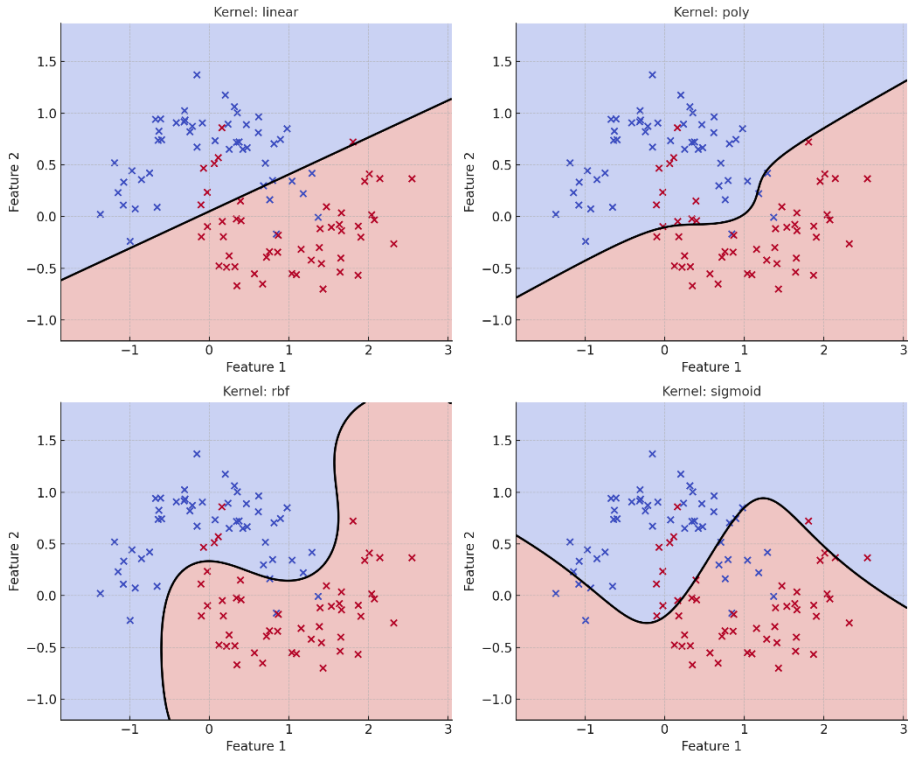
x_1 = Feature 1
x_2 = Feature 2

Let's transform the features of the 2D object so that it takes on a parabolic shape. Let's create a new 3D object by adding the new feature as the z-axis. It's a bit difficult to interpret 3D objects, but as you can imagine, the **two classes are now clearly separated by a horizontal plane**. You can use this plane to classify new cases according to whether they lie above or below the plane. **Mapping data into a higher-dimensional space like this is called kerneling**, where the **kernel is a quadratic polynomial in this case**.



With real-world data, there is no straightforward way to know which kernel function performs best. Scikit-learn provides you with a choice of **kernel functions to use with SVM**:

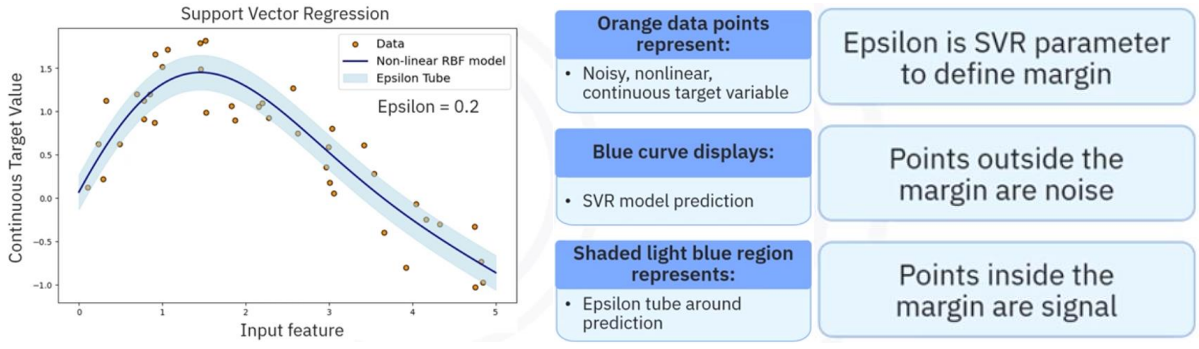
- A **linear** kernel is the default and corresponds to the usual SVM model.
- **Parabolic** embedding is implemented by choosing the polynomial option.
- **Radial basis functions**, or RBFs, which score high for points close to each other and an exponentially decreasing score as points become more distant.
- The **sigmoid** is the same function used for logistic regression.



SVR

Extension to regression

To see how support vector machines work for regression, consider this chart. It'll help you build some intuition without going into mathematics.



The synthetic data depicted by the orange data points represents a **noisy, non-linear, continuous target variable** as a function of an input feature. The blue curve displays the **support vector regression**, or **SVR** model prediction using a radial basis function, or **RBF kernel**. The shaded light blue region represents the **epsilon tube around the prediction**. Points falling within the epsilon tube are shaded yellow. **Epsilon** is a parameter of the SVR learning algorithm that you can select to **define a margin around the prediction curve**. Points falling **outside** the margin are interpreted as **noise**, and points **inside** as **signal**.

In the first case, epsilon is 0.2, and in the second, epsilon is 0.4.

SVM has many **advantages**.

- Effective in high-dimensional spaces.
- Robust to overfitting.
- Excels on linear separable data.
- Works with weakly separable data using weak margin option.

SVM also has some **limitations**.

- Slow training on large data sets.
- Sensitive to noise and overlapping classes
- Sensitive to the choice of kernel and regularization parameters, which are non-trivial to determine.